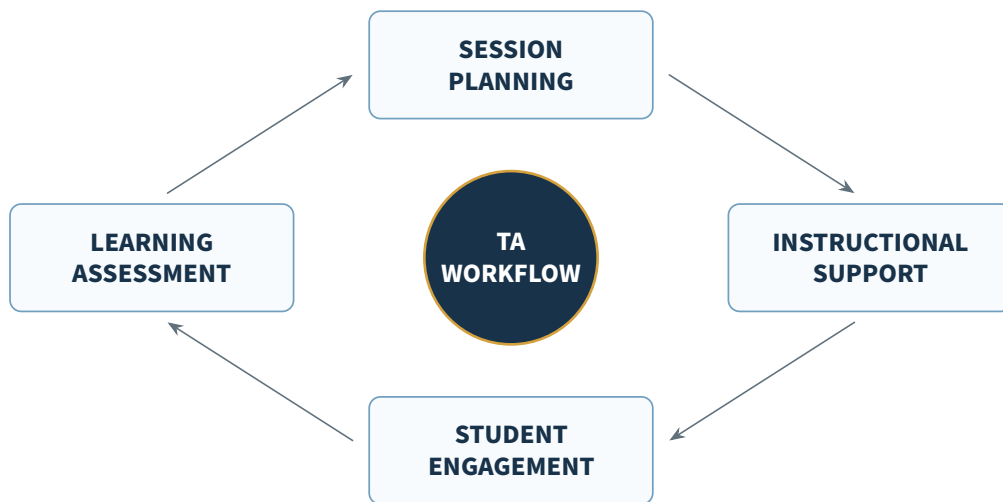


Machine Learning

Teaching Assistant Guide



COURSE OUTLINE

A step-by-step path through data preparation, shared mathematical foundations, supervised and unsupervised learning, and directions for further study.

PREPARED BY

Reeyan Afzal Khan

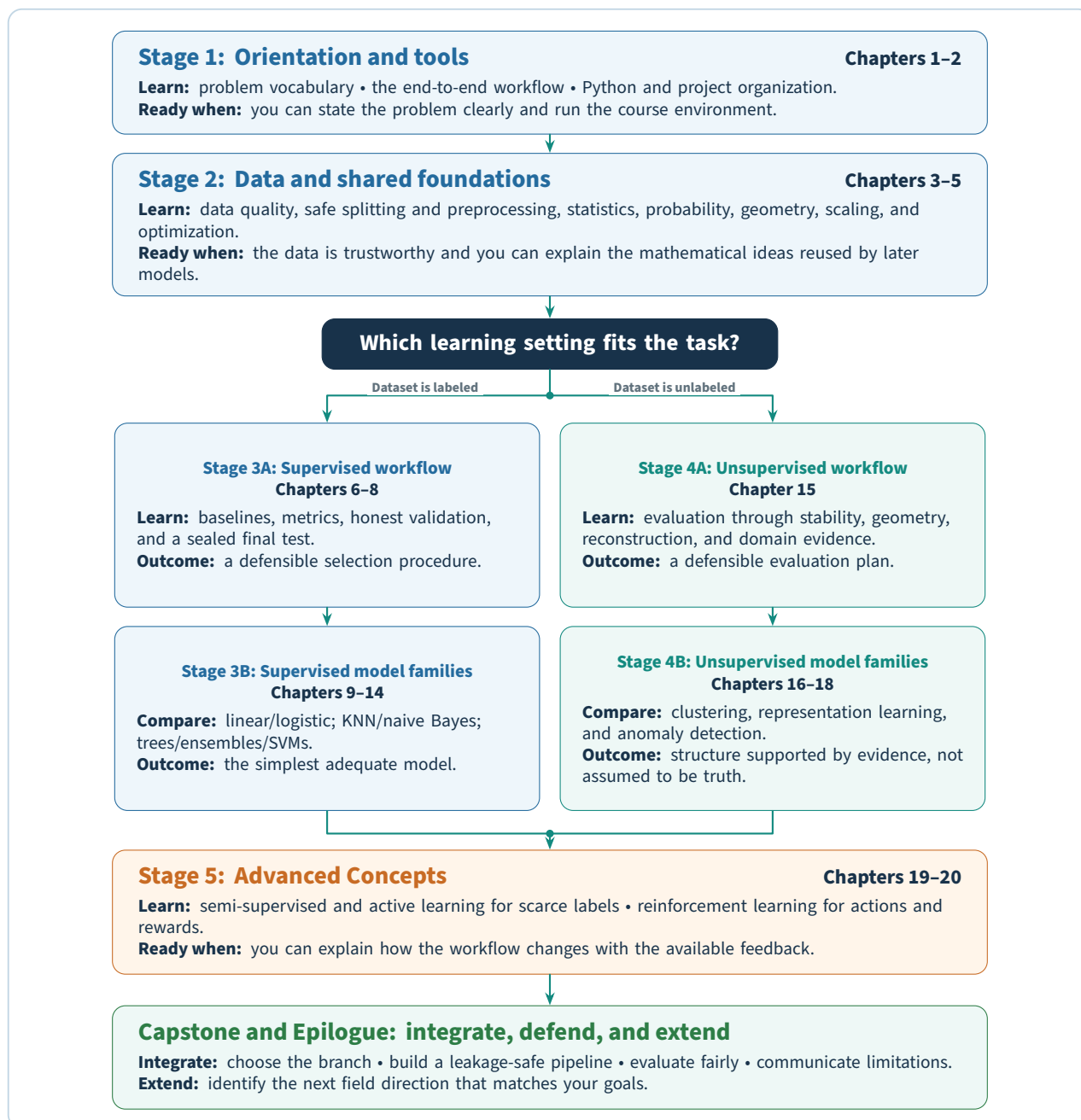
Graduate Research Assistant

Learning Path

Course Map: Learn One Branch at a Time	ii
Stage 1 Orientation and Tools	1
Chapter 1 Where to Begin	2
The Machine-Learning Map	7
Chapter 2 Your Python Toolkit and Environment Setup	11
Stage 2 Foundations Shared by Machine Learning	15
Chapter 3 Understanding and Preparing Data	16
Lab 1 of 10 — Environment and data foundations	26
Chapter 4 How Machine Learning Works	28
Chapter 5 Statistics, Geometry, and Similarity	32
Lab 2 of 10 — Learning loop, preprocessing, and geometry	43
Stage 3 Supervised Learning	46
Stage 3A — Supervised Workflow	47
Chapter 6 How Supervised Learning Works	48
Chapter 7 Evaluating Supervised Models	53
Chapter 8 Training, Generalization, and Model Selection	62
Lab 3 of 10 — Supervised workflow, evaluation, and model selection	71
Stage 3B — Supervised Model Families	76
Chapter 9 Linear Models for Regression	77
Lab 4 of 10 — Regression and time-aware validation	83
Chapter 10 Linear Models for Classification	86
Chapter 11 Nearest Neighbors and Naive Bayes	93
Lab 5 of 10 — Linear, neighbor, and probabilistic classification	98
Chapter 12 Decision Trees	101
Chapter 13 Ensemble Methods	107
Lab 6 of 10 — Decision trees and ensembles	114
Chapter 14 Support Vector Machines and Supervised Synthesis	118
Lab 7 of 10 — Support vector machines	123
Stage 4 Unsupervised Learning	125
Stage 4A — Unsupervised Workflow	126
Chapter 15 How Unsupervised Learning Works	127
Stage 4B — Unsupervised Model Families	131
Chapter 16 Clustering	132
Chapter 17 Dimensionality Reduction and Representation	140
Lab 8 of 10 — Unsupervised structure, clustering, and PCA	145
Chapter 18 Anomaly Detection	149
Stage 5 Advanced Concepts	153
Chapter 19 Semi-Supervised and Active Learning	154
Lab 9 of 10 — Anomaly detection and limited-label learning	158
Chapter 20 Reinforcement Learning Basics	162
Lab 10 of 10 — Reinforcement learning and final integration	168
Capstone Project	171
Epilogue: Where to Go Next	173

Course Map: Learn One Branch at a Time

Before the detailed Machine-Learning Map, use this one-page route to answer only one question: *what should I learn next?* The full field taxonomy appears after Chapter 1, once the basic vocabulary has been introduced.



Key idea

Do not memorize the map. Use it as a routing rule. First learn the workflow for a learning signal; only then compare model families within that workflow. The capstone asks you to choose and defend the route yourself.

Stage 1

Orientation and Tools

Where to Begin

Chapter at a glance

Before you start: No previous machine-learning knowledge is required.

By the end, you can: name the parts of a machine-learning problem, explain the full workflow, distinguish the main learning settings, and describe what a model must do to be useful.

Machine learning is a way of building computer systems that learn patterns from examples. Instead of writing one rule for every possible situation, we show the system data and let an algorithm find a useful rule.

Plain idea

Imagine teaching someone to estimate house prices. You could give them thousands of fixed rules, or you could show them many houses with their final prices and let them discover patterns. Machine learning follows the second approach.

1.1 The five parts of a machine-learning problem

Every beginner should be able to identify these five parts before choosing an algorithm.

Part	Meaning
Observation	One example or row, such as one house, patient, customer, or email.
Feature	An input used to make a prediction, such as floor area, age, or word count.
Target	The answer the model should learn to predict, such as price or spam/not spam.
Model	A mathematical rule that turns features into a prediction.
Metric	A number used to judge whether the model is useful on unseen data.

House	Area	Bedrooms	Distance to center	Sale price
A	90 m ²	2	8 km	\$220,000
B	140 m ²	3	4 km	\$360,000
C	110 m ²	2	12 km	\$245,000

Picture this table whenever the vocabulary feels abstract. Each house is an **observation**. The three middle columns are **features** known before the sale. The final column is the **target**. Training means using many completed rows to learn a rule; prediction means applying that rule to a new row whose sale-price cell is still blank.

Worked example

Suppose each row describes one house. Floor area, number of bedrooms, and distance to the city center are features. Sale price is the target. A regression model learns a rule that maps the three features to an estimated price. Mean absolute error could be the metric.

1.2 Parameters, hyperparameters, loss, and metrics

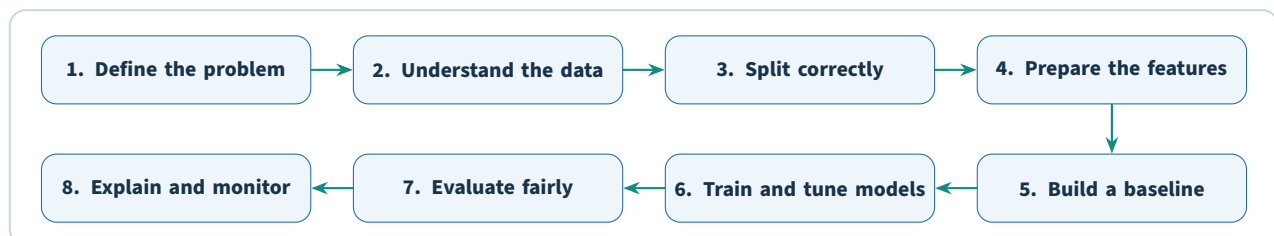
These terms sound similar but have different jobs.

Term	Who chooses it?	Example
Parameter	Learned from training data	A regression coefficient or a tree split
Hyperparameter	Chosen before or during model selection	Number of neighbors, tree depth, regularization strength
Loss	Used by the training process to measure wrongness	Squared error, cross-entropy
Metric	Used to report practical performance	Accuracy, recall, MAE, RMSE

A loss and a metric can be the same, but they do not have to be. Training may use a smooth mathematical loss while the final report uses a metric that matches the real-world goal.

1.3 The complete workflow

A reliable project follows a repeatable sequence.



Step 1: Define the decision. State what must be predicted, who will use the result, and what a wrong prediction costs.

Step 2: Understand and validate the data. Identify rows, keys, features, targets, units, missing values, timing, and possible bias. Apply only fixed, target-blind corrections at this stage.

Step 3: Split the data. Create training, validation, and test data in a way that matches future use.

Step 4: Prepare the features. Learn imputation, category encoding, scaling, and other data-dependent transformations from training data inside a pipeline.

Step 5: Build a baseline. Start with a simple rule, such as predicting the average or most common class.

Step 6: Train and tune. Fit candidate models on training data and choose settings with validation data.

Step 7: Evaluate once on the test set. Use the untouched test data for the final estimate.

Step 8: Explain, deploy, and monitor. Document limitations and check whether the data or performance changes over time.

The two information-boundary rules in this workflow are developed once in Section 3.7 and Section 3.5:

learned preprocessing uses training data, and the final test set stays outside model selection.

1.4 The main learning settings

Learning setting	How it works
Supervised learning	Each training row includes the correct target. Use it for regression and classification.
Unsupervised learning	No target is supplied. Use it to find groups, simpler structure, unusual cases, or item relationships.
Semi-supervised learning	A small set is labeled and a larger set is unlabeled. The unlabeled data may help when labels are expensive.
Reinforcement learning	An agent chooses actions and learns from rewards received over time.

Supervised and unsupervised learning form the core of this guide. Semi-supervised and reinforcement learning are introduced later using classical methods only.

Plain idea

Two related terms appear often in modern machine learning. **Self-supervised learning** creates a target from the data itself — for example, predicting a hidden word or missing image region — and is widely used to learn reusable representations before a downstream task. **Active learning** is a limited-label strategy in which the learner chooses which unlabeled examples should be sent to a human for labeling. The expanded field map before Chapter 1 shows where both fit.

1.5 Four learning settings, side by side

The four settings above are easier to remember as a picture. What changes between them is simply *what feedback the computer receives*.

Supervised	Semi-supervised	Unsupervised	Reinforcement
You are given the answer for every example. “ This email is spam. ” Goal: copy the mapping	You are given answers for only some examples. “ A few emails are labeled. ” Goal: learn from both sets	You are given no answers at all, only the examples. “ Here are 10 000 emails. ” Goal: find the structure	You are given a score after you act. “ That move lost you 3 points. ” Goal: act better next time

Analogy

Supervised learning is studying with the answer key beside you. Semi-supervised learning is having an answer key for only a few questions and using those answers to make sense of the rest. Unsupervised learning is being handed a shuffled box of photographs and asked to sort them into piles that make sense — nobody tells you what the piles should be. Reinforcement learning is learning a video game: no one explains the rules, but the score goes up or down after every move.

1.6 Generalization: the real goal

A model **generalizes** when it performs well on new examples from the same kind of situation. Training performance alone cannot prove generalization because a model may simply memorize details in the training set.

Plain idea

A student who memorizes the exact answers in a practice booklet may score perfectly during practice and still fail a new exam. A useful model learns the pattern behind the examples, not just the examples themselves.

Three outcomes are common:

Outcome	Model behavior
Underfitting	The model is too simple and performs poorly on both training and unseen data.
Good fit	The model captures useful structure and performs similarly on training and unseen data.
Overfitting	The model performs very well on training data but poorly on unseen data.

1.7 What a wrong prediction costs

Before choosing any algorithm, write down what happens when the model is wrong. The two directions of error are almost never equally bad.

Problem	Cost of a false alarm	Cost of a miss
Spam filter	A real email is hidden — irritating and occasionally serious	A spam email arrives — mildly annoying
Disease screening	An unnecessary follow-up test — costly and stressful	An untreated illness — potentially fatal
Fraud detection	A legitimate card is blocked — an angry customer	Money is lost — direct financial damage
Movie recommendation	A poor suggestion — ignored	A good movie is not shown — an invisible loss

Plain idea

This table is why accuracy alone can be a weak metric, and why Chapter 7 develops precision, recall, ranking, and thresholds. The algorithm cannot know that a missed cancer matters more than a wasted blood test. The evaluation objective must encode that judgment.

Try it yourself

Pick something a computer decides about you: which movies you are shown, whether a message goes to junk mail, or whether a bank approves a payment. Write down the five parts — observation, feature, target, model, and metric — and then the cost of each direction of error.

1.8 What to learn first

Follow the book's learning path rather than trying to memorize the entire field map at once:

1. Learn the vocabulary and the end-to-end workflow in this chapter.
2. Understand the software-toolkit roles and computational vocabulary in Chapter 2.
3. Assess data quality and prepare data safely in Chapter 3.

4. Use Chapter 4 to choose the learning branch from the feedback available.
5. Learn the shared statistical and geometric vocabulary in Chapter 5, focusing first on the concepts immediately reused by later models.
6. Complete Stage 3A first: supervised workflow, evaluation, and model selection; then Stage 3B changes one supervised model family at a time.
7. Reset the objective in Stage 4A and learn unsupervised evaluation; then Stage 4B studies clustering, representation, and anomaly detection.
8. Finish with limited-label and reward-based learning in Stage 5, then use the Epilogue to choose a modern direction.

Key idea

A machine-learning project is not only an algorithm. It is a complete process that begins with a real problem and ends with a fair evaluation on unseen data.

Check your understanding

1. What are the observation, features, target, model, and metric in a problem that predicts whether a loan payment will be late?
2. Why is a learned model parameter different from a hyperparameter chosen during model selection?
3. Why can excellent training performance still be evidence of a poor machine-learning system?

Chapter recap

- A dataset contains observations, features, and sometimes a target.
- Parameters are learned; hyperparameters are chosen and validated.
- The standard workflow is problem definition, data preparation, baseline, training, validation, final testing, and monitoring.
- The real objective is generalization, not perfect training performance.

The Machine-Learning Map

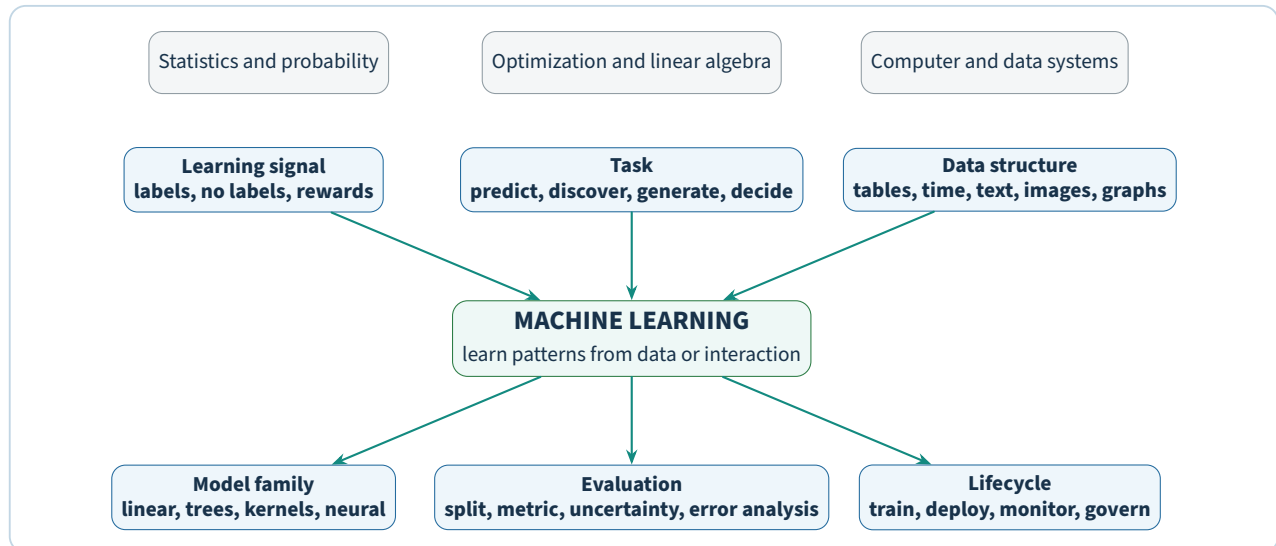
Machine learning is easier to navigate when you stop treating it as one list of algorithms. The same project can be described in several ways at once: by the *feedback* available, the *job* to be done, the *kind of data*, the *model family*, and the *stage of the lifecycle*. Use the maps on the next pages in that order.

Key idea

Do not begin with “Which algorithm should I use?” Begin with four questions: **What decision or output do I need? What feedback is available? What structure does the data have? How will success be measured on future data?**

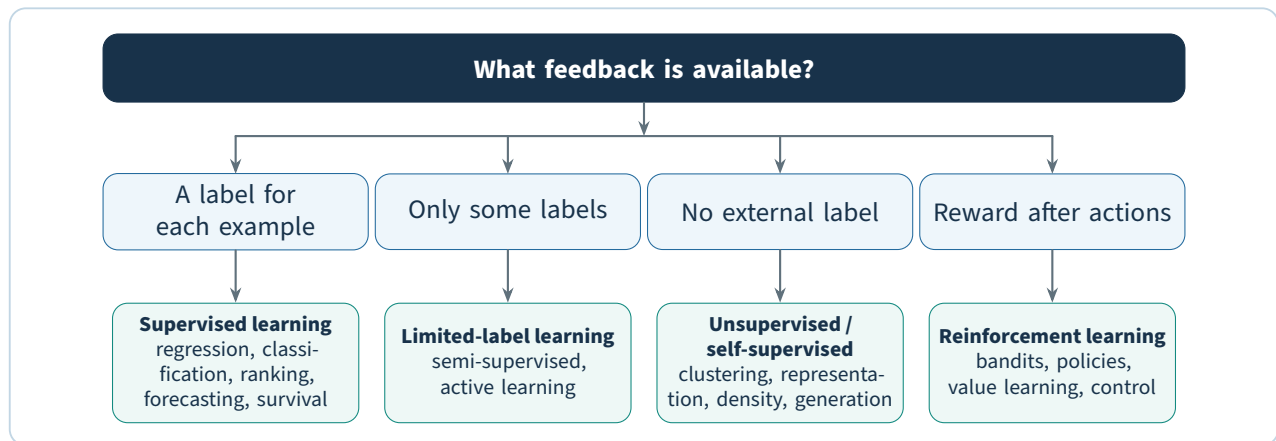
1. The whole picture

Machine learning is a major approach within artificial intelligence, but it also overlaps heavily with statistics, optimization, scientific computing, and data systems. The boundaries are not clean, so the useful map is a set of connected views rather than one perfect tree.



View	Main branches
Feedback	Full labels; limited labels; no external labels; rewards from interaction.
Task	Prediction, ranking, forecasting, structure discovery, representation, generation, anomaly detection, or sequential decision-making.
Data	Tabular, temporal, text, image/video, audio, graph/spatial, or multimodal.
Model family	Linear/probabilistic, neighbors, trees/ensembles, kernels, neural networks, generative models, graph models.
Evaluation	Generalization, realistic splits, baselines, task metrics, calibration, uncertainty, subgroup and failure analysis.
Lifecycle	Problem framing, data collection, preparation, training, evaluation, deployment, monitoring, maintenance, governance.

2. Start from the feedback you actually have



Plain idea

Self-supervised learning creates a training signal from the data itself — for example, predicting a hidden word or a missing part of an image. It is one of the main bridges from unlabeled data to modern representation learning. **Active learning** is different: the model chooses which examples would be most valuable for a human to label.

3. Start from the job you need done

Need	First framing	Where it lives
Predict a number	Regression	Core: Chapter 9
Predict a category	Classification	Core: Chapter 10
Predict an ordered category	Ordinal classification / regression	Introduced in Chapter 10
Rank a finite set of items	Ranking / recommendation	Beyond the core; evaluate at the operational cutoff
Predict later values	Forecasting / temporal prediction	Uses supervised learning with chronological validation
Estimate time until an event	Survival / event-time modeling	Beyond the core; censoring changes the evaluation problem
Find groups	Clustering	Core: Chapter 15, then Chapter 16
Compress or build a representation	PCA, embeddings, representation learning	Core PCA: Chapter 17; modern embeddings are beyond
Find unusual cases	Anomaly detection	Core: Chapter 18
Model a full distribution or generate samples	Density estimation / generative modeling	GMM is introduced in Chapter 16; modern generative models are beyond
Choose actions over time	Bandits / reinforcement learning	Extension: Chapter 20
Estimate what an intervention will change	Causal inference / experimentation	Adjacent to prediction; introduced as a direction in the Epilogue

Key idea

Prediction asks “what is likely to happen?” Causal inference asks “what would change if we acted differently?” A highly predictive model does not automatically answer the second question.

4. Start from the data structure

The input representation often narrows the sensible model families before tuning begins.

Data	Strong classical starting point	Modern directions and the main caution
Tabular rows	Linear/logistic models, trees, random forests, gradient boosting	Deep tabular models can help, but strong tree baselines remain essential.
Time series / events	Lagged features plus linear or tree models	Sequence models and transformers; preserve chronology, horizon, and label availability.
Text	Counts or TF-IDF with naive Bayes, logistic regression, or linear SVM	Embeddings, transformers, foundation language models; evaluate meaning, not fluency alone.
Images / video	Hand-designed features, PCA, linear/SVM models	CNNs, vision transformers, diffusion models; transfer learning is usually the practical route.
Audio / speech	Spectral features plus classical classifiers	Neural audio encoders and transformers; sampling and temporal alignment matter.
Graphs / networks	Engineered node, edge, and graph statistics	Graph neural networks; random row splits can leak through connected structure.
Spatial data	Domain features plus linear/tree models	Spatial statistics and geospatial deep learning; nearby observations may not be independent.
Multimodal	Separate feature pipelines combined carefully	Multimodal representation and foundation models; alignment between modalities is part of the problem.

5. Model families are reusable building blocks

A model family is *not* the same thing as a task. Trees, kernels, and neural networks can all be adapted to several prediction problems.

Family	Core idea	Examples in or beyond these notes
Baselines	Simple reference rule	Mean, median, majority, random, or domain rule
Linear / GLMs	Weighted feature combinations	Linear, ridge, lasso, logistic, softmax
Probabilistic	Model probability distributions	Naive Bayes, LDA, GMM; Bayesian models beyond the core
Neighbors	Use nearby cases	k -NN, local outlier methods, density neighborhoods
Trees / ensembles	Partition space; combine learners	CART, random forests, boosting, stacking
Kernels / margins	Separate in a transformed space	SVM, SVR, one-class SVM
Neural networks	Learn features and predictions together	MLP, CNN, transformers, graph neural networks
Generative / foundation	Learn distributions or reusable representations	Autoencoders, diffusion, language and multimodal models

Key idea

A good workflow climbs a ladder: **baseline** → **simple model** → **stronger model** → **tuned system**.
Stop when added complexity no longer improves the real decision enough to justify its cost.

6. Good first models for the core problems

Problem	Start with	Then compare with
Regression	Mean baseline, linear regression	Ridge/lasso, trees, random forest, gradient boosting, support vector regression
Binary classification	Majority baseline, logistic regression	k -NN, naive Bayes, trees, random forest, boosting, SVM
Multiclass classification	Majority baseline, softmax regression	Trees, random forest, boosting, k -NN, SVM
Clustering	K -means after scaling	Hierarchical clustering, DBSCAN, Gaussian mixtures
Dimensionality reduction	PCA	t-SNE or UMAP for exploratory visualization
Anomaly detection	Simple statistical rules	Local methods, isolation forest, one-class models

7. What these notes cover — and what lies beyond

Scope	Topics
Core in these notes	Problem framing and leakage-safe preparation; baselines and evaluation; classical supervised and unsupervised methods.
Introduced / bridged	Limited-label and reinforcement learning; image features and transfer learning; deployment, drift, and responsibility.
Beyond the core	Deep, foundation, generative, graph, Bayesian, causal, online, federated, privacy-preserving, and production ML.

Your Python Toolkit and Environment Setup

Chapter at a glance

Before you start: You should be comfortable creating folders and running a command in a terminal.
By the end, you can: Create a clean project workspace, manually download the course datasets into dedicated dataset folders, create and activate a virtual environment, install the project dependencies from the included requirements file, and save your practical work as Jupyter notebooks in `all_experiments/`.

Machine learning combines ideas with computation. From this chapter onward, major methods are paired with complete Python examples and small public Kaggle datasets. You will still type, run, modify, and explain the code yourself. The objective of the setup is not automation; it is to make every path predictable enough that the machine-learning ideas stay in focus.

2.1 The course project layout

Use one root folder for the course. Inside it, keep *datasets* and *experiments* separate:

```
UTP_MACHINE_LEARNING/  
  all_datasets/                # one subfolder per dataset  
    titanic_dataset/  
    medical_cost_personal_dataset/  
    breast_cancer_dataset/  
    ames_housing_dataset/  
    mall_customers_dataset/  
    optical_recognition_dataset/  
    credit_card_fraud_dataset/  
    student_performance_dataset/  
    heart_failure_dataset/  
    red_wine_quality_dataset/  
    bike_sharing_dataset/  
  
  all_experiments/            # one dedicated folder for each lab  
    lab01/  
    lab02/  
    ...  
    lab10/  
  
requirements.txt
```

Key idea

`all_datasets/` is only the dataset *root*. Do not throw every CSV into that root. Each downloaded dataset gets its own folder because a Kaggle download may contain one file, several related files, a README, or supporting metadata. Your code should therefore use paths such as `all_datasets/titanic_dataset/Titanic-Dataset.csv`.

2.2 Download the course datasets manually from Kaggle

Open the dataset page by clicking on the name of the dataset in the table below, download the archive, extract it, and place the extracted files in the dedicated folder.

2.2.1 Core datasets reused across several chapters

Kaggle dataset	Main use in these notes
Titanic Dataset	data quality assessment, baselines, model selection, logistic regression, trees
Medical Cost Personal Data	first pipeline, linear and ridge regression
Breast Cancer Wisconsin	geometry, evaluation, KNN, naive Bayes, SVM, limited labels
Ames Housing	random forests and gradient boosting
Mall Customers	unsupervised workflow, K-means, DBSCAN
Optical Recognition of Handwritten Digits	PCA and representation learning
Credit Card Fraud Detection	anomaly detection and imbalanced evaluation
Student Performance	unfamiliar-data assessment, delimiter/schema comparison, stage checkpoint
Heart Failure Prediction	classification evaluation on a new mixed-type schema
Red Wine Quality	ensemble transfer exercise
Bike Sharing	chronological splitting, leakage checks, regression transfer lab

2.2.2 One download, step by step

Use the same procedure for every dataset:

Step 1: Open the Kaggle Dataset link in the table.

Step 2: Click **Download**. If Kaggle gives you a ZIP file, extract it normally using your operating system.

Step 3: Create the dedicated course folder named in the second column if it does not already exist.

Step 4: Copy the required file or files into that dedicated folder. Keep additional documentation from the download there too if it helps you understand the data.

Step 5: Confirm that the path in your notebook exactly matches the folder name and filename, including capitalization.

Common mistake

A path error is not a machine-learning error. If `pd.read_csv(...)` raises `FileNotFoundError`, first print or inspect your working directory, then compare the actual folder and filename with the path in the notebook. Do not “fix” the problem by moving random files into the project root.

2.3 Install Python and make it available in the terminal

Download a currently supported release from the official Python downloads page.

Windows Run the official installer, install for your account, and allow it to add Python to PATH. Reopen the terminal, then use `py`.

macOS Run the signed `.pkg` installer with the default options, then restart Terminal. Do not remove the system Python in `/usr/bin`.

Linux Install Python 3 through your distribution’s package manager. It normally adds `python3` to PATH; do not replace the operating system’s Python.

If Windows does not recognize `py`, add Python's command directory to your system PATH so PowerShell and Command Prompt can find it. Open the Start menu, search for *Edit environment variables for your account*, select Path, click *Edit*, and add the directory shown by the Python installer. With the current install manager, this directory is normally:

```
%LocalAppData%\Python\bin
```

On macOS, the installer should configure the path automatically. If `which -a python3` does not show `/usr/local/bin/python3`, make sure `/usr/local/bin` is in the path used by the default Z shell:

```
echo 'export PATH="/usr/local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For Debian or Ubuntu:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
```

Verify the installation and see which executable the terminal found:

```
# Windows
py --version
py -m pip --version
where py

# macOS or Linux
python3 --version
python3 -m pip --version
which -a python3
```

2.4 Create an isolated virtual environment

Python's standard-library `venv` module creates a lightweight environment with its own installed packages. This keeps the course dependencies separate from other Python projects. Python `venv` documentation.

The repository includes a normal text file named `requirements.txt` in the project root. It contains the course dependencies:

```
numpy
pandas
matplotlib
scikit-learn
jupyter
```

If you installed standard Python, use the command for your operating system to create a virtual environment. The name `.venv` is a common convention, but you may choose a different name.

```
# Windows
py -m venv .venv

# macOS or Linux
python3 -m venv .venv
```

Then, while remaining in the project root, activate the virtual environment in PowerShell or Terminal:

```
# Windows
.venv\Scripts\Activate.ps1

# macOS/Linux
source .venv/bin/activate
```

After activation, upgrade pip and install the course requirements:


```
# Windows
py -m pip install --upgrade pip
py -m pip install -r requirements.txt

# macOS or Linux
python3 -m pip install --upgrade pip
python3 -m pip install -r requirements.txt
```

Package installation is normally a one-time setup. On later study sessions, activate `.venv` again; you do not need to reinstall the requirements each time.

Key idea

Do not install packages globally simply because a tutorial requests them. Activate the project environment first, then install the packages inside it.

2.5 Verify the scientific stack

Use the practical lab below as the primary environment check. It imports the core scientific packages and records their versions, so the notebook also serves as a reproducibility record. If all four imports succeed, the core environment is ready.

Check your understanding

1. Why should datasets, notebooks, and generated code listings live in separate folders?
2. Why is a project-local virtual environment safer than installing course packages globally?
3. If a notebook raises `FileNotFoundError`, what should you inspect before changing any data files?

Chapter recap

- `all_datasets/` is the dataset root; every Kaggle dataset lives inside its own dedicated sub-folder.
- A dataset folder may legitimately contain several related files, as with Optical Digits, Student Performance, and Bike Sharing.
- Install the included `requirements.txt` inside a project-local virtual environment.
- Save your practical work as notebooks in `all_experiments/`; a good notebook records code, plots, results, and interpretation.

Stage 2

Foundations Shared by Machine Learning

Understanding and Preparing Data

Chapter at a glance

Before you start: Chapters 1–2: the workflow, computational vocabulary, and the distinction between features and labels.

By the end, you can: assess and clean a dataset, distinguish deterministic corrections from learned preprocessing, choose a valid split, handle missing and categorical values, prevent leakage, and build a reproducible pipeline.

Now that the environment and basic library vocabulary are in place, the next job is to make the data trustworthy. A powerful algorithm cannot rescue a badly defined label, incorrect measurements, or an unfair train-test split.

Data preparation is easier when you separate three questions: *What does the data mean?* *What is demonstrably wrong?* *What must be learned from examples?* This chapter follows that order. It treats cleaning as a documented reasoning process, not as a collection of commands that make a table look tidy.

3.1 Start with a data quality assessment

Before changing the dataset, answer the following questions.

Step 1: What does one row represent? A person, transaction, image, day, machine, or something else?

Step 2: What does each column mean? Record its type, unit, valid range, and source.

Step 3: When is the label known? A feature available only after the outcome occurs creates leakage.

Step 4: Which rows are related? Repeated measurements from one person or device must usually stay in the same split.

Step 5: Who or what is missing? The dataset may not represent the full population where the model will be used.

3.2 A safe order of operations

The phrase “clean the data” hides several different jobs. Their order matters.



Stage	Examples	Why it belongs there
Before the split	Check required columns, parse known date formats, standardize documented units, normalize harmless whitespace, verify keys, and reject impossible values using domain rules.	These rules do not estimate anything from the label or from the distribution of the full dataset.
After the split	Learn medians, means, scales, frequent-category cutoffs, outlier thresholds, label-informed mean encodings, and any later feature-selection or resampling rules.	These choices depend on observed data and must be fitted on training rows only.
After modeling	Inspect test performance once, document limitations, and monitor future data quality and drift.	The test set remains an honest final check rather than another source of design decisions.

Key idea

A correction may happen before the split when it follows a fixed, label-blind rule that would be applied identically to any future row. If a value, cutoff, category grouping, or decision is estimated from the observed data, learn it from training data only.

3.3 Feature and label types

Type	Example	Typical treatment
Numerical	age, temperature, income	keep numeric; scale or transform when needed
Nominal category	country, product type	one-hot encode or use a model that handles categories
Ordinal category	low, medium, high	encode in the true order, but do not assume equal gaps without reason
Date or time	purchase date, hour	extract useful parts such as elapsed time, weekday, or season
Text	review, message	convert to numerical counts or other classical text features

Labels usually fall into two basic groups:

- A **continuous number** suggests regression, such as price or temperature.
- A **category** suggests classification, such as fraud/not fraud or animal species.

3.4 Clean structural problems before modeling

Structural cleaning makes the stored values agree with their documented meaning. It should be reproducible: another person using the same raw file and rules should produce the same cleaned file.

3.4.1 Confirm row identity and handle duplicates

Pattern	What it may mean	Safe response
Exact repeated row	Accidental repeated import, or a genuinely repeated event with too few identifying columns	Remove only after confirming that repetition has no meaning; record how many rows were removed.
Repeated key	Violation of the row definition, conflicting versions, or an expected one-to-many relationship	Investigate and define a rule: keep the latest version, aggregate, repair the key, or redesign the row definition.
Same person, different visits	Valid repeated measurements	Keep the rows, but use a group-aware split so one person cannot appear on both sides.
Near duplicate	Spelling, rounding, or record-linkage differences	Use an explicit matching rule and review uncertain cases; do not silently fuzzy-match everything.

3.4.2 Repair types, labels, dates, and units

Use the following order:

- Step 1: Preserve the raw value.** Keep the source file unchanged and create a cleaned copy or scripted transformation.
- Step 2: Parse explicitly.** Convert numbers and dates with known rules, and count values that fail to parse.
- Step 3: Normalize representation.** Trim accidental whitespace, standardize case where case has no meaning, and map documented synonyms to one label.
- Step 4: Normalize units.** Convert to one stated unit using a known conversion; never mix unit conversion with statistical scaling.
- Step 5: Validate the result.** Check allowed categories, ranges, key uniqueness, and date order with assertions.

3.4.3 Distinguish impossible values from unusual values

An **impossible value** violates a domain rule: a negative age, a discharge date before admission, or a percentage above 100. A **rare value** is surprising but possible: a very large purchase, an old patient, or a sensor spike during a genuine failure. These cases require different actions.

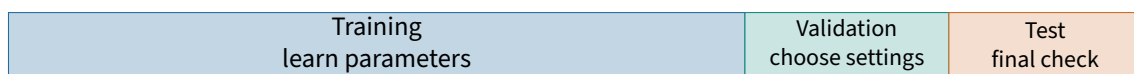
Action	When to use it
Correct	Only when an authoritative source or deterministic rule reveals the true value.
Set missing	When the stored value is known to be invalid but the true value is unknown. Impute later inside the training pipeline if appropriate.
Drop	When the row cannot answer the project question and a documented inclusion rule justifies removal. Report the count and check whether exclusions are concentrated in a group.
Flag	When a suspicious value may still be real. A binary quality flag lets the model or analyst retain that information.
Keep	When a value is valid, even if it is inconvenient. Rare cases may be exactly where a useful model is most needed.

Common mistake

Do not delete or cap values merely because they hurt the model's score. That uses the modeling result to redefine the data and can erase the difficult cases the model will meet after deployment.

3.4.4 Keep a cleaning log

For every rule, record the reason, affected columns, number of changed rows, and whether values were corrected, marked missing, excluded, or flagged. Retain row identifiers so changes can be traced without exposing them as model features. A short log makes mistakes reversible and lets future readers distinguish raw facts from project decisions.

3.5 Training, validation, and test sets

Training set Used to fit model parameters and data-dependent preprocessing steps.

Validation set Used to compare models, thresholds, and hyperparameters.

Test set Used only for the final estimate after all choices are complete.

Common proportions include 60–80% for training and the remainder divided between validation and test. The best proportions depend on dataset size; the logic of the separation matters more than a fixed percentage.

3.6 Choose a split that imitates the future

A random split is useful only when future rows are independent and drawn in the same way.

Stratified split Keeps class proportions similar across subsets.

Group split Keeps all rows from one person, device, school, or customer in one subset.

Time split Trains on the past and evaluates on later periods.

Spatial split Separates locations when nearby observations are strongly related.

Worked example

A medical dataset has five visits for every patient. Splitting rows randomly may place the same patient in both training and test data. The model can partly recognize that patient instead of learning a rule that works for new patients. A group split by patient is more honest.

Common mistake

The split is part of the model design. A high score from an unrealistic split is not evidence that the model will work in practice.

3.7 Data leakage

Data leakage occurs when information that would not be available at prediction time influences training. Leakage can enter through features, preprocessing, later model-selection steps such as feature selection, or

the split itself.

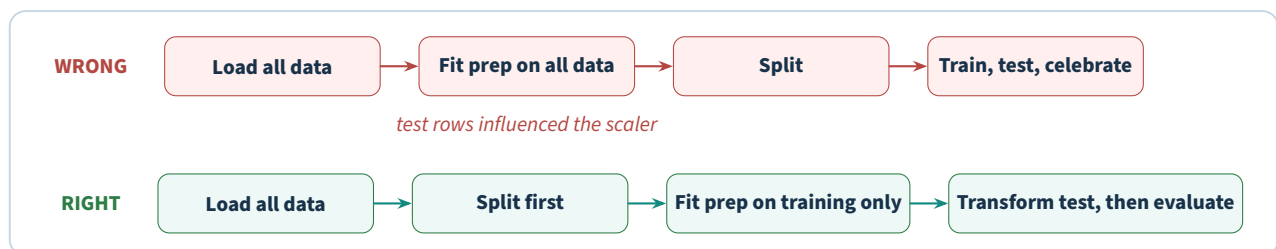
Examples include:

- predicting hospital readmission using a field recorded after discharge;
- calculating the mean for standardization using the entire dataset;
- selecting features using test-set performance;
- placing near-duplicate rows in both training and test data;
- using future values to predict an earlier point in time;
- including an identifier or proxy that effectively reveals the label, such as a claim-resolution code when predicting whether a claim will be approved.

The safe rule is simple: fit every data-dependent choice on training data only, then apply the learned transformation unchanged to validation and test data.

3.8 Leakage, drawn

Leakage is abstract until you see the order of operations. The only difference between the two pictures below is *when* the split happens — and that difference is the single most common cause of a model that looks excellent in testing and fails in reality.



Analogy

Leakage is revising for an exam using a copy of the actual exam paper. Your practice score will be superb and completely meaningless, because the score no longer measures what it claims to measure.

3.9 Missing values: meaning and strategy

Missing values do not all mean the same thing. A value may be missing because it was not measured, not applicable, too expensive to collect, or related to the outcome itself.

A practical process is:

- Step 1:** Measure how much is missing in each feature and group.
- Step 2:** Investigate why it is missing and whether the pattern carries information.
- Step 3:** Choose a treatment under the information boundary in Section 3.7.
- Step 4:** Add a missing-value indicator when the fact that a value is missing may matter.
- Step 5:** Check whether the treatment changes important distributions.

Common treatments are median imputation for numerical features, a dedicated “missing” category for categorical features, and model-based imputation when simple methods are not sufficient.

The labels **MCAR**, **MAR**, and **MNAR** describe increasingly demanding assumptions about why values are missing. They are useful vocabulary, but they are not diagnoses that can be read directly from a table. For a first project, record plausible causes, compare missingness across relevant groups, preserve informative indicators, and test the consequences of reasonable treatments.

3.10 Encoding categorical features

Most algorithms need numerical inputs. The encoding must preserve the meaning of the category.

Encoding method	How it works
One-hot encoding	Creates one binary column per category. It is a safe default for nominal features with a manageable number of values.
Ordinal encoding	Uses ordered numbers only when the categories truly have an order.
Frequency encoding	Replaces a category with how often it appears in training data.
Label-informed mean encoding	Uses label statistics; apply the leakage rule from Section 3.7.

A production pipeline should also know what to do with a category that did not appear during training. Mapping it to an “unknown” value is usually safer than causing an error.

3.11 Scaling and transforming numerical features

Scaling changes the measuring stick of a numerical feature without changing which row is larger or smaller. This matters whenever the algorithm uses distances, margins, coefficient penalties, or gradient steps. A feature measured in thousands can otherwise dominate another feature measured in single digits simply because of its units.

The three course scalers answer three different questions:

Method	Transformation	Best first use
Standard scaling	$z = (x - \mu) / \sigma$	General default for SVM, k -NN, clustering, and regularized linear models when mean and standard deviation are meaningful.
Min-max scaling	$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$	Put a feature into $[0, 1]$ when a bounded range is convenient and extreme values are not dominating the observed range.
Robust scaling	$x' = (x - \text{median}(x)) / \text{IQR}(x)$	Data with strong outliers, because the median and interquartile range are less affected by extreme observations.

A fitted scaler is data-dependent preprocessing, so its information boundary is exactly the one defined in Section 3.7.

Worked example

Suppose a video-view feature contains $[2400, 36, 23075, 70, 1, 905]$. Min-max scaling forces the smallest observed value to 0 and the largest to 1, so the extreme 23,075-view video stretches the entire range. Robust scaling instead centers on the median and divides by the middle 50% spread, so that one extreme view count has much less influence on the scale. The choice is not “which formula

is best?” but “which reference scale matches the geometry I want the model to use?”

Model family	Scale-sensitive?	Why
<i>k</i> -NN, <i>K</i> -means, DBSCAN	Usually yes	Their neighborhoods or clusters are defined by distance.
SVM / SVR	Usually yes	Margin and kernel distances depend directly on feature magnitude.
Ridge, lasso, elastic net, logistic regression	Usually yes	The penalty compares coefficient sizes across features; unequal units distort that comparison.
Decision trees and tree ensembles	Usually no	Splits depend on ordering and thresholds, which monotonic rescaling preserves.
Naive Bayes	Depends on variant	Scaling may change numerical conditioning but is not a universal requirement; match preprocessing to the distributional assumption.

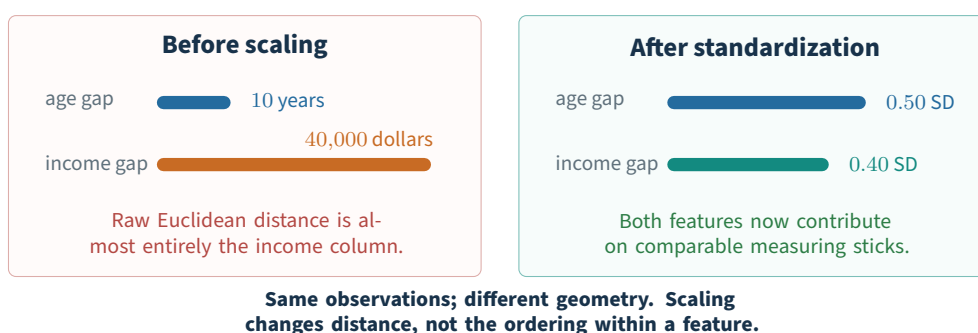
Other useful transformations include $\log(1 + x)$ for positive variables with a long right tail and power transforms for severe skewness. These are not interchangeable with scaling: a log transform changes the shape of the distribution, while a scaler mainly changes units and location.

Plain idea

Suppose one feature is age from 10 to 80 and another is annual income from 20,000 to 300,000. A Euclidean distance may be dominated by income simply because its numbers are larger. Scaling puts the features on comparable measuring sticks; it does not make an unimportant feature important or repair a bad definition of distance.

3.11.1 Scaling changes the geometry, drawn

Distance-based models do not know that one column is measured in years and another in dollars. They only see numerical gaps. Scaling changes those gaps into comparable units.



Key idea

For a distance-based method, changing units can change who counts as “near.” That is why scaling can change a *k*-NN, SVM, or clustering result even though no row and no label changed.

3.12 Outliers, imbalance, and rare cases

After impossible values and unit mistakes have been handled, an outlier may still be a valid rare event or evidence that several populations are mixed together. Investigate before deleting it. Data-driven rules such as percentile clipping or an interquartile-range cutoff must be learned from training data, because the cutoff depends on the observed distribution.

For class imbalance, the important job in this chapter is to *notice it and preserve it*. Record the class counts, use stratification when a random holdout is appropriate, and do not delete rare positive cases merely to make the table look balanced.

Key idea

Do not choose an imbalance “fix” before you know how the model will be evaluated. Chapter 7 develops precision, recall, precision–recall curves, thresholds, and ranking metrics. Chapter 8 then sets the safe imbalance policy; SMOTE is introduced with the other imbalance interventions in Chapter 8; its nearest-neighbor geometry becomes clearer after Chapter 11.

3.13 Feature engineering

Feature engineering creates useful inputs from raw data. Examples include:

- price per unit, body-mass index, or another meaningful ratio;
- elapsed time between two dates;
- weekday, month, or cyclical time features;
- interactions such as area multiplied by location score;
- counts or averages from past behavior, calculated without using the future.

Feature engineering by data type

Feature engineering depends on the structure of the information. Do not apply every transformation to every problem; choose transformations that preserve meaning and respect the prediction-time information boundary.

Data type	Common transformation ideas	Main safety question
Categorical	One-hot or ordinal encodings; grouped rare categories	Is the representation appropriate for nominal versus ordered values, and was any learned mapping fitted on training data only?
Numeric	Log, power, ratio, interaction, polynomial, or scaling transformations	Does the transformation preserve units and meaning, and were fitted quantities learned without using validation or test rows?
Temporal	Lags, elapsed time, rolling summaries, seasonal indicators	Was every input genuinely available at the prediction moment?
Text or images	Counts, embeddings, handcrafted descriptors, or learned representations	Does the representation preserve task-relevant structure without leaking label or future information?

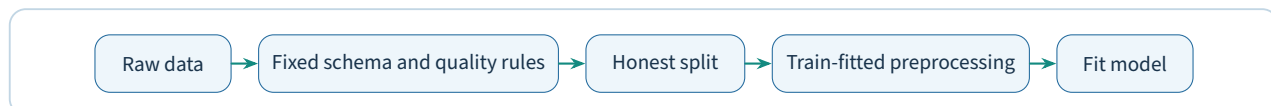
3.13.1 A safety test for engineered features

Before adding a derived column, ask two questions: *what information does it use?* and *when is that information available?* The answer determines where the transformation belongs.

Feature type	Example	Safe treatment
Deterministic	BMI from documented height and weight; extracting hour from a timestamp	May be computed reproducibly wherever the required raw fields are legitimately available.
Distribution-fitted	Power transform, frequency encoding, clipping threshold, learned category grouping	Fit from training rows only, normally inside a pipeline.
Label-aware	Label-informed mean encoding, supervised feature selection, a category merge chosen because it separates labels	Fit inside training folds; never compute from the complete labeled table before validation. Supervised selection is treated as a leakage-sensitive model-development step in Chapter 8.
Temporal aggregate	Lag, rolling mean, previous-week count	Use observations available by the prediction moment only; never back-fill a missing past feature with a future value.

3.14 Use a preprocessing pipeline

A pipeline stores the exact order of transformations. It learns imputation values, category mappings, and scaling values from training data, then applies those same learned values everywhere else. Later, the same container also keeps supervised feature-selection and resampling steps inside model selection (Chapter 8).



Document the data source, collection period, row definition, feature units, label timing, inclusion rules, split logic, random seed, known bias, and privacy limits.

Key idea

Prepare the training data first, learn every transformation from it, and reuse those fitted transformations unchanged on validation, test, and future production data.

3.15 The whole preparation as one object

Key idea

The pipeline is the unit you fit and later evaluate. The median age, scaling values, and category vocabulary are learned from the training rows and then reused unchanged on the holdout. Do not interpret or tune from the holdout yet; Chapter 7 introduces the metrics and shows that the same pipeline is refitted separately inside every cross-validation training fold. This is why a pipeline is more than convenient syntax: it enforces the information boundary.

Debugging task

A teammate computes median values, standardization statistics, and a label-informed mean encoding on the complete labeled table, then creates the train–test split. Identify every quantity that carried information from the future holdout backward. Rewrite the order of operations in six lines of pseudocode.

PRACTICAL MILESTONE

Lab 1 of 10

Environment and data foundations

From a reproducible environment to a defensible data quality assessment

Coverage. Chapters 2–3.

Source files. Lab01/Task01.ipynb, Task02.ipynb. Datasets live in all_datasets/.

Core data. Titanic; the environment check uses no dataset.

Goal. Verify the scientific-Python environment, establish the repository path convention, and assess a dataset systematically before fitting any model.

How much is scaffolded?

This first lab combines setup and data preparation into one foundations milestone. The environment check provides evidence that later results are reproducible; the Titanic data quality assessment shows that the table is understood before modeling begins.

Part A — Environment check.

```
import numpy as np
import pandas as pd
import matplotlib
import sklearn
```

```
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
print("scikit-learn:", sklearn.__version__)
```

Part B — Titanic data quality assessment.

```
from pathlib import Path
import pandas as pd
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "titanic_dataset" / "Titanic-Dataset.csv"
df = pd.read_csv(train_path)
```

```
# Keep the assessment explicit: rows, columns, types, missingness, and label balance.
print("shape:", df.shape)
print(df.head())
print("\nData types:\n", df.dtypes)
print("\nMissing values:\n", df.isna().sum().sort_values(ascending=False))
print("\nSurvival rate:\n", df["Survived"].value_counts(normalize=True))
```

```
# A few simple, interpretable features used later in the course.
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

summary = (
    df.groupby(["Sex", "Pclass"], dropna=False)["Survived"]
      .agg(["count", "mean"])
      .rename(columns={"mean": "survival_rate"})
)
```

```
print("\nSurvival by sex and class:\n", summary)
```

Try it yourself

Add three data quality checks of your own. At least one must concern missingness or duplicates, and at least one must concern whether a feature would be available at prediction time.

Lab deliverable

Submit one notebook containing the environment versions, repository-path check, schema and missingness assessment, label prevalence, duplicate check, and two prediction-time feature decisions with short justifications. No model score is required.

Check your understanding

1. Why must train/test splitting happen before any preprocessing step that learns from the data?
2. What is the difference between an identifier, a feature, and a label?
3. Why is a prediction-time information boundary necessary when deciding whether a feature is leakage?

Chapter recap

- A data quality assessment begins with the meaning of rows, columns, units, timing, and missing populations.
- Fixed, label-blind schema corrections can precede the split; quantities learned from observed data belong inside training.
- Duplicates, invalid values, rare values, and missing values are different problems and should not be handled by one automatic deletion rule.
- The holdout must imitate future use and respect groups, time, or location when necessary.
- Leakage occurs when unavailable or held-out information influences features, preprocessing, or model-development choices.
- Missing-value handling, encoding, scaling, and distribution-fitted feature engineering must learn from training data only.
- A preprocessing pipeline keeps the preparation rule attached to the model and ready for honest evaluation in Chapter 7.

How Machine Learning Works

Chapter at a glance

Before you start: Chapter 3: a trustworthy table, a clear observation unit, and a split that matches the real prediction setting.

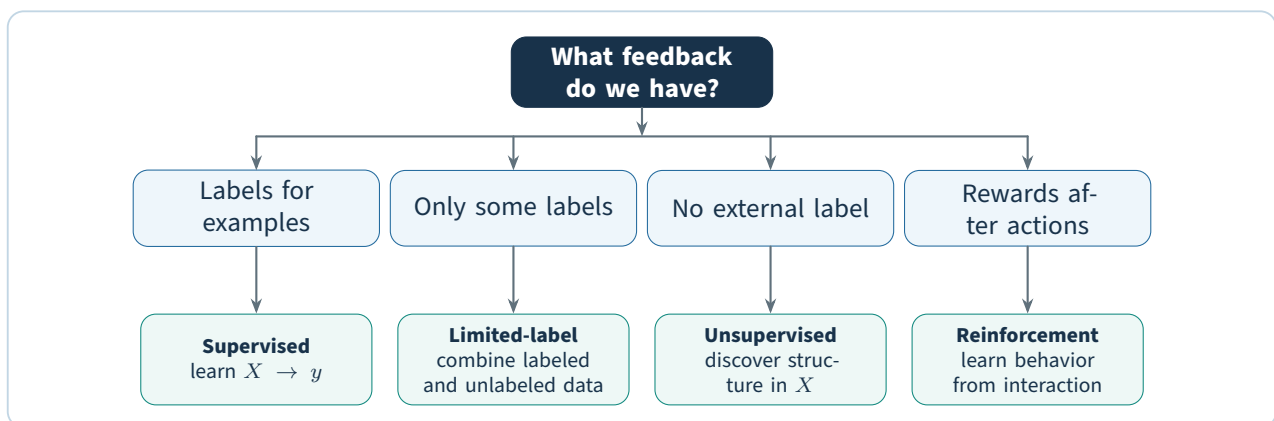
By the end, you can: separate learning settings by their feedback signal, explain what an algorithm is trying to learn in each setting, distinguish model fitting from evaluation, and identify which mathematical ideas you actually need next.

Up to this point, the book has deliberately avoided committing to one kind of machine-learning problem. You now know how to represent data and prepare it safely. Before studying a large block of mathematics, first identify the learning signal and the kind of structure a model must learn. That context tells you why later ideas such as probability, distance, dot products, and gradients matter: *what feedback is available, and what structure are we trying to learn?*

Key idea

A machine-learning algorithm extracts useful structure from data. What counts as “useful” changes with the feedback signal. With labels, the structure should predict those labels. Without labels, the structure must be judged by stability, geometry, reconstruction, density, downstream usefulness, or domain meaning. With rewards, the structure must support good decisions over time.

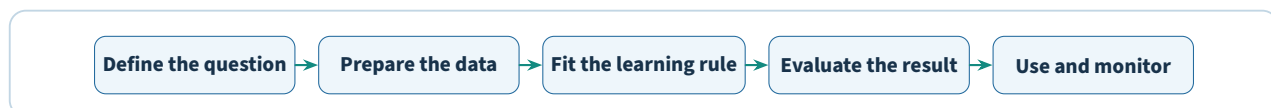
4.1 Start with the learning signal



Setting	Given	What is learned?	Typical question
Supervised	X and label y	A mapping from features to a label	“What value or class should I predict?”
Unsupervised	X only	Structure, representation, density, or unusualness in the features	“What patterns are present when no answer key is supplied?”
Semi-supervised / active	X plus some labels	A predictive mapping that also uses unlabeled structure or selectively requests labels	“How can scarce labels be used efficiently?”
Reinforcement	States, actions, rewards, transitions	A policy or value rule for sequential decisions	“Which action leads to good long-term outcomes?”

4.2 The common skeleton

Despite the different learning signals, most projects share a small backbone:



The details inside the boxes change. Supervised learning can compare predictions with known labels. Unsupervised learning often has no single correct answer and therefore needs several kinds of evidence. Reinforcement learning evaluates behavior across trajectories rather than independent labeled rows.

4.3 What changes between supervised and unsupervised learning

Question	Supervised learning	Unsupervised learning
What data are fitted?	Features X plus label y	Features X only
What guides fitting?	A loss comparing predictions with known answers	An objective such as compact clusters, reconstruction, variance preservation, density, or local isolation
What is the output?	Prediction rule, score, or probability	Groups, coordinates, components, density model, anomaly score, or discovered pattern
What is a baseline?	Mean/median/majority or a domain rule	Simple geometry or domain reference, random/naive partition, reconstruction reference, or stability comparison depending on the task
What does “good” mean?	Strong performance on realistic unseen labeled data	Structure that is stable, meaningful, useful downstream, or supported by appropriate internal/external criteria

Common mistake

Do not force supervised language onto every problem. A cluster does not automatically have a “correct class,” a PCA component is not a prediction, and an anomaly score is not automatically a

probability of fraud. First identify the learning objective; only then choose an evaluation method.

4.4 A first look at optimization: improve the objective step by step

Some models can be fitted by a direct formula; many others are trained iteratively. The common idea is to define an objective $J(\theta)$ and update the parameters in a direction that reduces it. For gradient descent,

$$\theta \leftarrow \theta - \alpha \nabla J(\theta),$$

where ∇J points in the direction of steepest increase and the learning rate α controls the step size.

Plain idea

Imagine walking downhill in fog. The local slope tells you which direction descends; the learning rate is your stride length. Tiny steps are safe but slow, sensible steps make steady progress, and steps that are too large can overshoot and diverge.

Worked example

For $J(w) = (w - 4)^2$, the minimum is at $w = 4$ and $J'(w) = 2(w - 4)$. Starting from $w = 0$ with $\alpha = 0.1$ gives $w \leftarrow 0 - 0.1(-8) = 0.8$ on the first step. Repeating the update moves w toward 4. You do not need the full calculus derivation yet; Chapter 5 explains what gradients mean geometrically.

Key idea

Optimization is part of *fitting*, not evaluation. A training loss can fall perfectly while generalization remains poor. Later chapters therefore separate “did the optimizer fit the training objective?” from “does the fitted procedure work on realistic unseen data?”

4.5 The path through the rest of the book

The book now branches cleanly.

Stage 3A: supervised workflow

Learn the prediction objective, baselines, evaluation, and model-selection discipline before comparing algorithms.

Stage 3B: supervised model families

Hold that workflow fixed while changing one family at a time from linear models through SVMs.

Stage 4A: unsupervised workflow

Reset the objective: no label is supplied during fitting, so evidence must come from geometry, stability, usefulness, or external meaning.

Stage 4B: unsupervised model families

Apply that workflow to clustering, representation learning, and anomaly detection.

Stage 5: other signals Change the amount or type of feedback: limited labels in semi-supervised/active learning, then rewards from interaction in reinforcement learning.

Key idea

The field map tells you where ideas belong. The learning path tells you what to learn next. Do not try to memorize the whole field at once: learn one workflow, then change one axis at a time.

4.6 Now learn only the mathematics that the workflow needs

The next chapter supplies a shared mathematical toolkit. Read it with a purpose rather than as a prerequisite exam. On the first pass, focus on center and spread, probability intuition, vectors and matrix shapes, dot products, distance, scaling, and gradient direction. Return to confidence intervals, distribution families, matrix inverses, and decomposition details when a later model gives you a reason to use them.

Core path

First-pass contract. You do not need to master every derivation before continuing. Your first goal is recognition: when a later chapter says “standardize before measuring distance,” “optimize a loss,” or “project onto a component,” you should know which mathematical idea is being reused and where to review it.

Check your understanding

1. What feedback signal separates supervised, unsupervised, limited-label, and reinforcement-learning problems?
2. Why can two learning settings share data preparation and monitoring practices while requiring different fitting objectives?
3. What is the difference between reducing a training objective and demonstrating generalization?

Chapter recap

- Learning settings are separated first by the feedback available, not by algorithm names.
- Supervised learning learns a label mapping; unsupervised learning learns structure without a supplied label; limited-label methods mix labeled and unlabeled evidence; reinforcement learning learns from rewards.
- Data preparation, representation, evaluation discipline, and monitoring remain shared foundations even though the fitting objective changes.
- Stage 3 first establishes the supervised workflow and then compares supervised model families; Stage 4 mirrors that structure for unsupervised learning.

Statistics, Geometry, and Similarity

Chapter at a glance

Before you start: Chapter 3, Chapter 4, and comfort with arithmetic, fractions, powers, and basic algebra.

By the end, you can: use the minimum statistical and geometric vocabulary needed by later models, recognize when scale or uncertainty changes an analysis, and know which second-pass topics can safely wait until a later chapter requires them.

Machine learning repeatedly asks two kinds of questions about data: *what does this collection look like?* and *how are these observations related?* Statistics answers the first with center, spread, probability, and association. Geometry and linear algebra answer the second with vectors, distances, dot products, and transformations. Keeping them together gives the shared vocabulary needed by both supervised and unsupervised learning, but this chapter is deliberately layered: the first pass gives you enough mathematics to continue; the second pass deepens ideas when you have a concrete reason to return.

Core path

First pass — learn these now: mean versus median, spread and quantiles, probability intuition, correlation versus causation, vectors, matrix shapes, dot products, norms, Euclidean distance, cosine similarity, feature scaling, and the idea that a gradient points uphill. These concepts recur immediately in the model chapters.

Second pass

Return later when needed: standard errors and confidence intervals, the bootstrap, detailed probability distributions, Bayes' theorem algebra, matrix inverses, eigenvectors/SVD, and curvature. They are valuable, but none should block a beginner from reaching the first models.

Part A: Describe data with statistics and probability

5.1 Describe a distribution with center and spread

A **distribution** describes which values occur and how often they occur. Two questions come first:

1. Where is the center?
2. How spread out are the values?

5.1.1 Mean and median

For values $x_1, \dots, x_N$, the population mean is

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i.$$

The capital sigma $\sum$ means “add the values.” The formula therefore says exactly what an ordinary average does: add all N values and divide by N .

The sample mean is usually written $\bar{x}$ and uses the same calculation on a sample.

The **median** is the middle value after sorting. It is less affected by extreme values.

Worked example

For 30, 32, 35, 38, 500, the mean is 127, while the median is 35. The mean answers “what would everyone receive if the total were shared equally?” The median answers “what value sits in the middle?” The median better represents a typical income in this strongly skewed example.

5.1.2 Variance and standard deviation

Variance measures average squared distance from the mean:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2.$$

The standard deviation is

$$\sigma = \sqrt{\sigma^2}.$$

Standard deviation is easier to interpret because it uses the original unit.

Worked example

For 2, 4, 6, the mean is 4. The distances from the mean are $-2, 0, 2$. Squaring gives 4, 0, 4, so the population variance is $8/3 \approx 2.67$ and the standard deviation is about 1.63.

5.1.3 Quantiles and the interquartile range

A quantile is a cut point in sorted data. The first quartile Q_1 has about 25% of values below it, and the third quartile Q_3 has about 75% below it. The interquartile range is

$$\text{IQR} = Q_3 - Q_1.$$

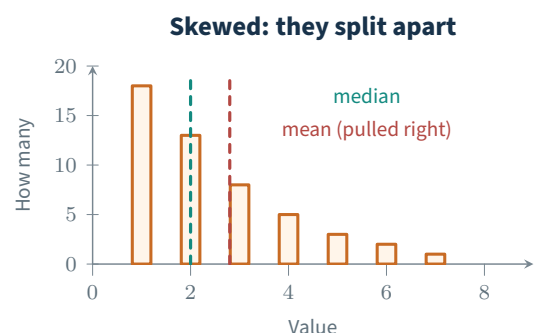
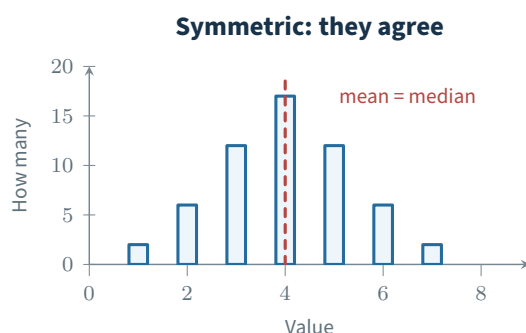
It measures the width of the middle half of the data and is resistant to extreme values.

Key idea

Use the mean and standard deviation for roughly symmetric data without severe outliers. Use the median and IQR when the distribution is skewed or contains extreme values.

5.2 Mean versus median, drawn

The mean and median agree on symmetric data and disagree sharply on skewed data. That disagreement is itself a useful diagnostic.



Analogy

Put ten ordinary people in a room and the mean and median income are similar. Now let a billionaire walk in. The median barely moves — there is still an ordinary person in the middle — but the mean leaps to something no one in the room actually earns. The median is stubborn; the mean is easily dragged.

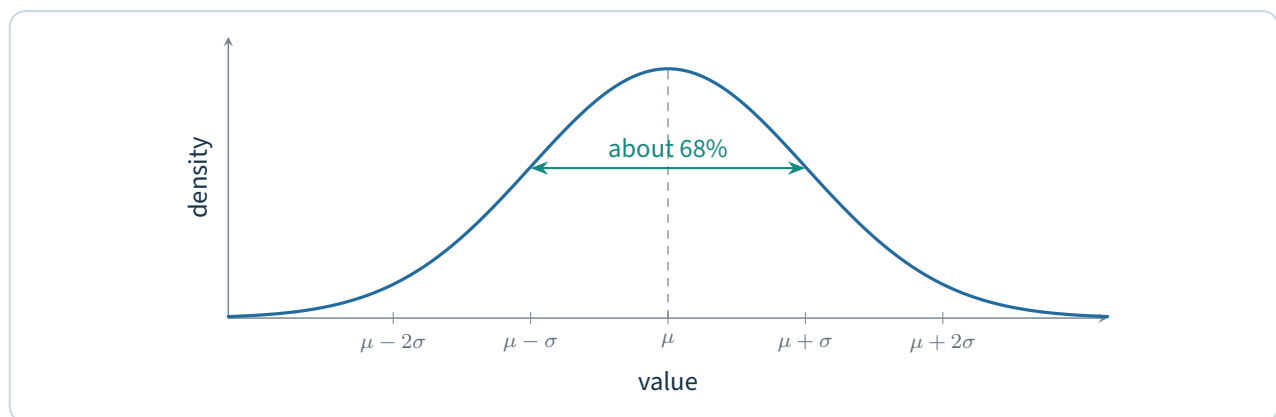
Key idea

If the mean is far above the median, the data has a long right tail. That can motivate a log transform or robust scaler. In Chapter 7, the same idea reappears when MAE and RMSE react differently to extreme prediction errors.

5.3 The normal distribution

The normal distribution is the familiar bell-shaped curve. It is controlled by a mean μ and standard deviation σ :

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right].$$



Many measurement errors and averages are approximately normal, but real data should not be assumed normal without inspection.

Plain idea

The mean moves the bell left or right. The standard deviation controls its width. A small standard deviation creates a narrow bell; a large one creates a wide bell.

5.4 Population, sample, and sampling bias

A **population** is the full group we want to understand. A **sample** is the smaller group actually observed. We calculate a statistic from the sample to estimate an unknown population parameter.

For a sample of size n , the common sample variance is

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

The denominator $n - 1$ corrects the tendency to underestimate population variance when deviations are measured around the sample mean.

Common mistake

A large sample is not automatically a good sample. If the sample systematically excludes part of the population, collecting more of the same biased data gives a more precise answer to the wrong question.

5.5 Standard error and confidence intervals

Standard deviation describes variation among individual observations. **Standard error** describes uncertainty in an estimate. For a sample mean,

$$\text{SE}(\bar{x}) = \frac{s}{\sqrt{n}}.$$

Larger samples reduce standard error, but the improvement follows $1/\sqrt{n}$. To cut standard error in half, we need about four times as much data.

For independent observations from the same distribution with finite variance, the central limit theorem says that sample means become approximately normally distributed as the sample size grows. The approximation may be poor for small samples, very heavy tails, or dependent observations. Under suitable conditions it supports an approximate confidence interval:

$$\bar{x} \pm z_{1-\alpha/2} \frac{s}{\sqrt{n}}.$$

A 95% confidence procedure produces intervals that contain the true parameter in about 95% of repeated samples, assuming the method's conditions are satisfied.

5.5.1 The bootstrap

When an estimator has no convenient standard-error formula, the **bootstrap** repeatedly samples rows *with replacement* from the observed sample and recalculates the estimator. The resulting distribution describes sensitivity to sampling variation.

Step 1: Draw n rows with replacement from the n observed rows.

Step 2: Recalculate the statistic, such as a median or a difference in model loss.

Step 3: Repeat many times and use suitable quantiles as an interval.

This ordinary row bootstrap assumes the rows are exchangeable. Time series, groups, repeated measurements, and paired model predictions require a structure-preserving procedure. A bootstrap interval is only as valid as its sampling design.

5.6 Probability basics

A probability lies between 0 and 1. For events A and B :

- $P(A)$ is the probability of A .
- $P(A \cap B)$ is the probability that both occur.
- $P(A \cup B)$ is the probability that at least one occurs.
- $P(A \mid B)$ is the probability of A when B is known.

The multiplication rule is

$$P(A \cap B) = P(A)P(B \mid A).$$

Events are independent when knowing one does not change the probability of the other:

$$P(A \cap B) = P(A)P(B).$$

5.7 Bayes' theorem

Bayes' theorem reverses a conditional probability:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}.$$

Prior	$P(A)$: belief before seeing new evidence.
Likelihood	$P(B A)$: how likely the evidence is if A is true.
Posterior	$P(A B)$: updated belief after seeing the evidence.

Worked example

A condition affects 1% of people. A test detects 90% of true cases and gives a false positive for 5% of people without the condition. Then

$$P(+) = 0.90(0.01) + 0.05(0.99) = 0.0585.$$

Therefore,

$$P(\text{condition} | +) = \frac{0.90(0.01)}{0.0585} \approx 0.154.$$

A positive result corresponds to about a 15.4% chance of the condition, not 90%, because the condition is rare.

Key idea

Always consider the base rate. Strong-looking evidence can still lead to a modest posterior probability when the event is rare.

5.8 Random variables and common distributions

A random variable assigns a number to an uncertain outcome. Its expectation is a probability-weighted average. For a discrete random variable,

$$\mathbb{E}[X] = \sum_x xP(X = x).$$

Common distributions include:

Bernoulli	One binary outcome with probability p .
Binomial	Number of successes in n independent Bernoulli trials.
Categorical	One result from several classes.
Poisson	Count of events in a fixed interval under simplifying assumptions.
Uniform	All values in a range have equal density, or all finite outcomes have equal probability.
Normal	Continuous values concentrated symmetrically around a mean.

Probability distributions connect directly to loss functions. Gaussian error leads naturally to squared-error regression, while a Bernoulli model leads naturally to binary cross-entropy.

5.9 Covariance and correlation

Covariance describes whether two variables tend to move together:

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)].$$

Pearson correlation standardizes covariance:

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}.$$

It lies between -1 and 1 and measures linear association.

Common mistake

Correlation does not prove causation. A hidden variable may affect both quantities, the direction may be reversed, or the relationship may result from how the data was selected.

A correlation near zero also does not rule out a curved or other nonlinear relationship.

Beyond the core

Likelihood and maximum likelihood estimation. Given data $x_1, \dots, x_n$, the likelihood treats the probability of the observed data as a function of a parameter θ :

$$L(\theta) = \prod_{i=1}^n p(x_i | \theta).$$

Maximum likelihood estimation chooses the parameter that makes the observed data most probable. In practice, we maximize the log-likelihood because it turns a product into a sum:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \sum_{i=1}^n \log p(x_i | \theta).$$

Many familiar machine-learning losses are negative log-likelihoods.

Beyond the core

Hypothesis tests, p-values, and effect size. A hypothesis test asks whether data would be surprising under a stated null model. A p-value is the probability, assuming the null model, of seeing a result at least as extreme as the observed one. It is not the probability that the null hypothesis is true. Report practical effect size and uncertainty, not only statistical significance.

Beyond the core

Entropy, cross-entropy, and KL divergence. Entropy measures uncertainty in a probability distribution. Cross-entropy measures how well predicted probabilities match true outcomes and is widely used in classification. KL divergence measures how one probability distribution differs from another. These ideas reappear in Chapter 10 and Chapter 7.

5.10 Scalars, vectors, and matrices

Scalar One number, such as temperature 23.

Vector An ordered list of numbers, such as one house $\mathbf{x} = [120, 3, 8]^T$.

Matrix A rectangular table of numbers, such as a dataset with one row per house and one column per feature.

For n observations and p features, the design matrix has shape

$$X \in \mathbb{R}^{n \times p}.$$

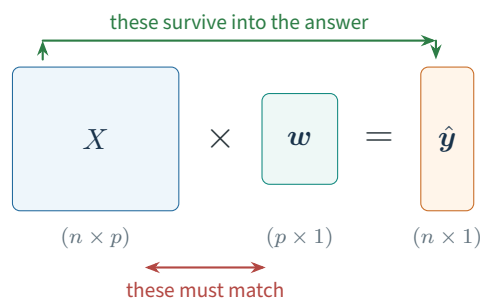
The first number is the number of rows; the second is the number of columns.

Worked example

If 500 students are described by age, study hours, attendance, and previous score, then X has shape 500×4 . One row is a four-number vector describing one student.

5.11 Matrix shapes: the rule that governs everything

Most linear-algebra errors in machine learning are shape errors. One rule prevents nearly all of them: the inner dimensions must match, and the outer dimensions survive.



Analogy

Think of matrices as adaptors. A cable with a p -shaped plug fits a socket with a p -shaped hole. What comes out the other end has whatever shape that device produces. If the dimensions disagree, the multiplication is not defined.

5.12 Vector arithmetic

Vectors of the same length can be added component by component:

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}.$$

Multiplying by a scalar stretches or reverses a vector:

$$3 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

5.13 Dot products: weighted sums and alignment

The dot product of two vectors is

$$\mathbf{a}^\top \mathbf{b} = \sum_{j=1}^p a_j b_j.$$

It has two useful interpretations.

1. It creates a weighted sum. Linear models use $\mathbf{w}^\top \mathbf{x} + b$.

2. It measures alignment. A large positive dot product means the vectors point in similar directions; a negative one means they point in opposing directions.

Worked example

For $w = [2, -1]^T$ and $x = [3, 4]^T$,

$$w^T x = 2(3) + (-1)(4) = 2.$$

A linear model with bias $b = 0.5$ would predict 2.5.

5.14 Norms: measuring vector size

The L_1 norm is the sum of absolute values:

$$\|x\|_1 = \sum_j |x_j|.$$

The L_2 norm is the ordinary straight-line length:

$$\|x\|_2 = \sqrt{\sum_j x_j^2}.$$

These norms appear in distance calculations and regularization.

5.15 Distance and similarity

A model often needs to decide whether two observations are close or similar.

Measure	Formula or idea	Useful when
Euclidean distance	$\ x - z\ _2$	ordinary geometric distance; features are scaled
Manhattan distance	$\ x - z\ _1$	movement along grid-like axes; more robustness to a single large difference
Cosine similarity	$\frac{x^T z}{\ x\ _2 \ z\ _2}$	direction matters more than magnitude, such as word-count vectors
Jaccard similarity	overlap divided by union	sets or binary presence/absence features
Hamming distance	number of unequal positions	binary strings or categorical codes

Common mistake

Distance-based models can be meaningless when features use very different scales. Standardize or otherwise justify the scale before calculating distances.

The same two points can look different under different measures

The course material lists Euclidean, Manhattan, Jaccard, Minkowski, and cosine measures because there is no universal definition of “close.” The data representation and the question determine the geometry.

Worked example

For points $P = (1, 2)$ and $Q = (4, 6)$,

$$d_2(P, Q) = \sqrt{(4 - 1)^2 + (6 - 2)^2} = 5,$$

while

$$d_1(P, Q) = |4 - 1| + |6 - 2| = 7.$$

Euclidean distance measures the straight-line path; Manhattan distance measures the axis-by-axis path.

The **Minkowski distance** unifies both:

$$d_p(\mathbf{x}, \mathbf{z}) = \left(\sum_j |x_j - z_j|^p \right)^{1/p}.$$

Setting $p = 1$ gives Manhattan distance and $p = 2$ gives Euclidean distance. Changing p changes which differences matter most, so it is a modeling choice rather than a cosmetic parameter.

Worked example

For sets $A = \{0, 1, 2, 5, 6\}$ and $B = \{0, 2, 3, 4, 5, 7, 9\}$, the intersection has three elements and the union has nine. Therefore

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{3}{9} = 0.33.$$

Jaccard similarity ignores shared absences and focuses on overlap among present items, which is why it is natural for sets and sparse binary indicators.

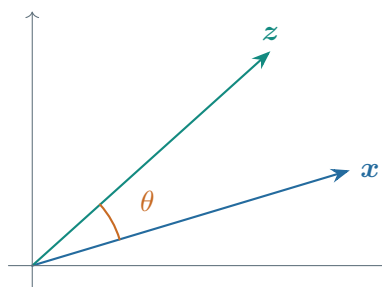
Key idea

Choose the representation and the distance together. Euclidean distance on one-hot categories, cosine similarity on raw physical measurements, or Jaccard similarity on dense continuous variables may all produce numbers while answering the wrong geometric question.

5.16 Cosine similarity visually

Cosine similarity depends on the angle θ between two vectors:

$$\cos(\theta) = \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\|_2 \|\mathbf{z}\|_2}.$$



A cosine similarity near 1 means similar direction, near 0 means roughly perpendicular, and near -1 means opposite direction.

5.17 Matrices and linear transformations

A matrix can transform one vector into another:

$$\mathbf{y} = A\mathbf{x}.$$

Depending on its values, A can rotate, stretch, shrink, reflect, or combine coordinates. A linear regression model applies this same idea to many rows at once:

$$\hat{\mathbf{y}} = X\boldsymbol{\beta}.$$

Matrix multiplication is possible when the inner dimensions match. If A is $m \times p$ and B is $p \times k$, then AB is $m \times k$.

Plain idea

Think of matrix multiplication as a factory line. Each output column is built from weighted combinations of the input columns. The shape tells you whether the machines can connect.

5.18 Transpose, identity, and inverse

The transpose switches rows and columns. If A is $m \times p$, then A^T is $p \times m$.

The identity matrix I acts like the number 1:

$$I\mathbf{x} = \mathbf{x}.$$

When an inverse exists,

$$A^{-1}A = I.$$

In practice, machine-learning software usually solves linear systems using stable numerical methods instead of explicitly calculating an inverse.

5.19 Gradients: which way changes the loss?

For a function with several parameters, the gradient collects all partial derivatives:

$$\nabla_{\boldsymbol{\theta}} J = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_p} \end{bmatrix}.$$

The gradient points toward the fastest increase in J . To reduce a loss, gradient descent moves in the opposite direction:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J.$$

This chapter needs only the direction: the gradient points uphill, so optimization moves against it. The full gradient-descent intuition, learning-rate cases, and worked example appear once in Section 4.4.

Beyond the core

Eigenvectors, eigenvalues, and singular value decomposition. An eigenvector of a square matrix keeps its direction after the matrix transformation, changing only in size. Singular value decomposition factors any matrix into rotations and axis-wise scaling. These ideas power principal component analysis in Chapter 17. On a first reading, understand the purpose: they reveal important directions in data.

Beyond the core

The Hessian and curvature. A Hessian matrix collects second derivatives and describes how the gradient changes. Positive curvature suggests a bowl-like objective; negative curvature suggests a hill-like direction. This is useful for advanced optimization but is not required for the algorithms in this beginner edition.

Key idea

Vectors represent observations and parameters; matrices represent datasets and transformations; dot products create weighted sums; distances compare observations; and gradients tell optimization which direction to move.

PRACTICAL MILESTONE

Lab 2 of 10

Learning loop, preprocessing, and geometry

Build one leakage-safe pipeline and inspect the geometry it creates

Coverage. Chapters 4–5.

Source files. Lab02/Task01.ipynb, Task02.ipynb.

Core data. Medical Cost Personal Data and Breast Cancer Wisconsin.

Goal. Complete a train–validation learning loop with preprocessing inside a pipeline, then demonstrate how scaling changes distance-based geometry.

How much is scaffolded?

Treat preprocessing as part of the fitted model. The second activity makes that rule concrete: standardization is not cosmetic when an algorithm depends on distances or angles.

Part A — End-to-end insurance learning loop.

```
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "medical_cost_personal_dataset" / "insurance.csv"
df = pd.read_csv(train_path)
```

```
X = df.drop(columns="charges")
y = df["charges"]

num_cols = ["age", "bmi", "children"]
cat_cols = ["sex", "smoker", "region"]
```

```
preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])
```

```
model = Pipeline([
    ("preprocess", preprocess),
    ("regressor", LinearRegression()),
])
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.20, random_state=42
)
model.fit(X_train, y_train)
pred = model.predict(X_valid)
```

```
print("Validation MAE:", round(mean_absolute_error(y_valid, pred), 2))
print("Validation RMSE:", round(np.sqrt(mean_squared_error(y_valid, pred)), 2))
```

Part B — Breast-cancer geometry and scaling.

```
from pathlib import Path
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.metrics import pairwise_distances
from sklearn.preprocessing import StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "breast_cancer_dataset" / "data.csv"
df = pd.read_csv(train_path)
```

```
feature_cols = [c for c in df.columns if c not in {"id", "diagnosis", "Unnamed: 32"}]
X = df[feature_cols]
```

```
# Compare raw Euclidean distance with distance after standardization.
raw = pairwise_distances(X.iloc[:5], metric="euclidean")
X_scaled = StandardScaler().fit_transform(X)
scaled = pairwise_distances(X_scaled[:5], metric="euclidean")
```

```
np.set_printoptions(precision=2, suppress=True)
print("Raw distances among first five rows:\n", raw)
print("\nScaled distances among first five rows:\n", scaled)
```

```
# Cosine similarity answers a different question: direction rather than magnitude.
cosine_distance = pairwise_distances(X_scaled[:5], metric="cosine")
print("\nCosine similarities:\n", 1 - cosine_distance)
```

Try it yourself

Remove one high-information insurance feature and compare validation error. Then choose three breast-cancer features on visibly different scales and compare raw versus standardized pairwise distances.

Lab deliverable

Submit validation MAE and RMSE for the reference insurance pipeline and one controlled feature-removal variant, plus a compact raw-versus-standardized distance comparison and a short explanation of why representation choices alter similarity.

Check your understanding

1. When would the median and IQR describe a distribution more faithfully than the mean and standard deviation?
2. If X has shape $n \times p$, what shape must a coefficient vector have for Xw to produce one prediction

per row?

3. Why can standardizing features change a nearest-neighbor or clustering result even though the observations themselves did not change?

Chapter recap

- Center and spread describe a distribution together; samples estimate populations but sampling bias is not repaired by sample size.
- Probability, Bayes' theorem, and correlation provide shared language for uncertainty and association without implying causation.
- A dataset with n rows and p features is an $n \times p$ matrix.
- Dot products are weighted sums and measures of alignment.
- Norms measure size; distances compare observations.
- Feature scale strongly affects distance-based methods.
- The negative gradient gives a local direction for reducing an objective.

Stage 3

Supervised Learning

Stage 3A

Supervised Workflow

Learn how supervised problems are framed, evaluated, and selected before changing model families.

How Supervised Learning Works

Chapter at a glance

Before you start: Stages 1 and 2: prepared data, basic statistics, vectors, and distance measures.

By the end, you can: Formulate regression and classification tasks, read a loss function, distinguish parametric from non-parametric models, and set up a baseline that any serious model must beat.

Supervised learning uses examples that contain both input features x and a known label y . The model learns a mapping f so that $f(x)$ is close to the correct label on data it has never seen. The previous chapter separated the major learning signals. We now enter the supervised branch. This chapter begins Stage 3A. It sets up the objective, loss, label types, and baseline that every supervised algorithm relies on. The next two chapters establish how supervised models are evaluated and selected without contaminating the final test set; Stage 3B begins only after that workflow is secure.

Plain idea

Supervised learning is learning by example with an answer key. Someone hands the model thousands of solved problems (“this house sold for \$310,000,” “this email was spam”) and the model’s job is to work out the general rule connecting the inputs to the answers, so that it can solve new problems where nobody knows the answer yet.

Core path

The entire chapter can be reduced to a loop: make a prediction, compare it with the known answer, measure the mistake, and adjust the rule. A **loss function** is simply the chosen mistake score. A **baseline** is the simple rule the model must beat.

6.1 Two kinds of labels

Regression The label is a number on a continuous scale: a price, a temperature, a length of stay. Predictions can be too high or too low by some amount.

Classification The label is a category: spam or not spam, which of ten digits, which species. Predictions are right or wrong, and a good model also reports *how confident* it is through class probabilities.

The same dataset can support either task depending on how the label is defined. Predicting tomorrow’s exact temperature is regression; predicting whether tomorrow will be “hot” or “cold” is classification. Ordered categories (small, medium, large) and counts sit between the two and sometimes need special treatment, but regression and classification cover the vast majority of practical problems.

6.2 The supervised-learning objective

A training dataset can be written as

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n.$$

The model has parameters θ and produces predictions $\hat{y}_i = f_{\theta}(x_i)$. Training usually minimizes the average loss over the training set,

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_{\theta}(x_i)),$$

a quantity called the **empirical risk**. Reading this equation piece by piece: $\mathcal{L}$ scores one prediction against one true answer, the sum walks through every training example, and dividing by n turns the total into an average. Training searches for the parameter values θ that make this average as small as possible.

Key idea

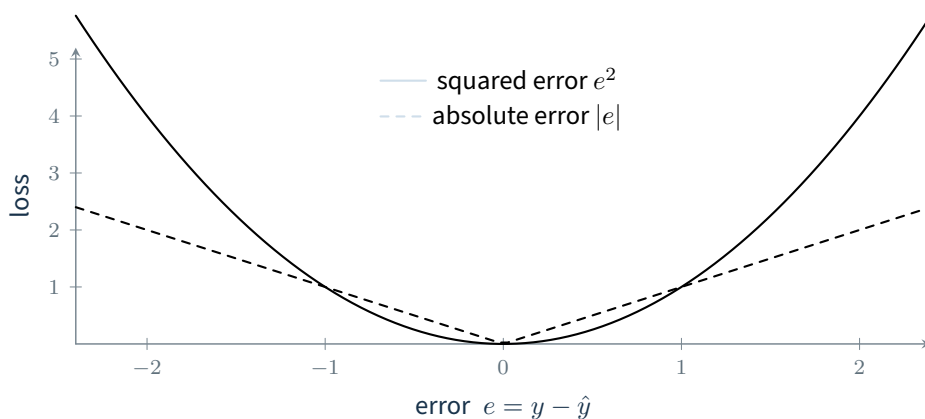
Training loss measures how well the model fits known examples. Validation and test metrics estimate how well it will perform on new examples. These are different things, and the entire craft of machine learning lives in that difference.

6.3 Loss functions: how wrongness is measured

A loss function converts “the prediction was wrong” into a number, and the choice of number shapes what the model learns to care about.

For regression, with error $e = y - \hat{y}$, the two workhorses are

$$\mathcal{L}_{\text{squared}}(e) = e^2, \quad \mathcal{L}_{\text{absolute}}(e) = |e|.$$



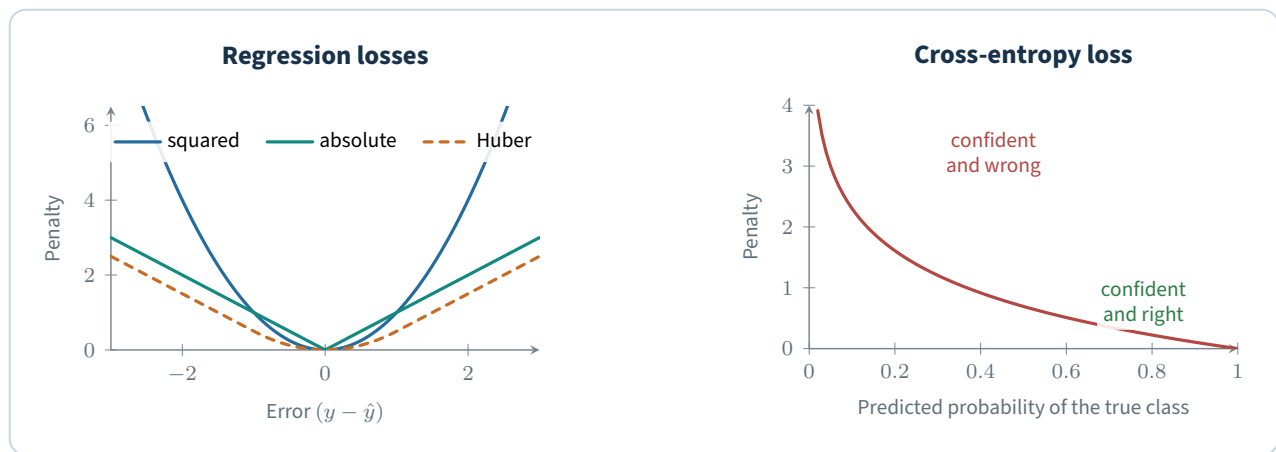
Plain idea

Squared error is a strict teacher: an error of 2 costs 4, but an error of 10 costs 100, so the model becomes obsessed with avoiding large mistakes — and therefore very sensitive to outliers. Absolute error charges every unit of error the same price, so the model tolerates a few large misses in exchange for being close most of the time.

For classification, the loss that people often care about — did the model get the label right or not, the **0–1 loss** — is flat almost everywhere, which gives an optimizer no slope to follow. Practical algorithms therefore minimize tractable stand-ins instead: logistic regression uses **cross-entropy** (Chapter 10), and support vector machines use the **hinge loss** (Chapter 14). These surrogate losses make optimization possible, but the final model must still be judged with application-relevant metrics.

6.3.1 What the loss shapes actually look like

A loss function is a rule for turning “how wrong were you?” into a single number. Different rules punish mistakes differently, and that choice quietly shapes the model you end up with.



Plain idea

Squared error is a curve, so a mistake twice as large is punished four times as hard. That makes the model chase outliers. Absolute error is a straight V, so every extra unit of error costs the same, which makes it far calmer around unusual points. Huber loss is the compromise: curved near zero, straight far away.

Cross-entropy has one dramatic feature: as the predicted probability of the true answer approaches zero, the penalty runs off to infinity. Being confidently wrong is treated as almost unforgivable, which is precisely what you want from a model that reports probabilities.

Key idea

Choose the loss to match the cost of being wrong. If one enormous error is much worse than several small ones, use squared error. If a few strange rows should not dominate the fit, use absolute error or Huber.

6.4 Parametric and non-parametric models

Model family	Structure and examples	Trade-off
Parametric models	Assume a fixed functional form with a fixed number of parameters, regardless of how much data arrives. Examples include linear regression, logistic regression, and naive Bayes.	They train fast, predict fast, and are easy to interpret, but the assumed form limits what they can express.
Non-parametric models	Let complexity grow with the data. Nearest neighbors keeps the entire training set; a decision tree grows as many branches as the data supports.	They can fit almost any shape of relationship, at the price of needing more data and more care to avoid overfitting.

Neither family is better in general. A parametric model with the right form is hard to beat on small data; a non-parametric model shines when the relationship is complicated and data is plentiful.

6.5 Always start with a baseline

A **baseline** is the simplest prediction rule imaginable, evaluated with the same metrics and splits as any real model:

- for regression, predict the training-set mean (or median) of the label for every observation;

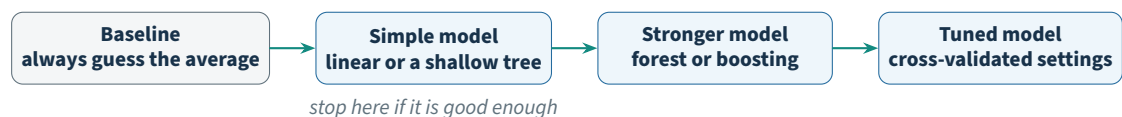
- for classification, predict the most common class for every observation;
- when domain rules exist (“flag any transaction over \$10,000”), evaluate those too.

Baselines protect against two common embarrassments. First, they expose data problems: if the majority-class baseline achieves 97% accuracy, the dataset is severely imbalanced and accuracy is the wrong metric. Second, they calibrate expectations: a complicated model that beats the mean predictor by a hair is probably not learning much, no matter how impressive its machinery.

Worked example

In a dataset where 950 of 1,000 emails are legitimate, the rule “predict *not spam* for everything” is 95% accurate while catching zero spam. Any model claiming success must therefore show more than high accuracy — for instance, good recall and usable precision on the rare spam class (Chapter 7).

6.6 Baselines you should always run



Analogy

The ladder above is how experienced practitioners actually work: climb one rung at a time and stop as soon as the result is good enough for the decision at hand. Beginners tend to leap straight to the top rung, and then have no way of knowing whether the complicated model earned its complexity.

Common mistake

A model that beats the baseline by a large margin on the very first try deserves suspicion before celebration. Check for leakage: an identifier column, a feature recorded after the outcome was known, or duplicate rows spread across both the training and test sets.

6.7 A map of the algorithms ahead

After Chapter 8, the six algorithm chapters that follow cover the supervised model families every practitioner is expected to know. They are grouped by the core idea each family is built on.

Chapter	Algorithms	Core idea
9	Linear, polynomial, ridge, lasso, elastic net regression	Fit a weighted sum of features; penalize large weights
10	Logistic regression, softmax, LDA, QDA, perceptron	Draw a linear boundary between classes
11	k -nearest neighbors, naive Bayes	Predict from similar examples; predict from per-feature probabilities
12	Decision trees (CART)	Ask a sequence of yes/no questions
13	Bagging, random forests, AdaBoost, gradient boosting, stacking	Combine many models into one predictor
14	Support vector machines, kernels, SVR	Maximize the margin; separate after mapping to a richer space

Each chapter follows the same rhythm: what the algorithm does in plain words, how training works step by step, a small worked example with real numbers, and finally when to use it and what its knobs (hyperparameters) control. Chapter 14 closes the supervised-learning stage with a side-by-side comparison of the models in the table.

Key idea

All supervised algorithms answer the same question — “what function turns these inputs into those outputs?” — but they differ in the shapes of functions they can represent, the assumptions they make, and the amount of data they need. Learning the algorithms means learning those trade-offs, not memorizing formulas.

Supervised Learning - Project Checklist

Question 1	Is the label a number, a category, or a special type such as a count or ordered category?
Question 2	What simple baseline should the model beat?
Question 3	Which loss guides training, and which metric matches the real cost of mistakes?
Question 4	How will the split imitate future predictions?
Question 5	Does the final model need interpretability, probabilities, speed, or a nonlinear boundary?

Check your understanding

1. How does a regression label differ from a classification label, and what does that change about the kind of prediction produced?
2. Why can the loss used for fitting differ from the metric used to report practical performance?
3. What does a learned model have to demonstrate beyond beating its own training loss?

Chapter recap

- Supervised learning uses feature-label pairs to learn a prediction rule.
- Regression predicts numbers; classification predicts categories or class probabilities.
- A loss guides training, while validation and test metrics estimate performance on new data.
- Parametric models use a fixed form; non-parametric models can grow in complexity with the data.
- Every serious model should be compared with a simple baseline.

Evaluating Supervised Models

Chapter at a glance

Before you start: Chapter 3 and Chapter 5: honest holdouts, label types, averages, spread, and sampling uncertainty.

By the end, you can: design a reliable supervised-learning evaluation, choose regression, classification, ranking, and probability metrics, use cross-validation, analyze uncertainty, and diagnose errors by subgroup.

In supervised learning, evaluation compares predictions with known outcomes, but the score is meaningful only when the split and metric match future use. Model evaluation should match the real purpose of the system. A metric is useful only when it reflects the consequences of the model's mistakes. Reliable evaluation also requires an appropriate data split, a meaningful baseline, and careful comparison across models.

Core path

Core path: the first five sections. Begin with consequences, choose a realistic split and baseline, learn the main regression/classification metrics, understand thresholds, then learn cross-validation. The chapter marks the remaining material as a second pass.

7.1 Evaluation begins with the real objective

Before choosing a metric, identify the decision the model will support. A hospital screening model, a search engine, and a demand forecast have different costs of error even if all produce numerical scores.

A useful evaluation plan states:

- the unit being predicted and the population of interest;
- how the data split imitates future use;
- the baseline that must be beaten;
- the primary metric used for model selection;
- secondary metrics and subgroup checks;
- acceptable failure rates and operational constraints.

Choosing many metrics after seeing the test results encourages selective reporting. Define the primary metric before final evaluation whenever possible.

7.2 Regression metrics

For true values y_i and predictions $\hat{y}_i$, mean squared error and root mean squared error are

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

MSE strongly penalizes large errors. RMSE is easier to interpret because it uses the same units as the label.

Worked example

Five delivery times, in minutes. Four predictions are nearly right; one is badly wrong.

True y	10	12	14	16	48
Predicted $\hat{y}$	11	13	13	15	20
Error $y - \hat{y}$	-1	-1	+1	+1	+28

The absolute errors are 1, 1, 1, 1, 28, so

$$\text{MAE} = \frac{1 + 1 + 1 + 1 + 28}{5} = \frac{32}{5} = 6.4, \quad \text{MedAE} = 1.$$

Squaring first gives 1, 1, 1, 1, 784, so

$$\text{MSE} = \frac{788}{5} = 157.6, \quad \text{RMSE} = \sqrt{157.6} \approx 12.55.$$

For R^2 , the mean of the true values is $\bar{y} = 100/5 = 20$, and

$$\sum (y_i - \bar{y})^2 = 100 + 64 + 36 + 16 + 784 = 1000, \quad R^2 = 1 - \frac{788}{1000} = 0.212.$$

Read the three error numbers together. **MedAE** = 1 says the typical prediction is off by one minute. **MAE** = 6.4 says the average miss is six minutes. **RMSE** $\approx$ 12.55 is almost twice the MAE. A single row is doing all of that damage: it contributes 784 of the 788 squared error, which is 99.5% of the total. Whenever RMSE is much larger than MAE, go and look at the worst few rows before you touch the model.

Common mistake

Do not discretize a continuous label into “low/medium/high” merely so that you can compute a confusion matrix. That changes the estimand: a regression problem about numerical error becomes a classification problem about thresholded categories. If those categories are genuinely the decision you care about, define them before modeling and train/evaluate that classification task directly. Otherwise keep the continuous label and use regression metrics.

7.3 Additional regression metrics

Mean absolute error is

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MAE weights errors linearly and is usually less sensitive to extreme mistakes than RMSE. Median absolute error is even more robust and describes a typical error.

Mean absolute percentage error is

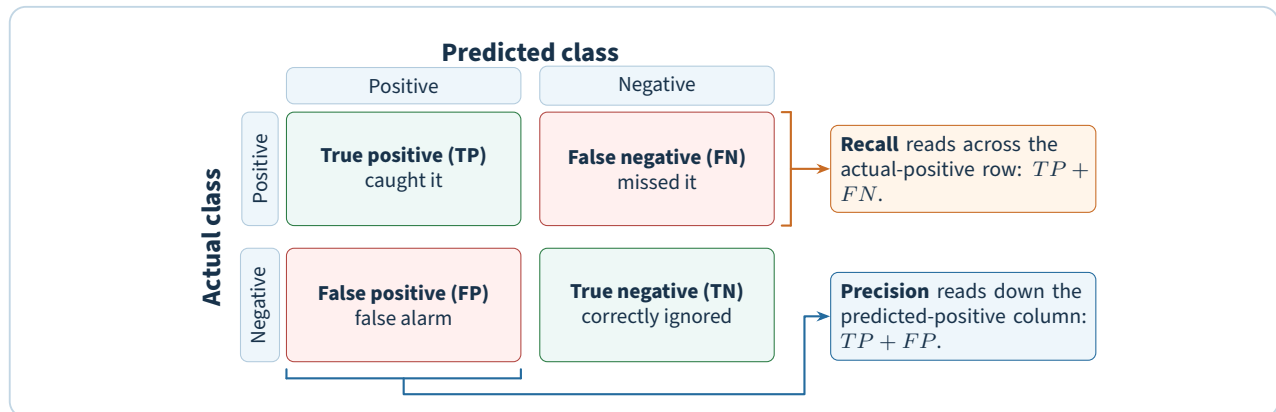
$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

It is easy to express as a percentage but becomes undefined or unstable when true values are zero or near zero. It can also penalize overprediction and underprediction asymmetrically.

For asymmetric real-world costs, a custom loss or quantile loss may be more appropriate. Quantile regression can estimate an interval of plausible outcomes rather than only a conditional mean.

7.4 Classification metrics: start with the confusion matrix

Almost every classification metric is a different way of reading the same four boxes.



Analogy

Precision asks, “When the alarm sounds, how often is there a real fire?” Recall asks, “Of all the real fires, how often did the alarm sound?” An alarm wired to go off constantly has perfect recall and usually poor precision. A disconnected alarm has zero recall and *undefined* precision because it never predicts positive. The useful operating point depends on false-alarm cost, missed-fire cost, and response capacity.

Metric	Formula	Answers the question
Accuracy	$\frac{TP + TN}{\text{all}}$	How often is the model right overall?
Precision	$\frac{TP}{TP + FP}$	When it says yes, is it right?
Recall	$\frac{TP}{TP + FN}$	Does it find the ones that matter?
F1	$2 \cdot \frac{P \cdot R}{P + R}$	One number balancing the two
Specificity	$\frac{TN}{TN + FP}$	Does it leave the negatives alone?

Common mistake

Confusion-matrix conventions differ across textbooks and software. Never memorize a corner position; read the axis labels and identify which dimension is actual class and which is predicted class.

Worked example

A spam filter is run on 1,000 emails. 100 of them really are spam. The filter flags 80 emails, and 60 of those flags are correct. That fixes all four cells:

$$TP = 60, \quad FP = 80 - 60 = 20, \quad FN = 100 - 60 = 40, \quad TN = 900 - 20 = 880.$$

(Check the total: $60 + 20 + 40 + 880 = 1000$.) Now every metric is arithmetic:

$$\text{Accuracy} = \frac{60 + 880}{1000} = 0.940, \quad \text{Precision} = \frac{60}{80} = 0.750, \quad \text{Recall} = \frac{60}{100} = 0.600,$$

$$F_1 = 2 \cdot \frac{0.750 \times 0.600}{0.750 + 0.600} = \frac{0.900}{1.350} \approx 0.667.$$

Now compare against the dumbest possible rule. “Never flag anything” gets $900/1000 = 90\%$ accuracy. Our model scores 94% — four points better than a rule that catches no spam at all, while quietly letting 40 spam emails through. Accuracy hid that; recall did not.

7.5 Additional confusion-matrix measures

In addition to precision and recall, useful classification measures include

$$\text{Specificity} = \frac{TN}{TN + FP}, \quad \text{False-positive rate} = \frac{FP}{FP + TN}.$$

Balanced accuracy averages recall for the positive and negative classes:

$$\text{Balanced accuracy} = \frac{1}{2}(\text{Recall} + \text{Specificity}).$$

It is more informative than ordinary accuracy when classes are imbalanced.

The Matthews correlation coefficient (MCC) summarizes the entire confusion matrix in one number:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

It ranges from -1 to 1 ; values near 0 indicate little association between prediction and truth, and its use of all four cells can make it informative under imbalance.

Worked example

Continuing the spam filter ($TP = 60$, $FP = 20$, $FN = 40$, $TN = 880$):

$$\text{Specificity} = \frac{880}{900} \approx 0.978, \quad \text{Balanced accuracy} = \frac{0.600 + 0.978}{2} \approx 0.789.$$

Ordinary accuracy said 0.940 ; balanced accuracy says 0.789 . The gap is the imbalance. Accuracy is dominated by the 900 easy negatives, while balanced accuracy gives the 100 spam emails equal say. MCC uses all four cells at once. The numerator is

$$TP \cdot TN - FP \cdot FN = 60(880) - 20(40) = 52,800 - 800 = 52,000,$$

and the denominator is

$$\sqrt{(80)(100)(900)(920)} = \sqrt{6,624,000,000} \approx 81,388,$$

so $\text{MCC} \approx 52,000/81,388 \approx 0.639$.

Three numbers, one model: 0.940 , 0.789 , 0.639 . None is wrong. They answer different questions, and reporting only the first would be the misleading choice.

For multiclass problems, metrics can be averaged in different ways; **macro averaging** gives every class equal weight, **micro averaging** pools all decisions and therefore gives common classes more influence, and **weighted averaging** weights each class by its frequency.

7.6 Thresholds, ROC curves, and precision-recall curves

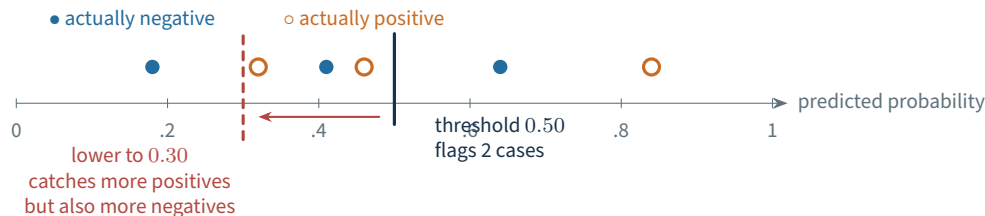
A probability model becomes a hard classifier only after a threshold is chosen. Lowering the threshold usually increases recall and false positives; raising it usually increases precision and false negatives.

The receiver operating characteristic (ROC) curve plots true-positive rate against false-positive rate across thresholds. ROC AUC is the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative example.

No area metric determines the operating threshold. Select a threshold or ranked cutoff with out-of-fold/validation predictions and application costs or capacity, then assess the locked rule once on test data.

7.6.1 Thresholds move the operating point

Suppose the model has already produced these six scores. Changing the threshold does not retrain the model; it only changes which scored cases become positive decisions.



Key idea

Lower threshold $\Rightarrow$ more predicted positives: recall usually rises and false positives usually rise.
Higher threshold $\Rightarrow$ fewer predicted positives: precision may rise, but more real positives can be missed. Choose the operating point from validation evidence and real costs or capacity.

7.6.2 Capacity and cost metrics

When only k cases can be reviewed, rank all eligible cases and report

$$\text{precision@ } k = \frac{\text{positives in top } k}{k}, \quad \text{recall@ } k = \frac{\text{positives in top } k}{\text{all positives}},$$

$$\text{lift@ } k = \frac{\text{precision@ } k}{\text{positive prevalence}}.$$

If false positives and false negatives have defensible monetary or utility costs, report the total expected or realized cost at the proposed operating point. Do not optimize a threshold metric when deployment actually takes a fixed-size ranked batch.

Worked example

A retention team can phone 5 customers this week. The model scores 20 eligible customers; sorting them from highest to lowest score gives this sequence of true outcomes, where 1 means the customer really did churn:

1 1 0 1 0 | 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0

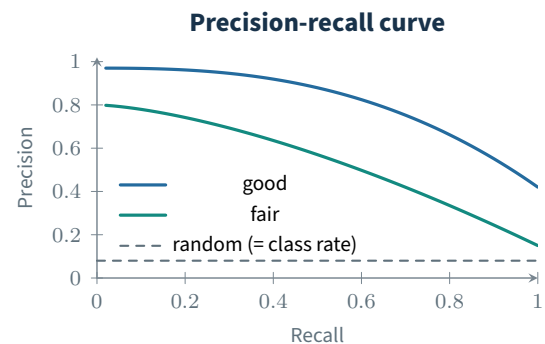
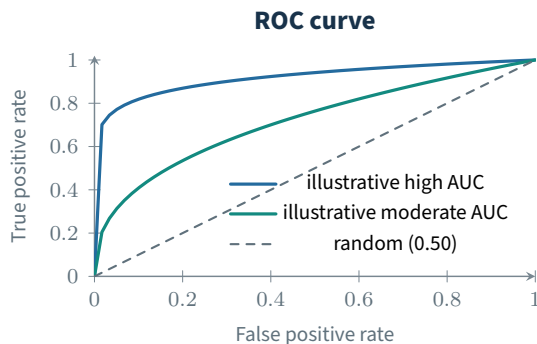
The bar marks the capacity cutoff at $k = 5$. There are 4 churners in the batch of 20, so the prevalence is $4/20 = 0.20$. Of the top five, three churned:

$$\text{precision@ } 5 = \frac{3}{5} = 0.60, \quad \text{recall@ } 5 = \frac{3}{4} = 0.75, \quad \text{lift@ } 5 = \frac{0.60}{0.20} = 3.0.$$

Read *lift* as the payoff for ranking: phoning five customers at random would reach $0.20 \times 5 = 1$ churner on average, and the model reached three. Note that no threshold was chosen anywhere in that calculation. The team's constraint is five calls, not a probability of 0.5, so the model's job is to sort the batch, not to label it.

Common mistake

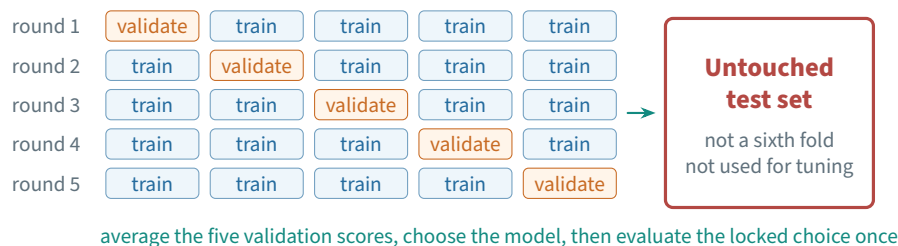
A high predicted churn risk does not mean a customer is highly persuadable. Precision@300 measures predictive ranking; it cannot prove that contacting those 300 customers causes additional retention. Causal intervention effects require a design such as a randomized experiment.

**Key idea**

When positives are rare and positive alerts drive action, inspect the precision-recall curve and the intended operating region. ROC AUC remains a valid ranking measure, but it can look strong while precision at a usable threshold is poor because the negative population is large. Report both only when each answers a stated question.

7.7 Cross-validation

In k -fold cross-validation, the training data is divided into k folds. The model trains on $k - 1$ folds and validates on the remaining fold, repeating until every fold has been used for validation. The fold scores estimate variation across data subsets.

Five-fold cross-validation stays inside the development data

Cross-validation must preserve the data structure:

- stratify folds for imbalanced classification;
- keep related observations in the same group fold;
- use forward or rolling validation for time series;
- fit preprocessing and feature selection separately inside each training fold.

After selecting a model and hyperparameters, refit on the allowed training data and evaluate once on the untouched test set.

7.7.1 Choose the cross-validation splitter that matches the data

Key idea

The splitter is part of the evaluation design. Use the same structure for every candidate model so the comparison changes the model, not the meaning of the validation task.

Second pass

The core first-pass material is now complete: objective, regression metrics, confusion-matrix metrics, thresholds, and cross-validation. The remaining sections add probability quality, repeated-experiment uncertainty, subgroup analysis, and distribution shift. Return to them when the application requires that extra precision.

7.8 Evaluating probabilities

Some applications need accurate probabilities, not merely correct rankings. Binary log loss is

$$-\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

The Brier score is the mean squared probability error:

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2.$$

Worked example

Four predictions, with the per-row cost under each score. Log loss charges $-\log p$ when $y = 1$ and $-\log(1 - p)$ when $y = 0$; Brier charges $(p - y)^2$ either way.

True y	Predicted p	Log loss	Brier
1	0.90	$-\ln 0.90 = 0.105$	0.01
1	0.60	$-\ln 0.60 = 0.511$	0.16
0	0.30	$-\ln 0.70 = 0.357$	0.09
0	0.05	$-\ln 0.95 = 0.051$	0.0025
mean		0.256	0.0656

Now add one confidently wrong row: the true answer is 1 and the model said 0.02. Its Brier cost is $(0.02 - 1)^2 = 0.960$. Its log loss cost is $-\ln 0.02 = 3.912$.

Look at what each metric can charge for a single row. Brier is capped: the worst any one row can cost is 1.0, so this disaster costs 0.960 out of a possible 1.0. Log loss has no ceiling at all — as $p \rightarrow 0$ with $y = 1$, the cost runs to infinity, and 3.912 is already about eight times the worst of the other four rows. Push the prediction to $p = 0.001$ and Brier barely moves to 0.998, while log loss leaps to 6.9. That is the practical difference. If being confidently wrong is a catastrophe in your application, log loss encodes that and Brier does not.

7.9 Uncertainty and repeated experiments

A reported metric is an estimate, not a permanent property of a model. It varies with the sampled observations, training randomness, and hyperparameters. Useful practices include confidence intervals, bootstrap

resampling, repeated cross-validation, and multiple training seeds.

When comparing two models on the same test cases, paired differences are more informative than comparing two unrelated averages. A tiny improvement may not be practically important even if it is statistically detectable.

7.10 Error and subgroup analysis

Aggregate metrics can hide systematic failures. Inspect errors by class, label range, data source, time period, device type, location, and relevant demographic or operational groups.

Error analysis asks:

- Which examples have the largest or most costly errors?
- Are labels ambiguous, delayed, or incorrect?
- Are failures concentrated in rare or underrepresented groups?
- Does performance degrade as data becomes older or differs from training data?
- Are confident mistakes more common than uncertain mistakes?

The purpose is not only to describe failure but to decide whether better data, features, labels, thresholds, or modeling assumptions are needed.

7.11 Evaluation under distribution shift

Independent and identically distributed test data may not represent deployment. Covariate shift changes the input distribution, label shift changes class frequencies, and concept drift changes the relationship between features and labels.

Stress tests can evaluate missing features, corrupted inputs, rare groups, seasonal changes, and plausible worst cases. Once deployed, input statistics, prediction distributions, calibration, and delayed outcomes should be monitored. A model that was valid at launch can become unreliable as the environment changes.

Key idea

No single metric proves that a model is useful. Reliable evaluation combines a realistic split, a relevant baseline, task-specific metrics, uncertainty, subgroup analysis, and examination of actual errors.

Evaluation - Practical Checklist

Objective	Define the real decision and the cost of each kind of error.
Split	Match future use; respect time, groups, location, and repeated observations.
Baseline	Compare with a simple rule using the same data and metric.
Uncertainty	Use cross-validation, repeated splits, or confidence intervals where appropriate.
Diagnosis	Inspect errors, subgroups, calibration, threshold behavior, and distribution shift.
Final report	State the data period, metric definition, uncertainty, limitations, and untouched test result.

Check your understanding

1. Describe a situation where recall should matter more than precision, and another where precision should matter more than recall.
2. What usually happens to recall and false positives when a classification threshold is lowered?
3. Why must the cross-validation splitter preserve groups, time order, or class balance when those structures matter to deployment?

Chapter recap

- The correct metric depends on the practical objective and error costs.
- Regression metrics measure error size in different ways; classification metrics summarize different confusion-matrix trade-offs.
- Thresholds change precision and recall without retraining the probability model.
- Cross-validation compares models across multiple development splits while preserving the structure of the data.
- Calibration, subgroup analysis, uncertainty, and distribution shift are part of evaluation, not optional extras.

Training, Generalization, and Model Selection

Chapter at a glance

Before you start: Chapter 6 and Chapter 7: supervised objectives, realistic holdouts, and metric choice.

By the end, you can: diagnose underfitting and overfitting, control model capacity, choose hyperparameters without test leakage, set a safe imbalance policy, and reproduce the full selection process without allowing development decisions to leak into the final test evaluation.

Now that the supervised objective and evaluation procedure are fixed, learn the controls that apply to nearly all supervised model families. Training must reduce error without turning validation and test data into part of the learning process; model selection must choose complexity without rewarding accidental validation noise.

Course choice

This chapter is about the training and model-selection process, not about memorizing model families before their chapters arrive. Follow the workflow boundary first; Chapters 9–14 explain the algorithms themselves. Feature selection and imbalance interventions belong to model development, so this chapter places their workflow rules next to tuning and validation. Some methods make more sense after you meet lasso, trees, or nearest neighbors; when that happens, return to this section rather than attaching the method to one model family.

Core path

Core path: underfitting/overfitting, capacity, regularization, learning curves, hyperparameter tuning, imbalance policy, feature selection boundaries, and reproducibility. Optimization mechanics were introduced in Section 4.4; detailed optimizer behavior is intentionally deferred to the epilogue.

8.1 What training is optimizing: empirical risk

Training commonly minimizes average loss plus a regularization term:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_{\theta}(x_i)) + \lambda \Omega(\theta).$$

The first term fits the observed data; Ω discourages undesirable complexity. This objective is only a proxy for the true goal of low expected error on future data.

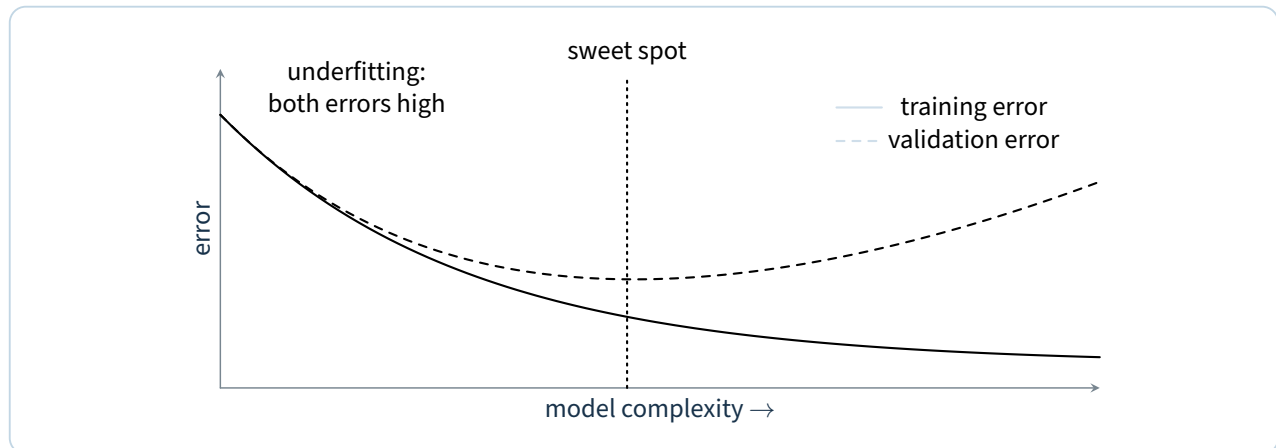
An optimization problem may have a unique global solution, many equivalent solutions, or numerous local and saddle regions. Reaching a low training loss does not guarantee good generalization.

8.2 Underfitting and overfitting

Underfitting The model is too simple to capture important patterns and performs poorly even on training data.

Overfitting The model learns training-specific noise and performs much worse on unseen data.

Good fit The model captures useful patterns and achieves similar, acceptable performance on training and validation data.

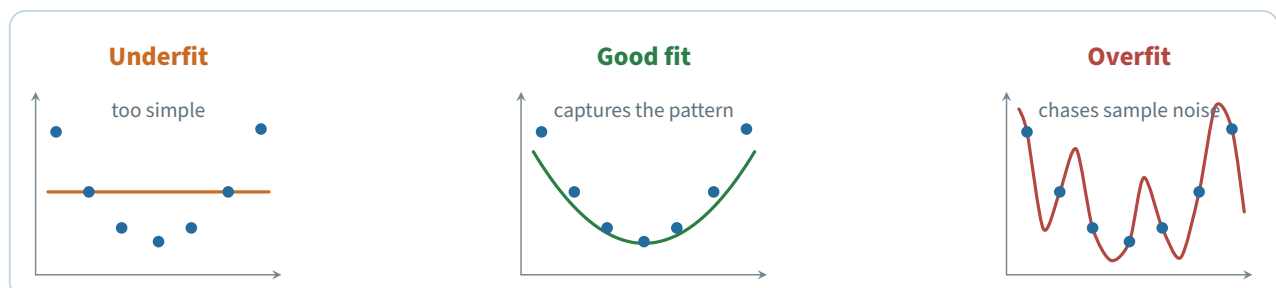


Plain idea

When model classes are nested and optimization succeeds, the best achievable training error cannot increase with flexibility. Validation error often falls and then rises as variance takes over, but real curves can be noisy or monotonic. Choose capacity from validation evidence and practical constraints, not from the training curve alone.

8.2.1 Underfit, good fit, and overfit on the same observations

The training-validation curves above are the diagnostic view. The geometric view below shows what those three regimes can look like in feature space.



Key idea

Overfitting is not “a curve is bad.” A curved relationship may be exactly right. Overfitting means the chosen flexibility follows accidents of the training sample that do not repeat on unseen data.

8.3 Model capacity and the bias-variance trade-off

Model capacity describes the complexity of functions a model can represent. A low-capacity model has high bias and may miss real structure. A high-capacity model can have high variance and respond strongly to noise or sampling changes.

For squared-error regression, the expected error at a point x decomposes exactly into three parts. Writing $\hat{f}$ for the model trained on a random dataset and f for the true function with noise variance σ^2 ,

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}.$$

Bias measures systematic error from a model that is too rigid; variance measures sensitivity to which particular training set was drawn; the noise term cannot be removed by any model. Regularization deliberately accepts a little bias to remove a lot of variance.

Worked example

The decomposition is easier to believe once you have computed it. Fix one test point where the true value is $f(x) = 2$ and the noise variance is $\sigma^2 = 1$. Train three models on three different random training sets and record what each predicts at that point.

Model	Predictions			bias ²	Variance	Total
	set 1	set 2	set 3			(+ $\sigma^2 = 1$)
Rigid	2.6	3.0	3.4	1.000	0.107	2.107
Balanced	1.0	2.0	3.0	0.000	0.667	1.667
Wild	-1.0	2.0	5.0	0.000	6.000	7.000

Work the first row through. The three predictions average to 3.0, and the truth is 2, so the bias is $3.0 - 2 = 1.0$ and $\text{bias}^2 = 1.0$. The spread around their own average is

$$\frac{(2.6 - 3)^2 + (3.0 - 3)^2 + (3.4 - 3)^2}{3} = \frac{0.16 + 0 + 0.16}{3} \approx 0.107,$$

and the total is $1.000 + 0.107 + 1 = 2.107$.

Two things are worth noticing. First, the rigid model is the most *consistent* of the three — its predictions barely move — and it is still beaten, because it is consistently aimed at the wrong place. Second, the wild model has *zero* bias: on average it is exactly right. Averaged over training sets it hits the truth perfectly, and it is by far the worst model here, because you never get the average — you get one training set and one prediction. Unbiasedness is not the goal. Low total error is.

Note also that $\sigma^2 = 1$ appears in every row and no model can touch it. If the best possible total error is 1.0, then a model scoring 1.667 is much closer to the ceiling than it looks.

Plain idea

Think of throwing darts. High bias, low variance: your darts land tightly grouped, but in the wrong corner of the board — consistent, yet consistently wrong. Low bias, high variance: your darts scatter widely around the bullseye — right on average, but unreliable on any single throw. The noise term is a wobbly dartboard: no amount of skill removes it.

More data generally reduces variance, while stronger assumptions can reduce variance at the cost of bias. The best model is not the most complicated one; it is the model whose assumptions and capacity produce the best reliable validation performance for the task.

8.4 Regularization

Regularization discourages unnecessarily large or complex parameter values. Two common penalties are

$$J_{L_1}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^p |\theta_j|,$$

$$J_{L_2}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^p \theta_j^2.$$

L_1 regularization can make some parameters exactly zero. L_2 regularization smoothly shrinks large parameters. The strength λ is normally selected with validation data.

Worked example

Why does L_1 hit exactly zero while L_2 only approaches it? Take a one-parameter least-squares problem written on a normalized scale so the unpenalized quadratic objective is $\frac{1}{2}(\beta - \hat{\beta}_{\text{OLS}})^2$, with $\hat{\beta}_{\text{OLS}} = 4$. Using $\lambda\beta^2/2$ for ridge and $\lambda|\beta|$ for lasso, the two penalized solutions have the closed forms:

$$\hat{\beta}_{\text{ridge}} = \frac{\hat{\beta}_{\text{OLS}}}{1 + \lambda}, \quad \hat{\beta}_{\text{lasso}} = \text{sign}(\hat{\beta}_{\text{OLS}}) \max(|\hat{\beta}_{\text{OLS}}| - \lambda, 0).$$

λ	Ridge	Lasso	What happened
0	4.000	4.000	no penalty; both give the OLS answer
1	2.000	3.000	ridge halves it; lasso subtracts a flat 1
5	0.667	0.000	ridge is small but alive; lasso has deleted the feature

Ridge *divides*, so no finite λ can ever reach zero — $4/(1 + \lambda)$ shrinks forever and arrives nowhere. Lasso *subtracts*, so once λ reaches 4 the coefficient hits zero and stays there. That single difference, division versus subtraction, is the whole of why lasso selects features and ridge does not.

8.4.1 Other ways to regularize

Regularization is broader than adding a penalty to the objective.

- **Early stopping** retains the model from the epoch with the best validation performance.
- **Data augmentation** adds meaningful input variation without changing labels.
- **Pruning** removes weak branches, features, or parameters.
- **Ensembling** averages several models to reduce variance.
- **Noise injection** trains the model to remain stable under small perturbations.

The correct method depends on why the model overfits. Regularization cannot repair a leaking split or a mismatch between training and deployment data.

8.5 Learning curves

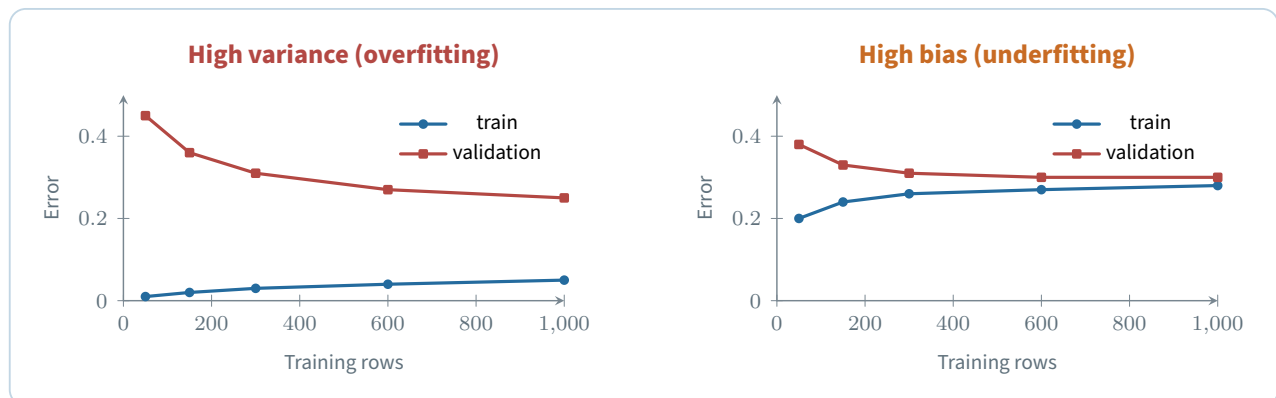
A learning curve plots training and validation performance against training-set size or training time.

- Poor training and validation performance suggests underfitting or an optimization problem.

- Strong training performance with weak validation performance suggests overfitting.
- Improvement in both curves as more data is added suggests that additional representative data may help.
- Highly unstable curves suggest sensitivity to the split, seed, or a small validation sample.

Learning curves distinguish problems that can be addressed by a larger model from those that need more data, better features, stronger regularization, or corrected labels.

8.5.1 Reading learning curves in practice



Key idea

Read the two curves as evidence. A wide, slowly closing gap suggests variance: more representative data, simpler capacity, or stronger regularization may help. Two curves that meet at a poor level suggest limited signal, optimization trouble, or underfitting; richer features or models are candidates, but more data can still help feature learning or reveal rare cases.

8.6 Hyperparameter tuning

Hyperparameters include regularization strength, tree depth, number of neighbors, learning rate, and model size. Common search methods are:

Method	Search strategy	Main consideration
Grid search	Evaluates every combination in a fixed Cartesian grid.	Becomes expensive as the number of dimensions and candidate values grows.
Random search	Samples combinations from the search space.	Often explores high-dimensional spaces more efficiently.
Sequential search	Uses earlier results to choose promising later experiments.	Concentrates the search on regions that appear useful.

The validation procedure must be identical across candidates. Repeatedly adapting decisions to one validation set can overfit that set, so a final untouched test set remains necessary.

8.6.1 Tuning without fooling yourself

Plain idea

Random search is often more efficient than a Cartesian grid when only a few of many hyperparameters matter, because every trial explores a new value on each dimension. A small grid remains useful for one or two well-understood controls. Neither method compensates for a flawed validation design.

8.6.2 Nested cross-validation for the selection procedure

Each outer validation fold is untouched by the tuning performed inside that outer training split. This estimates the performance of the *whole selection procedure* with less selection bias; it is not literally unbiased in every finite sample, and its fold scores are still dependent.

8.7 Imbalanced classification: change the policy, not the test set

Class imbalance changes what a useful training policy looks like. Accuracy can be misleading. A model that always predicts the common class may look accurate while completely missing the rare class. Possible responses include class weights, careful training-set resampling, threshold adjustment, and metrics such as recall, precision, balanced accuracy, or area under the precision-recall curve.

Four responses are worth knowing by name, roughly in the order you should try them.

Strategy	How it helps
Change the metric first	This costs nothing and is often the whole fix. If the rare class is what matters, stop reporting accuracy and report recall, precision, balanced accuracy, or average precision instead (Chapter 7).
Move the threshold	A probability model does not have to decide at 0.5. Choosing a lower cutoff on validation data raises recall directly, with no retraining at all.
Resampling	Either drop majority rows (undersampling) or create extra minority training examples (oversampling). SMOTE is a common method that creates synthetic minority examples between nearby minority observations; it is useful only inside the training portion of each validation split.

Course choice

SMOTE uses nearest-neighbor geometry, which Chapter 11 develops in detail. You do not need that geometry yet to learn the workflow rule: split first, and create synthetic examples only from the training portion of a fold. Never oversample the full labeled table before validation, because synthetic points can then carry information from held-out rows into training.

Common mistake

Resampling changes the training distribution. It does *not* justify changing the validation or test distribution to look balanced. Keep evaluation prevalence realistic unless the deployment question itself defines a different population of interest.

8.8 Feature selection is part of model development

Feature selection keeps a subset of the original variables. It can reduce measurement cost, training time, noise, and explanation burden, but it is not automatically beneficial. A regularized model with a few dozen

sensible features often needs no separate selector.

The important conceptual distinction is between *selection* and *extraction*. Feature selection keeps original columns; PCA in Chapter 17 creates new component features.

Family	How it works	Main trade-off
Filter methods	Score columns before fitting the final model, using variance, correlation, statistical tests, or mutual information.	Fast and model-agnostic, but may miss interactions between features.
Wrapper methods	Repeatedly fit a chosen model on different feature subsets; recursive feature elimination is a common example.	Can adapt to a model family, but is computationally expensive.
Embedded methods	Selection happens during model fitting. Lasso can drive coefficients to zero; tree-based models can support importance-based selection.	Efficient, but the chosen subset inherits the assumptions and instability of the fitted model.

Second pass

After Chapter 9, revisit lasso as an embedded selector. After Chapter 12 and Chapter 13, revisit importance-based selection. After Chapter 11, revisit why irrelevant dimensions can damage distance-based models. The workflow rule remains unchanged across all of them.

Common mistake

Supervised feature selection sees the label, so fitting a selector once on the complete labeled table before cross-validation leaks information. Put the selector inside the same pipeline that is validated with the model. If the number of selected features is tuned, that choice is a hyperparameter too.

Key idea

Ask three questions before selecting features: **Do I have enough features for selection to matter? Is the goal predictive performance, cheaper data collection, or interpretability? Can I validate the entire selection-plus-model procedure without touching the final test set?**

Debugging task

A researcher selects the top 20 features using all labeled rows, applies SMOTE to the full table, then runs cross-validation and reports the best result. Redesign the experiment so splitting, selection, resampling, and tuning respect the information boundary.

8.9 Reproducible training

A reproducible experiment records:

- data and preprocessing versions;
- training, validation, and test identifiers;
- software and hardware environment;
- model specification and configuration;
- hyperparameters, random seeds, and stopping rules;
- training curves, checkpoints, and evaluation outputs.

Exact numerical reproduction may be difficult on parallel hardware, but the conclusions should remain stable across reasonable reruns. Report variation rather than presenting one favorable seed as definitive.

Key idea

Optimization asks whether the model can fit the training objective. Generalization asks whether the learned solution works on new data. Successful machine learning requires both.

Training and Tuning - Practical Checklist

Objective	Define the training objective and know which parts are learned from data; optimizer details are second-pass when the estimator exposes them.
Diagnosis	Compare training and validation curves to identify underfitting or overfitting.
Regularization	Limit model capacity with penalties, pruning, simpler features, or early stopping.
Tuning	Search hyperparameters using validation data or nested cross-validation, not the final test set.
Reproducibility	Record the data version, split logic, preprocessing, model specification, random seeds, software versions, and hardware-relevant settings.

8.9.1 A short reproducibility checklist

✓ Record this before you call a model finished

- 1 The exact data snapshot, with a date and a row count.
- 2 Every random seed used.
- 3 Library versions.
- 4 The complete preprocessing and modeling procedure, not only the final estimator.
- 5 The chosen hyperparameters and the search space they came from.
- 6 The metric, the split strategy, and the final test score.
- 7 Known limitations: who the model is likely to be wrong about.

Key idea

Item 7 is easy to omit. Models are most uncertain on populations poorly represented in development data, and subgroup analysis may expose other failures. Recording known limits is part of responsible deployment, not an admission that the project failed.

Try it yourself

Before opening the test set, repeat the complete training and validation procedure across several stochastic seeds using the same prespecified splits. Report how predictions and validation metrics vary. Lock the seed policy and model choice, refit as planned, and evaluate the test set once; do not repeatedly probe test sensitivity.

Debugging task

A hyperparameter search reports its best configuration, then the researcher reruns the search with a narrower grid chosen after inspecting the test score. Explain how the test set entered training decisions and redesign the selection/final-evaluation boundary.

Decision memo

Two models have nearly equal mean validation performance. One has lower fold-to-fold variation, half the inference latency, and simpler monitoring. Decide which to keep and state why a tiny mean-score advantage is not automatically decisive.

Try it yourself

Choose one supervised pipeline and one fixed cross-validation design. Plot or tabulate training versus validation performance as model capacity changes. Then tune only the development data, lock the final procedure, and write down exactly which steps would be refitted before a one-time test evaluation.

PRACTICAL MILESTONE

Lab 3 of 10

Supervised workflow, evaluation, and model selection

From a no-skill baseline to a locked final model

Coverage. Chapters 6–8.

Source files. Lab03/Task01.ipynb through Task04.ipynb.

Core data. Titanic and Breast Cancer Wisconsin. Heart Failure is an optional transfer activity inside the same lab.

Goal. Compare supervised baselines fairly, inspect threshold-dependent errors and ranking metrics, tune only on development data, and preserve an untouched final test boundary.

How much is scaffolded?

This is the course's main workflow lab. Do not treat baseline comparison, threshold policy, cross-validation, and final evaluation as separate labs; they are successive decisions in one supervised-learning process.

Part A — Supervised baselines on Titanic.

```
from pathlib import Path
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "titanic_dataset" / "Titanic-Dataset.csv"
df = pd.read_csv(train_path)
```

```
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = df[features]
y = df["Survived"]

num_cols = ["Age", "SibSp", "Parch", "Fare"]
cat_cols = ["Pclass", "Sex", "Embarked"]
preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
models = {
    "logistic": LogisticRegression(max_iter=1000),
    "random_forest": RandomForestClassifier(
        n_estimators=300, random_state=42, min_samples_leaf=2
    ),
}

for name, estimator in models.items():
    pipe = Pipeline([("preprocess", preprocess), ("model", estimator)])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_valid)
    prob = pipe.predict_proba(X_valid)[: , 1]
    print(name, "accuracy=", round(accuracy_score(y_valid, pred), 3),
          "roc_auc=", round(roc_auc_score(y_valid, prob), 3))
```

Part B — Thresholds and ranking metrics.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    average_precision_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    RocCurveDisplay,
    PrecisionRecallDisplay,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "breast_cancer_dataset" / "data.csv"
df = pd.read_csv(train_path)
```

```
X = df.drop(columns=["id", "diagnosis", "Unnamed: 32"], errors="ignore")
y = (df["diagnosis"] == "M").astype(int)

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
model = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000)),
])
model.fit(X_train, y_train)
prob = model.predict_proba(X_valid)[: , 1]
```

```
# Explore threshold policies on development validation data, not a final test set.
for threshold in [0.50, 0.30]:
    pred = (prob >= threshold).astype(int)
    print(f"\nthreshold={threshold:.2f}")
    print(confusion_matrix(y_valid, pred))
    print(classification_report(y_valid, pred, digits=3))
    ConfusionMatrixDisplay.from_predictions(y_valid, pred)
    plt.title(f"Confusion matrix at threshold {threshold:.2f}")
    plt.tight_layout()
    plt.show()
```

```
print("ROC AUC:", round(roc_auc_score(y_valid, prob), 3))
print("PR AUC:", round(average_precision_score(y_valid, prob), 3))
```

```
RocCurveDisplay.from_predictions(y_valid, prob)
plt.title("ROC curve")
plt.tight_layout()
plt.show()
```

```
PrecisionRecallDisplay.from_predictions(y_valid, prob)
plt.title("Precision-recall curve")
plt.tight_layout()
plt.show()
```

Part C — Optional heart-failure transfer.

```
from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    PrecisionRecallDisplay,
    RocCurveDisplay,
    average_precision_score,
    classification_report,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "heart_failure_dataset" / "heart.csv"
df = pd.read_csv(train_path)
```

```
X = df.drop(columns="HeartDisease")
y = df["HeartDisease"]

num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(exclude="number").columns.tolist()
preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
model = Pipeline([
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=2000)),
])
model.fit(X_train, y_train)
prob = model.predict_proba(X_valid)[: , 1]
pred = (prob >= 0.50).astype(int)

print(classification_report(y_valid, pred, digits=3))
print("ROC AUC:", round(roc_auc_score(y_valid, prob), 3))
print("PR AUC:", round(average_precision_score(y_valid, prob), 3))
```

```
ConfusionMatrixDisplay.from_predictions(y_valid, pred)
plt.title("Heart-disease confusion matrix")
plt.tight_layout()
plt.show()
```

```
RocCurveDisplay.from_predictions(y_valid, prob)
plt.title("Heart-disease ROC curve")
plt.tight_layout()
plt.show()
```

```
PrecisionRecallDisplay.from_predictions(y_valid, prob)
plt.title("Heart-disease precision-recall curve")
plt.tight_layout()
plt.show()
```

Part D — Cross-validated model selection.

```
from pathlib import Path
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "titanic_dataset" / "Titanic-Dataset.csv"
df = pd.read_csv(train_path)
```

```
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = df[features]
y = df["Survived"]
```

```
num_cols = ["Age", "SibSp", "Parch", "Fare"]
cat_cols = ["Pclass", "Sex", "Embarked"]
preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])
```

```
pipe = Pipeline([
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=2000)),
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
search = GridSearchCV(
    pipe,
    param_grid={
        "model__C": [0.01, 0.1, 1.0, 10.0, 100.0],
        "model__class_weight": [None, "balanced"],
    },
    scoring="roc_auc",
    cv=cv,
```

```
n_jobs=-1,  
)  
search.fit(X_train, y_train)
```

```
# Only after selection is complete do we evaluate once on the untouched test set.  
test_prob = search.predict_proba(X_test)[: , 1]  
print("best parameters:", search.best_params_)  
print("training-CV ROC AUC:", round(search.best_score_, 3))  
print("final test ROC AUC:", round(roc_auc_score(y_test, test_prob), 3))
```

Decision memo

Choose a classification threshold for a setting in which false negatives cost substantially more than false positives. State the consequence of lowering the threshold and name the validation evidence you used.

Lab deliverable

Submit a comparison against a no-skill baseline, confusion-matrix and ROC/precision-recall evidence for at least two thresholds, the cross-validated selection result, and one final untouched-test result after all choices are locked. The Heart Failure transfer activity is optional and does not count as another lab.

Check your understanding

1. What train-versus-validation pattern suggests underfitting, and what pattern suggests overfitting?
2. How can regularization improve validation performance even while making the training fit worse?
3. Which decisions must be locked before the final test set is evaluated?

Chapter recap

- Underfitting, good fit, and overfitting are different relationships between model capacity and unseen-data performance.
- Regularization trades some training fit for a simpler rule that may generalize better.
- Learning curves help distinguish data shortage, insufficient capacity, and persistent training-validation gaps.
- Hyperparameter tuning and class-imbalance interventions are model-development choices and must stay inside the validation process; feature selection and resampling are model-development steps and must stay inside the validation process.
- The final test set evaluates the locked procedure; it does not choose thresholds, resampling rules, hyperparameters, or later feature-selection rules.
- Reproducibility requires recording data, split logic, preprocessing, search space, seeds, versions, and the final pipeline.

Stage 3B

Supervised Model Families

Hold the workflow fixed and change one model family at a time so comparisons stay meaningful.

Linear Models for Regression

Chapter at a glance

Before you start: Chapter 6, Chapter 8, and vectors/matrices from Chapter 5.

By the end, you can: Fit and interpret linear regression, compute a small fit by hand, extend the model with polynomial features, control overfitting with ridge/lasso/elastic net, and recognize when a robust, quantile, count, or Tweedie loss better matches the label.

Stage 3B begins here: the supervised workflow is now fixed, so the remaining chapters change the model family rather than the evaluation rules. Linear regression is one of the oldest and most interpretable supervised algorithms, and it is a valuable baseline for many regression problems. The general training ideas from Chapter 8 now become concrete: regularization, model capacity, and optimization can all be seen directly in a familiar regression model.

Core path

Read linear regression as “base amount plus contributions.” A house begins with a base price, then area, bedrooms, age, and location each add or subtract an amount. The coefficients are those per-feature contributions; fitting finds values that make past predictions reasonably close to past answers.

9.1 The model

Linear regression predicts a continuous value using a weighted sum of features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

The intercept β_0 is the prediction when every feature equals zero. A coefficient β_j describes how the prediction changes when x_j increases by one unit while the other features remain fixed.

Plain idea

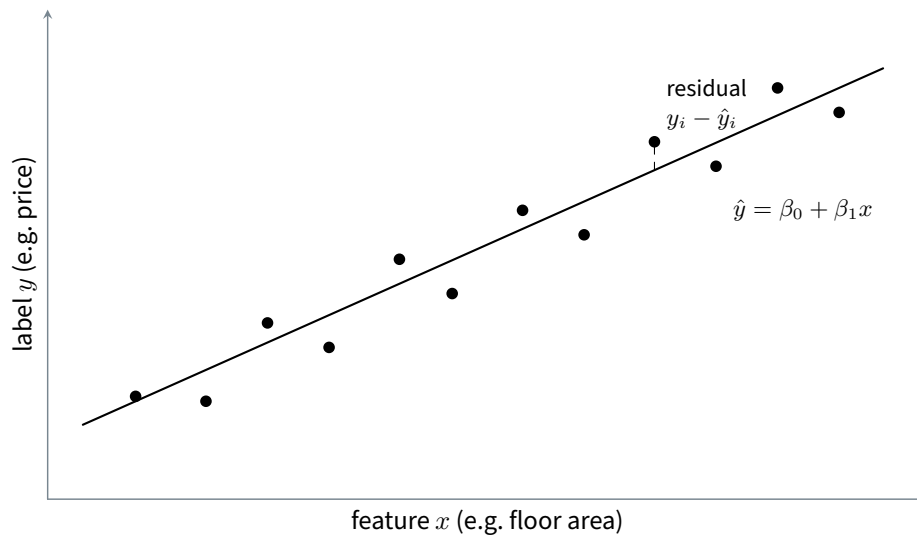
The model is a recipe with adjustable amounts: “a house’s predicted price is a base amount, plus so-much per square meter, plus so-much per bedroom, minus so-much per year of age.” Training does nothing more than find the amounts that make the recipe’s outputs match the known prices as closely as possible.

9.2 Least squares: how the line is chosen

Ordinary least squares (OLS) chooses coefficients that minimize the residual sum of squares:

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

A **residual** $y_i - \hat{y}_i$ is the vertical gap between a data point and the fitted line — the part of the label the model failed to explain.



Plain idea

Imagine each data point pulling on the line with a rubber band whose strength grows with the square of its length. Least squares finds the line where all these pulls balance out — the line with the smallest total squared vertical gaps.

With a single feature, the best-fitting slope and intercept have simple closed forms:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

The slope is the covariance between x and y divided by the variance of x — “how much they move together, per unit of how much x moves on its own.” The intercept then forces the line through the point of means $(\bar{x}, \bar{y})$.

Worked example

Fit a line to three points by hand: $(1, 2), (2, 3), (3, 5)$. The means are $\bar{x} = 2$ and $\bar{y} = 10/3$. The deviations from the means are $(-1, 0, 1)$ for x and $(-\frac{4}{3}, -\frac{1}{3}, \frac{5}{3})$ for y . Then

$$\hat{\beta}_1 = \frac{(-1)(-\frac{4}{3}) + (0)(-\frac{1}{3}) + (1)(\frac{5}{3})}{(-1)^2 + 0^2 + 1^2} = \frac{3}{2} = 1.5, \quad \hat{\beta}_0 = \frac{10}{3} - 1.5 \times 2 = \frac{1}{3} \approx 0.33.$$

The fitted line is $\hat{y} = 0.33 + 1.5x$, giving predictions 1.83, 3.33, and 4.83 with residuals +0.17, -0.33, and +0.17. The residuals sum to zero — a general property of least squares with an intercept.

9.3 Many features: the matrix view

With the intercept absorbed into a leading column of ones in the design matrix $X \in \mathbb{R}^{n \times (p+1)}$, the objective and its gradient are

$$J(\beta) = \frac{1}{n} \|X\beta - \mathbf{y}\|_2^2, \quad \nabla_{\beta} J = \frac{2}{n} X^T (X\beta - \mathbf{y}).$$

Setting the gradient to zero gives the **normal equations** $X^T X \beta = X^T \mathbf{y}$, whose solution (when $X^T X$ is invertible) is

$$\hat{\beta} = (X^T X)^{-1} X^T \mathbf{y}.$$

Two practical routes reach the same answer: solve a linear system using a stable matrix factorization, or run gradient descent on J (Chapter 8), which can scale to very large datasets. Forming an explicit inverse is mainly a teaching device; numerical software normally avoids it.

9.3.1 Reading the coefficients honestly

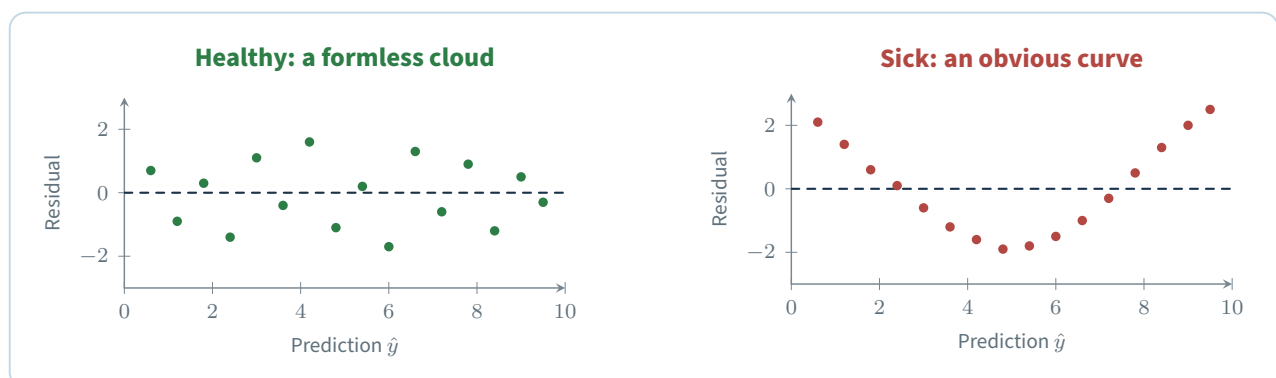
- Coefficients are in the label's units per unit of the feature, so their sizes are not comparable unless the features are standardized first.
- “Holding the other features fixed” is a mathematical statement, not a physical one. If floor area and bedroom count always grow together in the data, the model cannot cleanly separate their effects, and the individual coefficients become unstable even when predictions stay good. This is **multicollinearity**.
- Coefficients are associations, not causal effects (Section 5.9); adding or removing one feature can change the sign of another coefficient.

9.3.2 The assumptions, in plain words

OLS works best when: the relationship is approximately linear in the chosen features; the errors are independent of one another; the error size does not systematically grow or shrink with the prediction; and no feature is an exact combination of others. Violations do not make the model useless — they make the coefficient interpretations and classical uncertainty estimates unreliable, and they are usually visible in a plot of residuals against predictions.

9.4 Reading residuals: the diagnostic that matters most

Fitting a line is easy. Knowing whether the line was the right idea requires looking at what is left over. A **residual** is simply $y - \hat{y}$: the part the model failed to explain.



Key idea

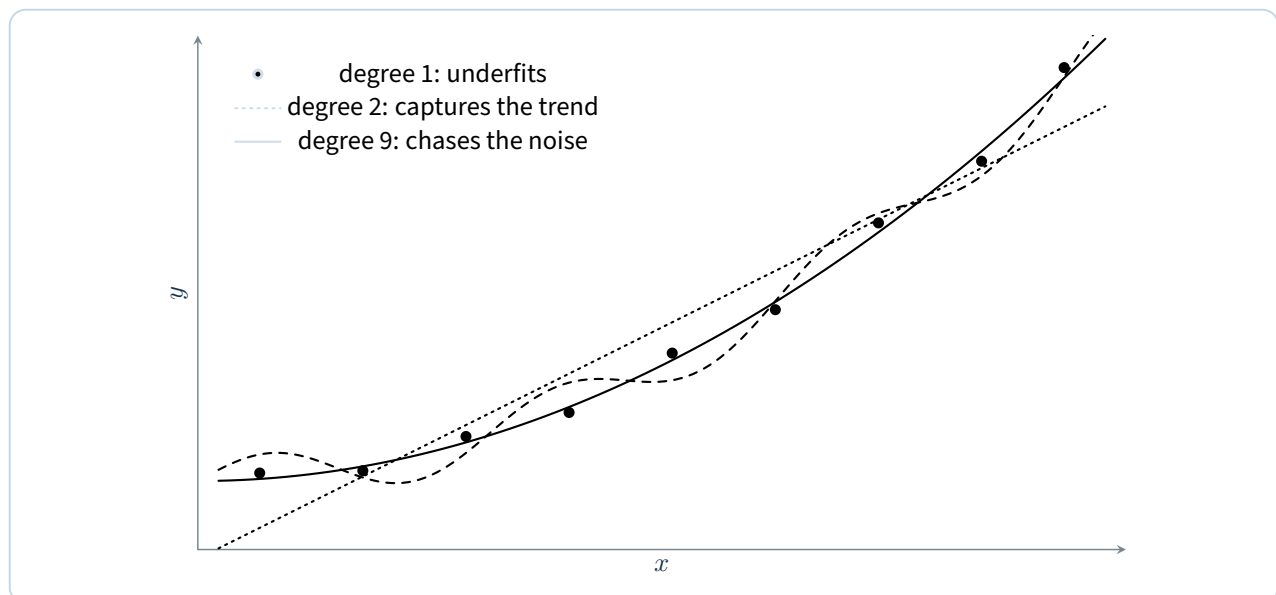
Residuals should look like structureless noise. Any visible pattern is information the model failed to use. A U-shape means the true relationship curves and you need a polynomial term or a nonlinear model. A widening funnel means the errors grow with the prediction, which often calls for predicting $\log y$ instead of y .

9.5 Polynomial regression: curves from a linear model

Linear regression is linear *in the parameters*, not necessarily in the raw inputs. Creating new features such as x^2 , x^3 , or x_1x_2 and fitting the same machinery yields curved predictions:

$$\hat{y} = \beta_0 + \beta_1x + \beta_2x^2 + \cdots + \beta_dx^d.$$

This flexibility is powerful and dangerous in equal measure: the degree d is a capacity dial, and turning it too far produces a curve that chases the noise.



Plain idea

A degree-1 model is a ruler forced through curved data: wrong everywhere in a systematic way. A degree-9 model is a wet noodle that threads through every point, wiggles included — perfect on the training data and unreliable between and beyond the points. The honest model sits in the middle, and validation data (not training error) is how you find it.

Polynomial features grow quickly — degree 3 with 10 raw features already creates hundreds of terms — which raises the next question: how can a model with many features be kept under control?

9.6 Ridge regression: shrink everything a little

Ridge regression adds an L_2 penalty on coefficient size to the least-squares objective. For centered features and label (so the intercept has been removed), its solution is

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \|X\beta - \mathbf{y}\|_2^2 + \lambda \|\beta\|_2^2 = (X^T X + \lambda I)^{-1} X^T \mathbf{y}.$$

Reading the objective: the first term wants to fit the data, the second term charges rent on large coefficients, and $\lambda \geq 0$ sets the rent. With $\lambda = 0$ ridge is ordinary least squares; as λ grows, all penalized coefficients shrink smoothly toward zero. For $\lambda > 0$, $X^T X + \lambda I$ is invertible in this centered formulation. In the usual model with an intercept, software normally leaves that intercept unpenalized and reconstructs it from the feature and label means.

9.7 Lasso: shrink some coefficients to exactly zero

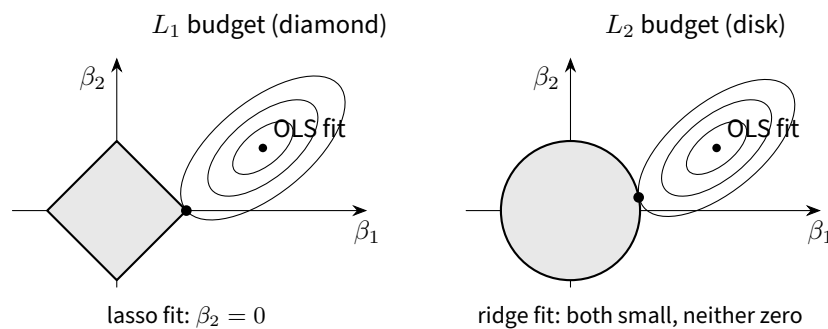
Lasso regression swaps the penalty for an L_1 norm:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \|X\beta - \mathbf{y}\|_2^2 + \lambda \|\beta\|_1.$$

The change looks small but has a striking consequence: as λ grows, lasso sets weak coefficients *exactly* to zero, removing those features from the model entirely. Lasso therefore performs automatic feature selection and produces sparse, readable models. Its main caution: among a group of strongly correlated features, lasso tends to keep one and discard the rest somewhat arbitrarily.

9.7.1 Why the penalties behave differently

Minimizing a penalized objective is equivalent to minimizing the data fit subject to a budget on the coefficients. The L_1 budget region is a diamond with sharp corners on the axes; the L_2 budget region is a smooth disk. The best constrained solution is where the elliptical contours of the least-squares loss first touch the budget region — and contours naturally strike a diamond at a corner, where some coefficients are exactly zero.



Plain idea

Regularization is like attaching a spring that pulls every coefficient toward zero. The data pulls coefficients toward whatever fits best; the spring pulls back. Ridge uses a soft spring that shrinks everything a little; lasso uses a stiff one that snaps weak coefficients all the way to zero, effectively deleting those features.

9.8 Elastic net and practical guidance

Elastic net combines both penalties,

$$\lambda(\rho \|\beta\|_1 + (1 - \rho) \|\beta\|_2^2),$$

gaining lasso's sparsity while sharing weight across correlated features like ridge. The mixing parameter ρ and strength λ are both tuned on validation data.

Working habits that apply to every model in this chapter:

- **Standardize the features first.** The penalties treat all coefficients equally, which is only fair if the features are on comparable scales. The intercept is normally left unpenalized.
- **Tune λ with cross-validation** (Chapter 7), searching over a logarithmic grid such as $10^{-3}, 10^{-2}, \dots, 10^2$.
- **Default choices:** ridge when most features plausibly matter a little; lasso when you expect only a few to matter and want to know which; elastic net when features come in correlated groups.
- For data with heavy outliers, replacing squared error with a robust alternative such as the **Huber loss** — quadratic for small errors, linear for large ones — keeps a few extreme points from dominating the fit.

Key idea

Linear regression finds the best weighted-sum predictor; feature transformations let that “line” bend; and regularization keeps the added flexibility honest by charging for large coefficients. This trio — a simple model, richer features, and a complexity penalty tuned on validation data — is the template that the rest of machine learning elaborates.

Linear Regression Family - Quick Guide

Best when	The label is numerical, a weighted-sum relationship is a useful approximation, and interpretability matters.
Start with	Ordinary linear regression, residual plots, and a mean/median baseline.
Preprocessing	Encode categories; standardize before ridge, lasso, or elastic net; inspect skew and outliers.
Main controls	Polynomial degree, regularization strength λ , and elastic-net mixing parameter.
Watch for	Nonlinearity, influential outliers, multicollinearity, leakage, and unsafe extrapolation beyond the training range.

9.9 When ordinary squared error is the wrong model for the label

Ordinary least squares is a strong baseline for a continuous label, but the label's data-generating structure can justify a different loss or distribution. Do not choose these models because they sound advanced; choose them because the outcome and decision make their assumptions useful.

Label structure	Useful starting point	Main caution
Continuous values with occasional large errors	Robust regression loss	Robustness changes which errors receive the most influence; validate against the real evaluation objective.
Conditional quantiles matter	Quantile regression	Different quantiles answer different questions and should not be interpreted as conditional means.
Non-negative counts	Count-aware generalized linear model	The chosen distributional assumptions must be plausible for the observed mean–variance relationship.
Positively skewed outcomes	Log-scale modeling or an appropriate positive-response model	Back-transformation changes the interpretation of average predictions and errors.

Use the same split and validation discipline from Chapter 7. A specialized loss is useful only when it improves the metric that represents the real task and its predictions remain plausible.

PRACTICAL MILESTONE

Lab 4 of 10

Regression and time-aware validation

Linear models under ordinary and chronological validation

Coverage. Chapter 9.

Source files. Lab04/Task01.ipynb, Task02.ipynb.

Core data. Medical Cost Personal Data. Bike Sharing is a transfer activity within the same lab.

Goal. Compare ordinary linear regression with ridge regularization, inspect residual behavior, and contrast random splitting with a chronological holdout when time ordering matters.

How much is scaffolded?

The main learning objective is not a particular regression score. It is choosing a validation design that matches how future predictions will actually be made.

Part A — Linear and ridge regression.

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "medical_cost_personal_dataset" / "insurance.csv"
df = pd.read_csv(train_path)
```

```
X = df.drop(columns="charges")
y = df["charges"]
num_cols = ["age", "bmi", "children"]
cat_cols = ["sex", "smoker", "region"]

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.20, random_state=42
)

predictions = {}
```

```
for name, estimator in {
```

```

    "ordinary least squares": LinearRegression(),
    "ridge": Ridge(alpha=10.0),
}.items():
    model = Pipeline([("preprocess", preprocess), ("model", estimator)])
    model.fit(X_train, y_train)
    pred = model.predict(X_valid)
    predictions[name] = pred
    print("\n", name)
    print("MAE:", round(mean_absolute_error(y_valid, pred), 2))
    print("RMSE:", round(np.sqrt(mean_squared_error(y_valid, pred)), 2))
    print("R2:", round(r2_score(y_valid, pred), 3))

```

```

# Visual diagnostics for the regularized model.
ridge_pred = predictions["ridge"]
plt.figure(figsize=(6, 4))
plt.scatter(y_valid, ridge_pred, alpha=0.65)
lims = [min(y_valid.min(), ridge_pred.min()), max(y_valid.max(), ridge_pred.max())]
plt.plot(lims, lims, linestyle="--")
plt.xlabel("Actual charges")
plt.ylabel("Predicted charges")
plt.title("Ridge validation: actual versus predicted")
plt.tight_layout()
plt.show()

```

```

residual = y_valid.to_numpy() - ridge_pred
plt.figure(figsize=(6, 4))
plt.scatter(ridge_pred, residual, alpha=0.65)
plt.axhline(0, linestyle="--")
plt.xlabel("Predicted charges")
plt.ylabel("Residual")
plt.title("Ridge validation residual plot")
plt.tight_layout()
plt.show()

```

Part B — Optional chronological-validation transfer.

```

from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

```

```

PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "bike_sharing_dataset" / "day.csv"
df = pd.read_csv(train_path)
df["dteday"] = pd.to_datetime(df["dteday"])
df = df.sort_values("dteday").reset_index(drop=True)

```

```

# cnt = casual + registered, so casual and registered would leak the label.
features = [
    "season", "yr", "mnth", "holiday", "weekday", "workingday",
    "weathersit", "temp", "atemp", "hum", "windspeed",
]
X = df[features]
y = df["cnt"]

```

```

# Chronological development holdout: earlier dates train, later dates validate.
cut = int(len(df) * 0.80)
X_train, X_valid = X.iloc[:cut], X.iloc[cut:]
y_train, y_valid = y.iloc[:cut], y.iloc[cut:]
dates_valid = df["dteday"].iloc[cut:]

```

```

cat_cols = ["season", "yr", "mnth", "holiday", "weekday", "workingday", "weathersit"]
num_cols = ["temp", "atemp", "hum", "windspeed"]
preprocess = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ("num", "passthrough", num_cols),
])
model = Pipeline([
    ("preprocess", preprocess),
    ("model", RandomForestRegressor(
        n_estimators=400, min_samples_leaf=2, random_state=42, n_jobs=-1
    )),
])
model.fit(X_train, y_train)
pred = model.predict(X_valid)

print("MAE:", round(mean_absolute_error(y_valid, pred), 1))
print("RMSE:", round(np.sqrt(mean_squared_error(y_valid, pred)), 1))

```

```

plt.figure(figsize=(8, 4))
plt.plot(dates_valid, y_valid.to_numpy(), label="actual")
plt.plot(dates_valid, pred, label="predicted")
plt.xlabel("Date")
plt.ylabel("Daily rentals")
plt.title("Chronological validation: actual versus predicted demand")
plt.legend()
plt.tight_layout()
plt.show()

```

Lab deliverable

Submit linear-versus-ridge validation metrics, one residual diagnostic, and a short comparison between random and chronological validation on the bike-sharing transfer activity. Explain which split better matches deployment and why.

Check your understanding

1. What does a regression coefficient mean when the other modeled features are held fixed, and why is that not automatically a causal effect?
2. How do ridge and lasso change coefficient estimates differently as regularization becomes stronger?
3. How can an interaction or polynomial feature let a model remain linear in its parameters while representing a nonlinear relationship in the original inputs?

Chapter recap

- Linear regression predicts a weighted sum of features.
- Least squares chooses coefficients that minimize squared residuals.
- Polynomial features create curved relationships while the model remains linear in its coefficients.
- Ridge shrinks all coefficients; lasso can set some coefficients to zero; elastic net combines both behaviors.
- Robust, quantile, Poisson, and Tweedie models are useful when the label structure justifies a loss beyond ordinary squared error.
- Validation data should choose feature complexity, regularization strength, and whether a specialized label model actually helps.

Linear Models for Classification

Chapter at a glance

Before you start: Chapter 6, Chapter 8, and probability/odds from Chapter 5.

By the end, you can: Fit and interpret logistic regression, extend it to many classes with softmax, explain how discriminant analysis reaches a linear boundary from a different direction, recognize ordered-category labels, and place the perceptron in its historical role.

Classification asks the model to choose a category. The simplest useful family of classifiers draws a straight line (in higher dimensions, a flat plane) through feature space and predicts one class on each side. This chapter covers the three classical ways to find that line: logistic regression, discriminant analysis, and the perceptron.

Core path

Core path: logistic regression, cross-entropy, decision thresholds, and softmax regression. Ordinal labels are a useful special case. LDA and the perceptron are comparison models for a second pass rather than prerequisites for later chapters.

10.1 Why not just use linear regression?

A tempting shortcut is to represent the classes as $y \in \{0, 1\}$ and fit ordinary least squares. It fails in two instructive ways: the model happily predicts values like -0.3 or 1.7 , which cannot be probabilities, and squared error punishes the model for being *too correct* — an extremely obvious positive example far from the boundary pulls the line toward itself just as an outlier would. Classification needs an output that lives in $[0, 1]$ and a loss that rewards confident correct answers.

10.2 From probabilities to odds to a linear model

If an event has probability p , its **odds** are $p/(1 - p)$: a probability of 0.75 means odds of 3 (“three to one”). Odds run from 0 to infinity, and their logarithm — the **log-odds** or **logit** — runs over the whole real line. Logistic regression makes one clean assumption: *the log-odds are a linear function of the features*,

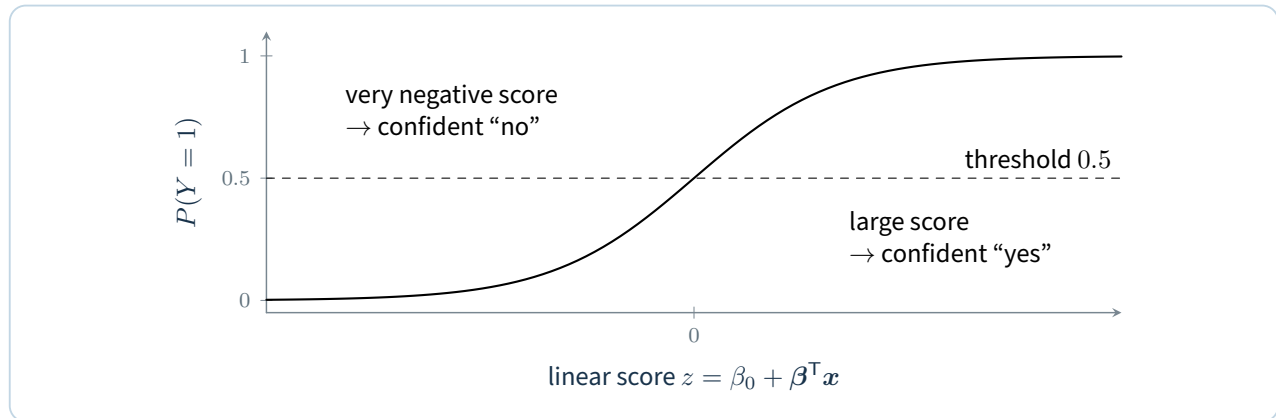
Probability	Odds	Read aloud
50%	1 : 1	equally likely either way
75%	3 : 1	three chances for, one against
90%	9 : 1	nine chances for, one against

Logistic regression uses log-odds because an unrestricted score can then be converted into a valid probability between 0 and 1 . You do not normally calculate the logarithm by hand; the library performs the conversion.

$$\log \frac{p}{1-p} = \beta_0 + \beta^\top \mathbf{x}.$$

Solving for p inverts the logarithm and produces the familiar S-shaped **sigmoid**:

$$P(Y = 1 | \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta^\top \mathbf{x}.$$



Plain idea

The sigmoid is a “squashing” function: it takes any score, from minus infinity to plus infinity, and gently bends it into the range 0 to 1 so it can be read as a probability. Scores near zero mean the model is unsure; scores far from zero mean it is confident.

The coefficients have a crisp interpretation: increasing x_j by one unit multiplies the odds by e^{β_j} . A coefficient of 0.7 roughly doubles the odds ($e^{0.7} \approx 2.01$); a coefficient of -0.7 roughly halves them.

Worked example

Suppose a pass/fail model based on hours studied has $\beta_0 = -4$ and $\beta_1 = 0.08$. A student with $x = 60$ hours has score $z = -4 + 0.08 \times 60 = 0.8$ and pass probability $\sigma(0.8) \approx 0.69$. A student with $x = 30$ hours has $z = -1.6$ and $\sigma(-1.6) \approx 0.17$. Each additional hour multiplies the odds of passing by $e^{0.08} \approx 1.083$ — about an 8% increase in the odds, regardless of the starting point.

10.3 Training with cross-entropy

For binary classification with predicted probability p , the loss for one observation is the **cross-entropy** (equivalently, the negative log-likelihood of a Bernoulli model from Chapter 5):

$$\mathcal{L}(y, p) = -[y \log p + (1 - y) \log(1 - p)].$$

When $y = 1$ the loss is $-\log p$: near zero if the model was confident and right, and enormous if the model was confident and wrong. The gradient of the average loss has a strikingly simple form,

$$\nabla_{\beta} J = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) \mathbf{x}_i,$$

so each update is driven directly by the prediction error $p_i - y_i$. There is no closed-form solution; the parameters are found by iterative optimization such as gradient descent or Newton-type methods (Chapter 8). With the usual convex objective and an appropriate solver, optimization is not trapped by inferior local minima.

10.3.1 The decision boundary and the threshold

Predicting class 1 whenever $p \geq 0.5$ is the same as predicting class 1 whenever the score $z \geq 0$, so the boundary between the classes is the flat surface $\beta_0 + \beta^T \mathbf{x} = 0$ — hence “linear” classifier. The threshold need not stay at 0.5: when a missed positive costs more than a false alarm, a lower threshold may be appropriate; in the opposite case a higher one may be. Chapter 7 shows how to choose it with validation or out-of-fold predictions.

10.3.2 Regularization

Exactly as in Chapter 9, an L_2 or L_1 penalty on β controls overfitting when features are numerous or correlated. Features should usually be standardized before coefficient penalties are compared, the penalty strength is tuned on validation data, and regularization also cures a quirk of the unpenalized model: on perfectly separable data, plain logistic regression pushes its coefficients toward infinity chasing ever-more-confident probabilities.

10.4 Softmax regression for multiple classes

For K classes, each class k receives its own weight vector, and the **softmax** function converts the K linear scores into probabilities:

$$P(Y = k \mid \mathbf{x}) = \frac{\exp(\beta_k^T \mathbf{x})}{\sum_{j=1}^K \exp(\beta_j^T \mathbf{x})}.$$

Each class’s score is exponentiated (making it positive) and divided by the total (making the probabilities sum to one); the largest score wins the largest probability. Training minimizes the multiclass cross-entropy $\mathcal{L} = -\log P(Y = y \mid \mathbf{x})$, the direct generalization of the binary loss.

A simpler alternative that works with any binary classifier is **one-vs-rest**: train K separate binary models (“class k against everything else”) and predict the class whose model is most confident.

10.5 Ordered categories are not ordinary multiclass labels

Some labels are categories with a meaningful order: *low* < *medium* < *high*, satisfaction ratings, or disease stages. This is an **ordinal** label. Ordinary softmax classification ignores the order, while coding the labels as 1, 2, 3 and using ordinary regression invents equal numerical gaps that may not exist.

Approach	When it makes sense
Ordinary multiclass classifier	Use when categories are genuinely unordered or when order is irrelevant to the decision.
Ordinal model / order-aware loss	Prefer when confusing adjacent levels is meaningfully less serious than confusing the two extremes.
Regression on integer codes	Use only when the numerical spacing itself is defensible; do not use it merely because the labels can be numbered.

There is no single universally appropriate model for ordinal labels. The important first-course skill is recognizing the ordered label structure and choosing an order-aware method or metric when the application requires it. Validation still decides whether that added structure improves the real task.

10.6 Linear discriminant analysis

Logistic regression is a **discriminative** model: it learns $P(y \mid \mathbf{x})$ directly and never asks what the features of each class look like. **Linear discriminant analysis (LDA)** takes the opposite, **generative** route: model what each class's data looks like, then use Bayes' theorem to classify.

LDA assumes each class k generates its features from a Gaussian with its own mean $\boldsymbol{\mu}_k$ but a *shared* covariance matrix Σ , and that class k occurs with prior probability π_k . Bayes' theorem then assigns a new point to the class with the largest **discriminant score**

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^\top \Sigma^{-1} \boldsymbol{\mu}_k + \log \pi_k.$$

Every term is estimated by counting and averaging: π_k from class frequencies, $\boldsymbol{\mu}_k$ from class means, and Σ from pooled within-class spread. Because $\mathbf{x}$ enters only linearly, the boundary between any two classes is again a straight line — LDA and logistic regression draw the same *kind* of boundary, found by different reasoning.

Plain idea

Picture two hills of sand of identical shape, one centered on each class's average, scaled by how common each class is. LDA classifies a new point by asking which hill stands taller at that spot. With identically shaped hills, the tie line between them is straight — and it shifts toward the rarer class, since its hill is lower everywhere.

When to prefer which:

- LDA estimates its parameters in one pass with simple formulas, so it is fast and remarkably stable on *small* datasets — when its Gaussian assumption is roughly right, it can beat logistic regression there.
- Logistic regression assumes less about the features and is more robust when classes are far from Gaussian (heavy tails, strong outliers, many binary features).
- LDA doubles as a supervised dimensionality-reduction method: for K classes it finds up to $K - 1$ directions that best separate them, a useful companion to PCA (Chapter 17).

Quadratic discriminant analysis (QDA) drops the shared-covariance assumption and gives each class its own Σ_k . The boundaries become curved (quadratic), which is more flexible but requires estimating many more parameters — QDA needs considerably more data per class to stay reliable.

10.7 The perceptron

The **perceptron** (1958) is the historical ancestor of this whole family. It predicts with the sign of a linear score, $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$ with $y \in \{-1, +1\}$, and learns by an appealingly simple rule: visit the training examples one at a time, and whenever one is misclassified, nudge the boundary toward it,

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i, \quad b \leftarrow b + \eta y_i.$$

Correctly classified points cause no update at all. If the classes are linearly separable, the perceptron is guaranteed to find a separating boundary in finitely many mistakes; if they are not, the basic algorithm need not settle down. It outputs no probabilities, and among the many separating lines it stops at one determined by the update path. Logistic regression and the support vector machine (Chapter 14) address these weaknesses differently — one models probabilities, while the other maximizes a margin.

10.8 Choosing among linear classifiers

Model	Useful strengths	Important limitations
Logistic regression	Strong probability baseline, convex training, interpretable odds ratios	Linear boundary; probabilities can still be miscalibrated under misspecification or shift
LDA	Very fast, stable on small data, gives class-separating directions	Assumes Gaussian classes with shared covariance
QDA	Curved boundaries, per-class covariance	Many parameters; needs much more data
Perceptron	Simplest possible trainer; historically important	No probabilities; unreliable when classes overlap

Key idea

Logistic regression, LDA, and the perceptron separate classes with flat boundaries, but choose those boundaries differently. Logistic regression maximizes the conditional likelihood; LDA models each class and applies Bayes' theorem; the perceptron nudges a line after each mistake and may not settle when classes overlap. QDA is the exception here: separate class covariances produce curved boundaries. The coming chapters add other forms of nonlinearity through neighborhoods, trees, ensembles, and kernels.

Linear Classifiers - Quick Guide

Best when	A linear boundary is reasonable, fast prediction matters, or coefficients need to be interpreted.
Start with	Logistic regression for probabilities; softmax regression for several classes.
Preprocessing	Encode categories, standardize numerical features for regularization, and handle imbalance with suitable metrics or weights.
Main controls	Regularization strength, penalty type, and the probability threshold used for decisions.
Watch for	Nonlinear class structure, poorly calibrated probabilities, rare classes, and causal over-interpretation (Section 5.9).

10.9 Using linear classifiers in practice

10.9.1 Probabilities, not just labels

10.9.2 Moving the threshold on purpose

Plain idea

Read the table as a dial, not a set of separate models. It is a single trained model being asked to be more or less cautious. A cancer screening service would choose the top row and accept the false alarms. A system that automatically freezes bank accounts would choose the bottom row, because wrongly freezing an account is expensive.

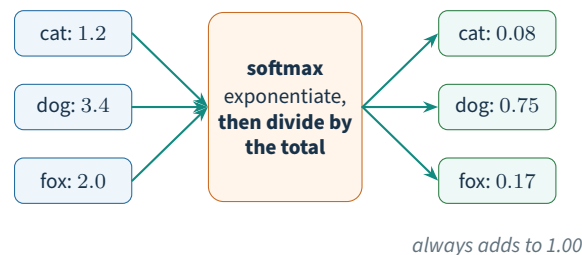
Choose the operating point from out-of-fold predictions and a stated cost or capacity constraint. Then refit the model on all training data and assess that locked rule on the test set once.

10.9.3 Reading the coefficients

Worked example

Suppose the coefficient for “hours studied” is 0.62. Then $e^{0.62} \approx 1.86$. Interpretation: each extra hour of study multiplies the *odds* of passing by about 1.86 — roughly an 86% increase in the odds, holding the other features fixed. Note that this is odds, not probability; the effect on probability depends on where you started.

10.9.4 More than two classes



Key idea

Softmax is the sigmoid’s multi-class cousin. It takes one raw score per class, makes them all positive by exponentiating, and then divides by the total so the results form a genuine probability distribution. The largest score always becomes the largest probability, but softmax also tells you *how much* larger.

Try it yourself

Fit a logistic regression, obtain out-of-fold probabilities on the training data, then sweep the threshold from 0.05 to 0.95 in steps of 0.05, recording precision and recall at each point. Plot recall on the x -axis against precision on the y -axis. You have just drawn a precision-recall curve by hand, as introduced in Chapter 7, without using the test set.

Debugging task

A teammate lowers the probability threshold until test-set recall reaches the desired value, then reports that same test recall as final performance. Explain the leakage in model selection and show where threshold tuning belongs instead.

Decision memo

For a rare positive class, choose between an unweighted logistic model, class weighting, and a lower decision threshold. State which quantity each intervention changes, which validation metric you would inspect, and why the three options are not interchangeable.

Check your understanding

1. Why is logistic regression called a linear classifier even though it outputs a nonlinear sigmoid probability?
2. Why should the probability model and the operational classification threshold be treated as separate objects?

3. What changes when binary logistic regression is extended to a multiclass softmax model?

Chapter recap

- Logistic regression turns a linear score into a probability with the sigmoid function.
- The decision threshold converts a probability into a class and can be changed to match error costs.
- Cross-entropy strongly penalizes confident wrong predictions.
- Softmax regression extends the idea to multiple classes.
- LDA uses a probability model for each class; the perceptron learns a separating boundary but not probabilities.

Nearest Neighbors and Naive Bayes

Chapter at a glance

Before you start: Distance measures from Chapter 5, Bayes' theorem from Chapter 5, and model-selection ideas from Chapter 8.

By the end, you can: Run k -nearest neighbors by hand, choose k and a distance sensibly, explain why naive Bayes multiplies per-feature probabilities, and apply Laplace smoothing to a small text example.

This chapter pairs two refreshingly simple classifiers. k -nearest neighbors predicts from raw similarity: “you are probably like the examples closest to you.” Naive Bayes predicts from probability bookkeeping: “which class makes these feature values least surprising?” Both can be computed by hand, both make strong baselines, and both teach lessons that echo through the rest of machine learning.

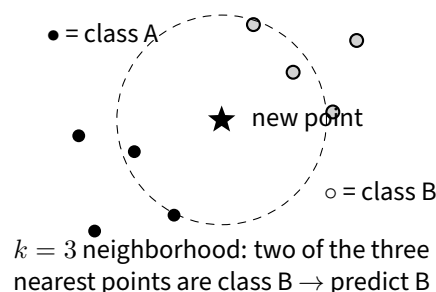
Core path

Think of two human strategies. Nearest neighbors asks, “what happened to the most similar past cases?” Naive Bayes asks, “which possible answer makes the evidence I see most believable?” Learn those questions first; the distance and probability calculations merely make the questions repeatable.

11.1 k -nearest neighbors

k -NN has no training phase worth the name: it simply *stores* the training data. All the work happens at prediction time:

1. compute the distance from the new observation to every stored training observation;
2. keep the k closest ones — the **neighbors**;
3. for classification, predict the majority class among the neighbors; for regression, predict the average of their labels.



Worked example

Training data: $A = (1, 1)$ and $B = (2, 1)$ labeled Red; $C = (4, 4)$ and $D = (5, 4)$ labeled Blue.

Classify the new point $(2, 2)$ with $k = 3$ using Euclidean distance:

$$\begin{aligned} d_A &= \sqrt{1^2 + 1^2} \approx 1.41, & d_B &= \sqrt{0^2 + 1^2} = 1.00, \\ d_C &= \sqrt{2^2 + 2^2} \approx 2.83, & d_D &= \sqrt{3^2 + 2^2} \approx 3.61. \end{aligned}$$

The three nearest are B (Red), A (Red), and C (Blue). The vote is Red 2, Blue 1, so the prediction is Red.

11.1.1 Choosing k

The single hyperparameter k is a smoothness dial:

- **Small** k (say $k = 1$) yields a jagged, flexible boundary that can capture fine structure but also copies every mislabeled or noisy training point — low bias, high variance.
- **Large** k averages over a wide neighborhood, producing a smooth, stable boundary that may blur real distinctions; at the extreme $k = n$, the model always predicts the overall majority class — high bias, low variance.

Choose k with validation data or cross-validation, use an odd k in binary problems to avoid tied votes, and consider **distance-weighted voting**, in which each neighbor's vote is weighted by $1/d$ so nearer neighbors count more.

11.1.2 What k -NN is sensitive to

Sensitivity	Why it matters	Practical response
Feature scale	Distance inherits the unit problem from Section 3.11.	Reuse that scaling/weighting rule rather than redefining it here.
Irrelevant features	Every feature affects the distance, so noise features dilute the signal.	Feature selection helps k -NN more than most models.
High dimensions	With many weak or noisy features, distance contrast can shrink — the curse of dimensionality — and “nearest” may stop meaning “similar.”	Feature selection or dimensionality reduction (Chapter 17) can help.
Dataset size	Predicting requires comparing against the whole training set.	Tree-based indexes (KD-trees, ball trees) and approximate nearest-neighbor methods make this workable at scale.

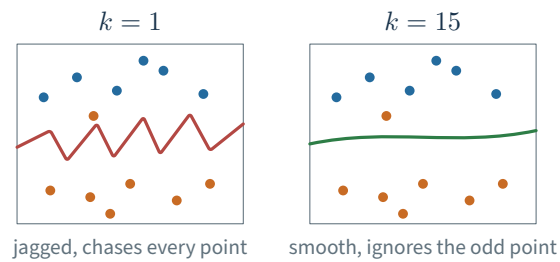
k -NN also does regression — predict the average (or distance-weighted average) label of the neighbors — which yields a locally adaptive, assumption-free fit, at the price of the same sensitivities listed above.

Plain idea

k -NN never builds a theory of the data; it just answers every question with “let me check what similar past cases did.” That makes it honest and flexible — and completely dependent on the distance actually measuring similarity that matters.

11.2 How k reshapes the boundary

The single control on k -NN is k itself, and its effect is easiest to see as a picture of the decision boundary.



Plain idea

The lone orange point sitting up in the blue region is probably noise. With $k = 1$ the boundary detours around it, carving out a little island — the model has memorized a mistake. With $k = 15$ that point is outvoted by its fifteen neighbors and the boundary sails straight past. Small k means low bias and high variance; large k means the reverse.

11.3 Naive Bayes

Naive Bayes turns classification into the question Bayes' theorem was built for: given the observed features, which class is most probable?

$$P(y \mid x_1, \dots, x_p) \propto P(y) \prod_{j=1}^p P(x_j \mid y).$$

Decoding the formula: $P(y)$ is the **prior** — how common the class is before looking at any features; each $P(x_j \mid y)$ is the **likelihood** of one feature value within that class; and the product is where the “naive” assumption lives — the features are treated as conditionally independent given the class (Chapter 5). That assumption is often false (words in an email are not independent), yet the resulting classifier can still rank classes well in practice.

Prediction picks the class with the largest score, $\hat{y} = \arg \max_y P(y) \prod_j P(x_j \mid y)$, and in software the product is computed as a sum of logarithms so that multiplying hundreds of small probabilities does not underflow to zero.

11.3.1 Three standard variants

Variant	Feature type	How it models the features
Gaussian NB	Continuous features	Each $P(x_j \mid y)$ is a normal density with the class-specific mean and variance of feature j .
Multinomial NB	Count data, such as word counts	$P(x_j \mid y)$ is the probability of word j in class y 's text, and repeated words count repeatedly. It is the workhorse for text classification.
Bernoulli NB	Binary presence/absence features	Explicitly models both a word appearing and a word being absent.

11.3.2 Laplace smoothing

Estimating $P(\text{word} \mid \text{class})$ by raw frequency has a fatal flaw: any word never seen in a class's training text gets probability zero, and one zero annihilates the whole product. **Laplace (add-one) smoothing** pretends

every word was seen once more than it was:

$$P(w \mid y) = \frac{\text{count}(w, y) + 1}{\text{total words in } y + V},$$

where V is the vocabulary size. No probability is ever exactly zero, and the estimates barely change for well-observed words.

Worked example

A tiny spam filter with vocabulary $\{\text{money, free, meeting, lunch}\}$ ($V = 4$) is trained on 10 words of spam and 10 words of ham, with equal priors $P(\text{spam}) = P(\text{ham}) = 0.5$:

raw counts	money	free	meeting	lunch
spam (10 words)	5	4	1	0
ham (10 words)	1	1	5	3

With smoothing, every likelihood is $(\text{count} + 1)/(10 + 4)$; for example $P(\text{money} \mid \text{spam}) = 6/14$ and $P(\text{money} \mid \text{ham}) = 2/14$. Classify the message “free money”:

$$\text{spam score} = 0.5 \times \frac{6}{14} \times \frac{5}{14} = \frac{15}{196}, \quad \text{ham score} = 0.5 \times \frac{2}{14} \times \frac{2}{14} = \frac{2}{196}.$$

Normalizing, $P(\text{spam} \mid \text{message}) = 15/(15 + 2) \approx 0.88$: the message is classified as spam. Note that without smoothing, a message containing “lunch” could never be spam, no matter what else it said.

11.3.3 Strengths and limitations

- Training is a single counting pass, prediction is instant, and the model scales effortlessly to millions of features — which is why it remains a standard first model for text.
- It performs surprisingly well with little training data, because it estimates only per-feature statistics rather than feature interactions.
- Its predicted probabilities are often *overconfident* (the independence assumption can double-count correlated evidence), so validate the probability values and recalibrate them when decisions require calibrated risk (Chapter 7).
- It cannot represent interactions: if a pattern only exists in a *combination* of features (“free” and “money” together, but neither alone), naive Bayes cannot learn it.

Key idea

k -NN and naive Bayes make opposite simplifications. k -NN keeps the training examples rather than estimating a fixed parametric form, but it still assumes that the chosen distance and local neighborhood are meaningful. Naive Bayes compresses the data into per-class, per-feature summaries and assumes conditional independence. Both are quick to build, valuable as baselines, and clear about what they ignore.

k -NN and Naive Bayes - Quick Guide

	k -nearest neighbors	Naive Bayes
Core idea	Predict from nearby training examples	Combine class prior with per-feature evidence
Best when	Local similarity is meaningful and data is not too large	Fast probabilistic classification is needed, especially for count or text features
Main controls	Number of neighbors, distance, weighting	Distribution variant, smoothing strength
Main risks	Scale, irrelevant features, slow prediction, high dimensions	Unrealistic independence assumption, probability calibration

PRACTICAL MILESTONE

Lab 5 of 10

Linear, neighbor, and probabilistic classification

Compare three different ways to construct a classification rule

Coverage. Chapters 10–11.

Source files. Lab05/Task01.ipynb, Task02.ipynb.

Core data. Titanic and Breast Cancer Wisconsin.

Goal. Interpret logistic probabilities and threshold policies, then compare a distance-based classifier with a probabilistic naive-Bayes model under the same evaluation discipline.

How much is scaffolded?

Focus on assumptions and failure modes. Logistic regression, KNN, and Gaussian naive Bayes can all classify, but they reach decisions through very different representations of the data.

Part A — Logistic regression and threshold policy.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "titanic_dataset" / "Titanic-Dataset.csv"
df = pd.read_csv(train_path)
```

```
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = df[features]
y = df["Survived"]
num_cols = ["Age", "SibSp", "Parch", "Fare"]
cat_cols = ["Pclass", "Sex", "Embarked"]

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])

model = Pipeline([
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=2000)),
])

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
model.fit(X_train, y_train)
prob = model.predict_proba(X_valid)[: , 1]
print("Validation ROC AUC:", round(roc_auc_score(y_valid, prob), 3))
```

```
for threshold in [0.30, 0.50, 0.70]:
    pred = (prob >= threshold).astype(int)
    print(
        threshold,
        "precision=", round(precision_score(y_valid, pred, zero_division=0), 3),
        "recall=", round(recall_score(y_valid, pred, zero_division=0), 3),
    )

thresholds = np.linspace(0.10, 0.90, 17)
precisions, recalls = [], []
for threshold in thresholds:
    pred = (prob >= threshold).astype(int)
    precisions.append(precision_score(y_valid, pred, zero_division=0))
    recalls.append(recall_score(y_valid, pred))
```

```
plt.figure(figsize=(6, 4))
plt.plot(thresholds, precisions, marker="o", label="precision")
plt.plot(thresholds, recalls, marker="o", label="recall")
plt.xlabel("Decision threshold")
plt.ylabel("Score")
plt.title("Threshold policy changes precision and recall")
plt.legend()
plt.tight_layout()
plt.show()
```

Part B — KNN versus Gaussian naive Bayes.

```
from pathlib import Path
import pandas as pd
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "breast_cancer_dataset" / "data.csv"
df = pd.read_csv(train_path)
```

```
X = df.drop(columns=["id", "diagnosis", "Unnamed: 32"], errors="ignore")
y = (df["diagnosis"] == "M").astype(int)
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
# Select k only inside the training partition.
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
k_values = [1, 3, 5, 7, 11, 21]
cv_scores = {}

for k in k_values:
    candidate = Pipeline([
        ("scale", StandardScaler()),
        ("model", KNeighborsClassifier(n_neighbors=k, weights="distance")),
    ])
    scores = cross_val_score(candidate, X_train, y_train, cv=cv, scoring="roc_auc")
    cv_scores[k] = scores.mean()
    print(f"k={k:2d} mean training-CV ROC AUC={scores.mean():.3f}")

best_k = max(cv_scores, key=cv_scores.get)
```

```
print("selected k:", best_k)
```

```
models = {
    "KNN": Pipeline([
        ("scale", StandardScaler()),
        ("model", KNeighborsClassifier(n_neighbors=best_k, weights="distance")),
    ]),
    "GaussianNB": Pipeline([
        ("scale", StandardScaler()),
        ("model", GaussianNB()),
    ]),
}
```

```
# Validation compares the locked KNN choice with the alternative model family.
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_valid)
    prob = model.predict_proba(X_valid)[: , 1]
    print(
        name,
        "validation accuracy=", round(accuracy_score(y_valid, pred), 3),
        "validation ROC AUC=", round(roc_auc_score(y_valid, prob), 3),
    )
```

Lab deliverable

Submit the logistic-regression probability/threshold comparison and a KNN-versus-Gaussian-NB validation table. Explain the role of scaling for KNN and give one reason naive Bayes can work even when its independence assumption is imperfect.

Check your understanding

1. Why is feature scaling usually essential for KNN, and what role does k play in the bias-variance trade-off?
2. What conditional-independence assumption gives naive Bayes its name?
3. Why can KNN become expensive at prediction time even though its training step is simple?

Chapter recap

- k -NN predicts from the labels of nearby training observations.
- Small k can overfit; large k can smooth away real local structure.
- Scaling is crucial for distance-based methods; feature selection is treated as a model-development step in Chapter 8; revisit it after seeing lasso and trees.
- Naive Bayes applies Bayes' theorem with conditional-independence assumptions.
- Laplace smoothing prevents unseen feature-class combinations from receiving zero probability.

Decision Trees

Chapter at a glance

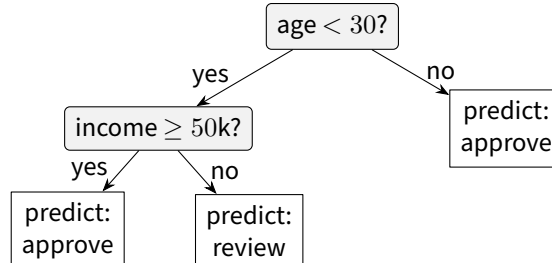
Before you start: Chapter 8 plus basic probability from Chapter 5; entropy is helpful but re-explained here.

By the end, you can: Grow a classification tree by hand using Gini impurity and information gain, build regression trees with variance reduction, and control tree size with stopping rules and pruning.

A decision tree predicts by asking a sequence of simple questions — “Is age below 30?” and “Is income at least 50,000?” — and routing each observation down the branches until it reaches a **leaf**, which holds the prediction. Small trees are among the most human-readable models: their learned rules can be printed, drawn, and discussed. Trees are also the building blocks of random forests and gradient boosting (Chapter 13), two strong families for tabular data.

Core path

A decision tree is a flowchart learned from examples. Training chooses questions that separate the known answers well. Prediction follows one path of yes/no answers to a final box. **Impurity** is only a score for how mixed the answers remain in a box.



Plain idea

A decision tree is a flowchart of yes/no questions, like the game “twenty questions.” Training is simply the process of discovering which question to ask first, second, and third so that each question splits the data as cleanly as possible.

12.1 How a tree is grown

Standard tree learning (the CART algorithm) is **greedy and recursive**:

1. At the current node, consider candidate splits: for a numerical feature, thresholds between sorted values; for categorical features, the available split rules depend on the tree formulation and representation.
2. Score each candidate by how much it *purifies* the data — how well it separates the classes (or, for regression, how much it shrinks the spread of the label).
3. Apply the best split, sending observations left and right.
4. Repeat on each child until a stopping rule fires; each final node becomes a leaf that predicts its majority

class (classification) or mean label (regression).

Greedy means the algorithm never looks ahead or reconsiders: it takes the best immediate split at every step. That keeps training fast and usually works well, though it does not guarantee the globally best tree.

12.2 Measuring impurity

To pick a split, the algorithm needs a number for “how mixed is this node?” With p_k the fraction of class k in a node, two standard measures are the **Gini impurity** and the **entropy**:

$$G = 1 - \sum_{k=1}^K p_k^2, \quad H = - \sum_{k=1}^K p_k \log_2 p_k.$$

Both are zero for a pure node (one class only) and largest for a perfect mixture. Gini reads as “the probability of mislabeling a random point if labels were assigned according to the node’s class mix”; entropy reads as “how many yes/no questions of uncertainty remain.” A split is chosen to maximize the **information gain**: the parent’s impurity minus the size-weighted impurity of the children,

$$\text{Gain} = I(\text{parent}) - \frac{n_{\text{left}}}{n} I(\text{left}) - \frac{n_{\text{right}}}{n} I(\text{right}),$$

where I is either measure. Gini and entropy almost always agree on the best split; Gini is slightly cheaper to compute and is the common default.

Worked example

A node holds 10 loan applications: 6 approved (+) and 4 rejected (−). Its Gini impurity is $G_{\text{parent}} = 1 - (0.6^2 + 0.4^2) = 0.48$.

Candidate split A produces children $[4+, 0-]$ and $[2+, 4-]$:

$$G_{\text{left}} = 0, \quad G_{\text{right}} = 1 - \left[\left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2 \right] = \frac{4}{9} \approx 0.444,$$

$$\text{Gain}_A = 0.48 - \left(\frac{4}{10} \times 0 + \frac{6}{10} \times 0.444 \right) = 0.48 - 0.267 = 0.213.$$

Candidate split B produces children $[3+, 2-]$ and $[3+, 2-]$. Each child has the same 60/40 mix as the parent, so each has $G = 0.48$, the weighted impurity is unchanged, and $\text{Gain}_B = 0$: split B tells us nothing, and split A wins. (Entropy agrees: split A’s gain is $0.971 - 0.551 \approx 0.42$ bits.)

12.3 Finding the best split

The split search follows the verbal rule from earlier in the chapter: examine candidate features and thresholds, compute the weighted impurity after each split, and keep the candidate with the largest impurity reduction.

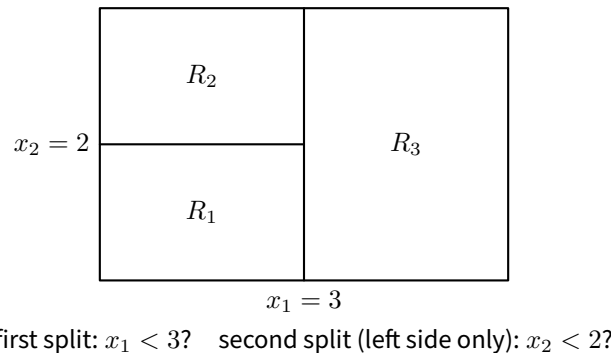
Consider six one-dimensional observations where the classes separate cleanly. If one threshold places every class-0 point on one side and every class-1 point on the other, both children have zero impurity. The information gain is therefore equal to the parent impurity, which is the largest possible gain for that node.

Reference note

Different tree formulations may report an observed boundary value or the midpoint between two adjacent observed values. If both thresholds send all training observations to the same children, the fitted partition is identical even though the printed threshold differs. Midpoint thresholds are often convenient because they define a boundary between observations rather than directly on one observation.

12.4 The geometry: rectangles in feature space

Because every question compares one feature to one threshold, a tree carves feature space into axis-aligned rectangles, one per leaf, and predicts a constant inside each.



This geometry explains both a strength and a weakness: boundaries that align with the axes (rules, thresholds, interactions of a few features) are captured perfectly, while a simple diagonal boundary must be approximated by a staircase of many small splits.

12.5 Regression trees

For a numerical label, impurity becomes the spread of y within a node — its variance (equivalently, the mean squared error around the node mean) — and a split is chosen to reduce it the most. Each leaf predicts the mean label of its training observations, so the fitted function is piecewise constant.

Worked example

A node holds labels $y = \{2, 3, 10, 11\}$ with mean 6.5 and variance 16.25. Splitting into $\{2, 3\}$ and $\{10, 11\}$ gives children with means 2.5 and 10.5 and variance 0.25 each — a weighted variance of 0.25. The split explains almost all the spread, and the two leaves predict 2.5 and 10.5.

One consequence deserves a warning label: a regression tree can never predict a value outside the range of its training labels. Asked about a house larger than any it has seen, it predicts the mean of its largest-house leaf — trees do not extrapolate.

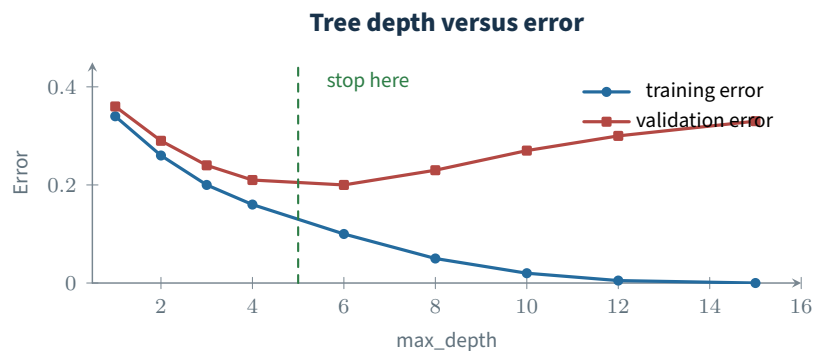
12.6 Stopping and pruning

An unrestricted tree keeps splitting until every leaf is pure — typically one training observation per leaf, which is memorization, not learning. Two complementary defenses:

Method	How it works	Trade-off
Pre-pruning (early stopping)	Refuse to grow past limits set in advance: a maximum depth, a minimum number of observations required to split a node or to sit in a leaf, or a minimum impurity decrease.	Simple and fast; the limits are hyperparameters tuned on validation data.
Post-pruning	Grow a deliberately large tree first, then cut back branches whose contribution does not justify their complexity. Cost-complexity pruning charges a penalty α per leaf and keeps the subtree that best trades training fit against size; α is chosen by cross-validation.	Growing and then cutting can rescue good splits that become visible only after a mediocre one, which early stopping would never discover.

12.7 Depth: the dial between too simple and memorized

This section applies the bias–variance framework from Section 8.3 to one tree-specific control: depth. The framework itself is not re-derived here.



12.7.1 The controls that limit growth

Setting	What it does
<code>max_depth</code>	Hard limit on how many questions can be asked in a row.
<code>min_samples_split</code>	Refuse to split a node holding fewer than this many rows.
<code>min_samples_leaf</code>	Every final leaf must contain at least this many rows.
<code>max_leaf_nodes</code>	Cap the total number of leaves.
<code>ccp_alpha</code>	Cost-complexity pruning: grow fully, then prune back.
<code>class_weight</code>	Make rare-class rows count for more.

12.7.2 Cost-complexity pruning, properly

12.8 Strengths, limitations, and practical notes

- **Interpretability:** the model *is* an explanation; individual predictions can be justified by reading off the path.
- **Low preprocessing:** numerical splits depend only on value order, so trees need no feature scaling. Category and missing-value support is implementation-specific, so the representation must match the chosen tree formulation.
- **Nonlinearity for free:** interactions and thresholds emerge automatically, with no feature engineering.
- **Instability:** a small change in the training data can produce a substantially different tree, because one changed early split reroutes everything below it. Single trees are therefore high-variance models — the direct motivation for the ensembles of Chapter 13.
- **Feature importance:** use the interpretation and bias cautions in Section 12.10.

Key idea

A decision tree converts learning into a sequence of best-question choices, scored by impurity reduction. Its readable rules and zero-fuss data handling make it the friendliest nonlinear model — and its instability makes it the perfect raw material for ensembles, where many imperfect trees are combined into one accurate, stable predictor.

Decision Trees - Quick Guide

Best when	Nonlinear rules, interactions, mixed feature scales, and human-readable if-then paths are useful.
Preprocessing	Usually no scaling is needed; categories still need a supported representation; missing-value handling depends on the implementation.
Main controls	Maximum depth, minimum samples per split/leaf, maximum leaf nodes, and pruning strength.
Strengths	Handles nonlinear relationships and interactions; predictions are easy to trace through a small tree.
Watch for	High variance, unstable splits, misleading feature importance, and blocky extrapolation.

12.9 Interpreting a fitted tree

A decision tree is one of the very few models you can print out and check by eye. Take advantage of that.

Key idea

That is the fitted decision rule itself, not a post-hoc approximation. A small tree is unusually easy to inspect among nonlinear models, which helps explain its popularity in rule-oriented settings. Inspection alone does not make a model fair, causal, stable, or suitable for a high-stakes decision; those claims require separate evidence and governance.

12.10 Feature importance, and why to distrust it

Worked example

Permutation importance works by an elegant trick: shuffle one column at random, destroying its relationship with the label, and measure how much the score drops. If shuffling “age” costs three points of accuracy and shuffling “customer ID” costs nothing, then age was genuinely being used and the ID was not.

Key idea

Feature importance describes which columns this fitted model used. Apply the general association-versus-causation rule from Section 5.9; importance is not causal evidence.

Debugging task

A depth-20 tree has nearly perfect training accuracy and unstable feature importance across random splits. Diagnose the likely failure mode, then compare depth 2, 5, and 20 using identical validation folds and a sealed test set.

Decision memo

Choose between a shallow single tree and a random forest for a decision that must be explained to a nontechnical reviewer. Your memo must separate predictive performance from interpretability and state what level of performance loss, if any, you would accept for the simpler model.

Check your understanding

1. What does a decision-tree split try to improve at each node?
2. Why can an unconstrained tree achieve excellent training performance yet generalize poorly?
3. Why are ordinary decision trees largely insensitive to monotonic rescaling of a numerical feature?

Chapter recap

- A decision tree repeatedly splits the feature space with yes/no questions.
- Classification trees reduce impurity; regression trees reduce label variation.
- Deep trees can memorize training data and become unstable.
- Depth limits, minimum leaf sizes, and pruning control complexity.
- A small tree is interpretable, but an ensemble of trees is often more accurate.

Ensemble Methods

Chapter at a glance

Before you start: Decision trees from Chapter 12 and the bias–variance/model-selection ideas from Chapter 8. The variance framework is referenced rather than retaught.

By the end, you can: Explain why combining models works, train bagged trees, Random Forests, and Extra Trees, follow AdaBoost’s reweighting and gradient boosting’s residual-fitting step by step, use voting/stacking without leakage, and inspect ensemble behavior without confusing explanation with causation.

No single model is right all the time — but different models tend to be wrong in *different* ways. An **ensemble** exploits this by training many models and combining their predictions, so that individual mistakes partly cancel. Random forests and gradient-boosted trees are particularly strong general-purpose candidates for tabular data, though no family dominates every dataset or operational constraint.

Core path

Core path: understand why averaging helps, then learn bagging, random forests, and gradient boosting. Voting and stacking are useful extensions after those three ideas are clear.

13.1 Why averaging helps

This is the ensemble-specific application of the variance side of Section 8.3; the general bias–variance trade-off remains defined there.

Suppose B models each make predictions with the same error variance σ^2 . If their errors were completely independent, the variance of their average would be σ^2/B — averaging 100 models would slash the noise a hundredfold. Real models trained on the same data are correlated, and with average pairwise correlation ρ the variance of the average is approximately

$$\rho \sigma^2 + \frac{(1 - \rho)}{B} \sigma^2.$$

Reading the formula: the second term melts away as the ensemble grows, but the first does not. Adding models is cheap variance reduction, yet the floor is set by how *correlated* the models are — so ensemble design is really diversity engineering. The two great families differ in how they create that diversity: **bagging** trains models independently on perturbed data, and **boosting** trains them sequentially, each one targeting the current ensemble’s mistakes.

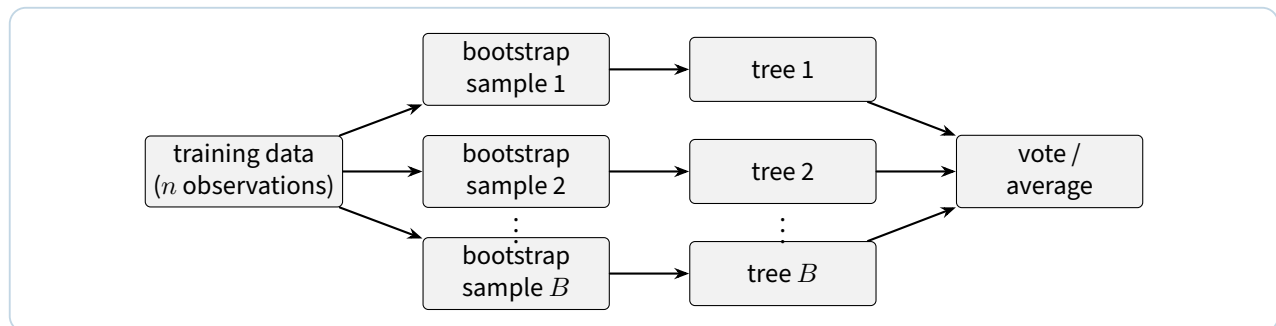
Plain idea

Ask one expert and you get their blind spots. Ask a hundred experts who all read the same newspaper and you get the newspaper’s blind spots. Ask a hundred experts with genuinely different experiences and the errors wash out — crowd wisdom needs disagreement, not just headcount.

13.2 Bagging: bootstrap aggregating

Bagging manufactures diversity by giving every model a slightly different view of the data. A **bootstrap sample** is created by drawing n observations from the training set *with replacement*: some observations

appear twice or more, and each model misses about 37% of the data entirely (the chance an observation is never drawn is $(1 - 1/n)^n \approx e^{-1} \approx 0.37$). A common base learner is a deep decision tree, chosen because its instability gives averaging something to reduce. One learner is trained on each sample, and predictions are combined by voting (classification) or averaging (regression).



A useful internal estimate is available because each tree never saw its **out-of-bag (OOB)** observations: every row can be predicted by trees that skipped it. OOB performance can guide early development, but repeated choices based on it make it another validation signal; it does not replace a final untouched test.

13.3 Random forests

A **random forest** is bagging plus one more dose of randomness: at every split of every tree, only a random subset of the features is considered. $\sqrt{p}$ is a common classification starting point; regression implementations and applications use choices ranging from a fraction of the features to all p , so validate the setting. Without feature subsampling, one dominant feature may headline the top of every tree, making the trees — and their errors — highly correlated; forcing splits to explore other features can *decorrelate* the trees, which the averaging formula rewards directly.

Practical profile:

- **Key hyperparameters:** the number of trees B (more usually stabilizes the Monte Carlo average until gains plateau), the number of features tried per split, and tree depth or minimum leaf size.
- **Little tuning required:** sensible defaults perform well, which makes random forests the standard “strong model with ten minutes of effort” on tabular data.
- **Extras:** OOB error estimates and feature-importance rankings come nearly free; apply the importance caveat from Section 12.10.
- **Costs:** the model is large, slower to predict than a single tree, and no longer readable as rules.

13.3.1 Extra Trees: randomize the split thresholds too

Extremely Randomized Trees (Extra Trees) are a close relative of Random Forests. Both build many decorrelated trees, but Extra Trees randomize candidate split thresholds more aggressively; some formulations also differ in whether each tree uses bootstrap samples or the full training sample. That extra randomness can reduce variance and training cost while adding some bias.

Compare Extra Trees with Random Forest under the same folds and metric. Neither is universally better; the useful lesson is that both trade individual-tree strength for diversity across the ensemble.

13.4 Boosting: learning from mistakes

Boosting takes the opposite route to diversity: train simple models *in sequence*, each one concentrating on what the ensemble so far gets wrong, and add them up. Where bagging reduces variance by averaging fully grown trees, boosting reduces *bias* by stacking many **weak learners** — typically shallow trees, even

one-split “stumps” — into a single strong one.

	Bagging / random forest	Boosting
Training	Parallel, independent	Sequential, each model depends on the last
Base models	Fully grown, low-bias, high-variance trees	Shallow, high-bias, low-variance trees
Main effect	Reduces variance	Reduces bias (and variance via shrinkage)
Overfitting risk	Low; robust to noise	Real; needs a learning rate and early stopping
Tuning effort	Small	Moderate to large

13.4.1 AdaBoost: reweight the hard cases

AdaBoost, the first practical boosting algorithm, keeps a weight on every training observation and turns up the weight on whatever the current ensemble misclassifies:

1. Start with equal weights $w_i = 1/n$.
2. Fit a weak learner h_m to the weighted data and compute its weighted error rate ε_m .
3. Give the learner a say proportional to its quality:

$$\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}.$$

4. Multiply the weights of misclassified observations by e^{α_m} and of correct ones by $e^{-\alpha_m}$, then renormalize — the next learner will stare hardest at the mistakes.
5. After M rounds, predict with the weighted vote $\text{sign}(\sum_m \alpha_m h_m(\mathbf{x}))$.

The formula for α_m behaves exactly as intuition demands: a learner with $\varepsilon_m = 0.3$ earns $\alpha_m = \frac{1}{2} \ln(7/3) \approx 0.42$ and its mistakes get their weights multiplied by $e^{0.42} \approx 1.5$; a coin-flip learner ($\varepsilon_m = 0.5$) earns no vote at all. The escalating focus on hard cases is AdaBoost’s power and its weakness — persistently mislabeled observations attract ever more attention, so AdaBoost is sensitive to label noise.

13.4.2 Gradient boosting: fit the residuals

Gradient boosting generalizes the idea to any differentiable loss. After $m - 1$ rounds the ensemble is F_{m-1} ; the next small tree h_m is fitted to the *negative gradient* of the loss at the current predictions — which, for squared error, is simply the residuals $y_i - F_{m-1}(\mathbf{x}_i)$ — and added with a small step:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu h_m(\mathbf{x}), \quad 0 < \nu \leq 1.$$

The **learning rate** ν shrinks each tree’s contribution. Smaller values usually need more trees and sometimes improve generalization, but the interaction is data-dependent; choose the pair with validation or early stopping.

Worked example

Labels $y = (3, 5, 8, 10)$. Round 0 predicts the mean: $F_0 = 6.5$ for everyone, residuals $(-3.5, -1.5, 1.5, 3.5)$. A one-split stump fitted to the residuals separates the first two observations

from the last two and predicts their residual means, -2.5 and $+2.5$. With $\nu = 0.5$,

$$F_1 = (6.5 - 1.25, 6.5 - 1.25, 6.5 + 1.25, 6.5 + 1.25) = (5.25, 5.25, 7.75, 7.75),$$

and the residuals shrink to $(-2.25, -0.25, 0.25, 2.25)$. Each round nibbles away at what remains unexplained; a hundred small corrections later, the ensemble traces the data closely.

Plain idea

Gradient boosting is a committee formed one member at a time, where each new member's only job is to fix the committee's current mistakes — and is deliberately given a quiet voice (ν) so no single member can dominate. AdaBoost is the special case where “fix the mistakes” means “re-argue the misclassified cases,” arising from a particular (exponential) loss.

13.4.3 Modern gradient-boosting libraries

Three refinements of gradient boosting dominate practical work; all keep the algorithm above and add engineering and regularization:

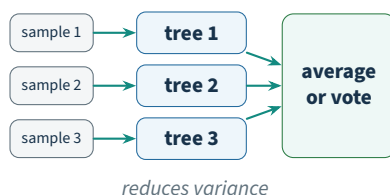
Library	Distinctive approach	Practical strength
XGBoost	Adds explicit regularization of tree size and leaf values to the objective and uses second-derivative information for better splits.	Popularized the modern toolkit, including early stopping and column subsampling.
LightGBM	Grows trees leaf-wise instead of level-wise and buckets feature values into histograms.	Extremely fast on large datasets.
CatBoost	Handles categorical features natively with label statistics computed in a careful order to avoid leakage.	Notably strong when the data contains many categorical columns.

The hyperparameters that matter most, in rough order: number of trees (via early stopping), learning rate, tree depth or leaf count, and the row/column subsampling fractions.

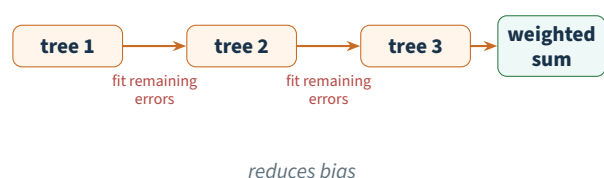
13.5 Bagging and boosting, side by side

Both build many small models, but they differ in one decisive way: bagging builds them independently and in parallel, while boosting builds them in a chain where each one repairs the last.

Bagging: parallel and independent



Boosting: sequential and corrective



Analogy

Bagging is asking a hundred people to guess the number of sweets in a jar and taking the average — individual errors cancel out. Boosting is a relay of tutors: the first teaches the class, the second focuses only on the students who failed, and the third focuses on those who still fail. Averaging fixes

noise; relaying fixes ignorance.

Key idea

Bagging commonly averages high-variance, fairly deep trees; boosting commonly adds shallow trees that correct the current residual structure. These are strong starting patterns, not laws. Depth, leaf size, learning rate, and number of rounds interact and should be validated together.

13.6 Voting and stacking

Ensembles can also mix *different* algorithms. **Voting** simply combines finished models — by majority label (hard voting) or by averaging predicted probabilities (soft voting, usually better). **Stacking** goes further: the base models' predictions become the input features of a second-level **meta-model** that learns how much to trust each base model, and when.

Stacking hides a leakage trap: if the meta-model is trained on predictions the base models made *on their own training data*, it learns to trust overfitted outputs. The fix is to feed it **out-of-fold** predictions — each base model's predictions on data it did not train on, produced via cross-validation — and to keep the meta-model simple (logistic or linear regression is typical).

13.7 Practical ensemble workflow

- Want a strong tabular model with minimal fuss? **Random forest**.
- Chasing the best accuracy and willing to tune? **Gradient boosting** with early stopping.
- Noisy labels? Prefer forests over AdaBoost-style boosting, or lower the boosting learning rate and depth.
- Need every last drop for a competition or a critical system? Stack diverse models — but only with out-of-fold discipline, and only if the added complexity is worth maintaining.

Key idea

Ensembles turn the weakness of trees — instability — into strength. Bagging and random forests average many varied trees to reduce variance; boosting adds dependent trees one correction at a time. Together, tree ensembles remain among the strongest general-purpose classical methods for tabular data, but validation, latency, interpretability, and maintenance still decide whether they are appropriate.

Tree Ensembles - Quick Guide

	Random forest	Gradient boosting
Core idea	Average many varied trees	Add shallow trees sequentially to repair errors
Usually strong for	Reliable general-purpose tabular prediction	High predictive accuracy on structured/tabular data
Main controls	Number of trees, sampled features, depth, leaf size	Number of trees, learning rate, depth, subsampling
Main risks	Large models, less transparent than one tree	Overfitting through too many rounds, sensitive tuning, slower training

13.7.1 Controls worth tuning

Random forests are usually forgiving: start with enough trees for stable averages, then control individual-tree complexity with depth, leaf size, and the number of features considered at each split. Gradient boosting is more sensitive because learning rate and number of rounds trade off directly. The table below is a practical starting range, not a universal recipe.

Common mistake

The built-in early-stopping split must respect the deployment structure. A random validation fraction is inappropriate when rows are ordered in time or grouped by person, site, or device. In those settings, use a library interface that accepts a correctly separated validation set, or tune the round count with an explicit structure-preserving validation scheme.

Setting	Try	Effect
<code>learning_rate</code>	0.01–0.1	Smaller learns more carefully; needs more trees.
<code>n_estimators</code>	300–2000	With early stopping, set it generously.
<code>max_depth</code>	3–8	Deeper catches interactions but overfits sooner.
<code>subsample</code>	0.6–1.0	Row sampling; adds randomness, fights overfitting.
<code>colsample_bytree</code>	0.6–1.0	Column sampling; same idea, per feature.
<code>reg_lambda</code>	0–10	Extra penalty on leaf values.

13.7.2 When combining model families helps

Plain idea

Soft voting averages probabilities and treats every model as equally trustworthy. Stacking trains a small extra model to learn *which* base model to believe in which situation — perhaps the forest is reliable for common cases and the linear model for the extremes. Stacking is stronger, but it needs enough data to train that extra layer without overfitting.

Common mistake

Ensemble gains require members that are individually useful and not perfectly redundant. Averaging several already-large random forests trained in the same way often adds little, because each is itself an average of similar trees. When voting or stacking, compare genuinely different views—for example, a linear model and a tree ensemble—and keep the combination only if validation shows a worthwhile gain.

13.8 Inspecting ensemble predictions

Permutation importance is introduced with trees in Section 12.10; it applies to fitted ensembles as well. Two additional inspection tools are partial dependence and SHAP. Treat both as descriptions of model behavior, not evidence that changing a feature will cause the real-world outcome to change.

Permutation importance asks how much the fitted model suffers when one feature is broken. Correlated features can mask one another. Partial dependence varies one feature while averaging predictions over the others; with strongly correlated inputs, that can create combinations that are rare or impossible in the real data.

13.8.1 Optional: SHAP for local explanations

SHAP (SHapley Additive exPlanations) decomposes a prediction relative to a reference output:

$$\hat{y}_i = \mathbb{E}[\hat{y}] + \sum_{j=1}^p \phi_{ij}.$$

The contribution ϕ_{ij} explains how this fitted model's output for row i differs from its reference output. Correlated features can split attribution between one another, so explanation stability should be checked rather than assumed.

Apply the association-versus-causation distinction from Section 5.9: an explanation can reveal what the model used without proving what would happen under an intervention.

13.9 Ensemble comparison at a glance

Method	What changes from one tree	Main reason to try it
Bagging	Bootstrap many trees and average/vote	Reduce variance and make a high-variance learner more stable
Random Forest	Bagging plus random feature subsets at splits	Make the trees less correlated, so averaging helps more
AdaBoost	Train weak learners sequentially with emphasis on previous errors	Reduce bias by focusing later learners on difficult cases
Soft voting	Average probabilities from different model families	Combine complementary decision rules rather than many copies of one rule

Try it yourself

Compare a single tree, a random forest, and a gradient-boosting model with the same cross-validation splits. Record mean validation score, descriptive fold spread, fit time, and prediction time. Decide whether any gain justifies the added cost, then evaluate the locked choice on the test set once.

PRACTICAL MILESTONE

Lab 6 of 10

Decision trees and ensembles

Move from one interpretable tree to variance-reducing and boosting ensembles

Coverage. Chapters 12–13.

Source files. Lab06/Task01.ipynb through Task03.ipynb.

Core data. Titanic and Ames Housing. Wine Quality is an optional transfer activity inside the same lab.

Goal. Inspect the structure of a single tree, then compare it with ensemble methods using consistent validation and computational evidence.

How much is scaffolded?

Use the single tree as the interpretable reference point. Ensembles are justified only when their validation gain is worth the added complexity, fit time, and monitoring cost.

Part A — Interpretable decision tree.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "titanic_dataset" / "Titanic-Dataset.csv"
df = pd.read_csv(train_path)
```

```
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = df[features]
y = df["Survived"]
num_cols = ["Age", "SibSp", "Parch", "Fare"]
cat_cols = ["Pclass", "Sex", "Embarked"]

preprocess = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = Pipeline([
    ("preprocess", preprocess),
    ("tree", DecisionTreeClassifier(
        max_depth=4, min_samples_leaf=12, random_state=42
    )),
])
```

```

model.fit(X_train, y_train)
train_pred = model.predict(X_train)
valid_pred = model.predict(X_valid)
print("training accuracy:", round(accuracy_score(y_train, train_pred), 3))
print("validation accuracy:", round(accuracy_score(y_valid, valid_pred), 3))

```

```

feature_names = model.named_steps["preprocess"].get_feature_names_out()
print(export_text(
    model.named_steps["tree"],
    feature_names=list(feature_names),
    max_depth=3,
))

```

```

# Plot only the top levels so the learned rules remain readable.
feature_names = model.named_steps["preprocess"].get_feature_names_out()
plt.figure(figsize=(12, 6))
plot_tree(
    model.named_steps["tree"],
    feature_names=feature_names,
    class_names=["did not survive", "survived"],
    max_depth=2,
    filled=True,
    fontsize=8,
)
plt.title("Top of the fitted Titanic decision tree")
plt.tight_layout()
plt.show()

```

Part B — Ames Housing ensembles.

```

from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

```

```

PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "ames_housing_dataset" / "AmesHousing.csv"
df = pd.read_csv(train_path)

```

```

# Order and PID are identifiers, not useful house characteristics for this lesson.
X = df.drop(columns=["SalePrice", "Order", "PID"], errors="ignore")
y = np.log1p(df["SalePrice"])

num_cols = X.select_dtypes(include="number").columns
cat_cols = X.select_dtypes(exclude="number").columns
preprocess = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ]), cat_cols),
])

models = {
    "random_forest": RandomForestRegressor(
        n_estimators=150, max_features=0.8, min_samples_leaf=2,
        random_state=42, n_jobs=-1
    ),
    "gradient_boosting": GradientBoostingRegressor(
        n_estimators=150, learning_rate=0.05, max_depth=3,
        loss="huber", random_state=42
    ),
}

```

```

}
cv = KFold(n_splits=3, shuffle=True, random_state=42)

for name, estimator in models.items():
    pipe = Pipeline([("preprocess", preprocess), ("model", estimator)])
    neg_mse = cross_val_score(
        pipe, X, y, cv=cv, scoring="neg_mean_squared_error", n_jobs=1
    )
    rmse = np.sqrt(-neg_mse)
    print(name, "CV log-RMSE:", round(rmse.mean(), 4),
          "+/-", round(rmse.std(), 4))

```

Part C — Optional wine-quality transfer.

```

from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split

```

```

PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "red_wine_quality_dataset" / "winequality-red.csv"
df = pd.read_csv(train_path)

```

```

X = df.drop(columns="quality")
y = df["quality"]
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.25, random_state=42
)

models = {
    "random_forest": RandomForestRegressor(
        n_estimators=400, min_samples_leaf=2, random_state=42, n_jobs=-1
    ),
    "gradient_boosting": GradientBoostingRegressor(random_state=42),
}

```

```

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_valid)
    results[name] = pred
    print(name)
    print(" MAE:", round(mean_absolute_error(y_valid, pred), 3))
    print(" RMSE:", round(np.sqrt(mean_squared_error(y_valid, pred)), 3))

```

```

plt.figure(figsize=(6, 4))
plt.scatter(y_valid, results["gradient_boosting"], alpha=0.65)
plt.xlabel("Observed quality")
plt.ylabel("Predicted quality")
plt.title("Wine quality validation: gradient boosting")
plt.tight_layout()
plt.show()

```

Lab deliverable

Submit one interpretable tree summary, an ensemble comparison on Ames Housing, and a short complexity-versus-performance decision. The Wine Quality transfer activity may be used to test whether the same conclusion transfers to a different dataset; it does not create another lab.

Check your understanding

1. How does bagging reduce variance, and how does boosting differ in the way later learners are constructed?
2. Why does a random forest consider only a subset of features at many splits?
3. Why must predictions used to train a stacking meta-model be out-of-fold rather than fitted predictions from the same rows?

Chapter recap

- Ensembles combine several models to reduce error.
- Bagging trains models on bootstrap samples and averages their predictions.
- Random forests add feature randomness to make trees less correlated.
- Boosting builds models sequentially, concentrating on current errors or residuals.
- Learning rate and number of boosting rounds should be tuned together with validation data.

Support Vector Machines and Supervised Synthesis

Chapter at a glance

Before you start: Linear classifiers from Chapter 10, model-capacity ideas from Chapter 8, and dot products/norms from Chapter 5.

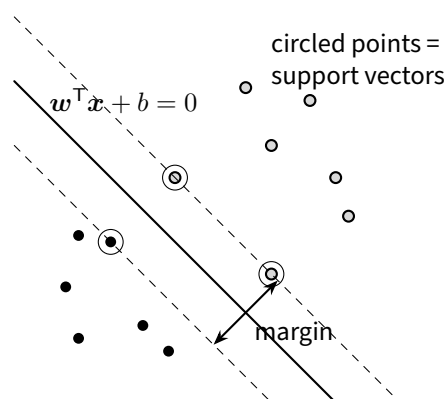
By the end, you can: Explain the maximum-margin idea, read the hard- and soft-margin objectives, use the kernel trick to draw curved boundaries, compare the full supervised toolkit, and decide when an SVM is a sensible candidate relative to the other supervised model families.

Chapter 10 ended with an open question: when many straight lines separate the classes, which one should a classifier choose? The support vector machine (SVM) gives an influential answer: choose the line with the *widest margin* — the one that stays as far as possible from the nearest points of both classes. Points on or inside that margin are the **support vectors**, and they determine the boundary.

Core path

Core path: keep the margin picture, then learn soft margins, C , kernels, and γ . The practical goal is to know when an SVM is a sensible comparison model and why scaling matters.

14.1 The widest street



Plain idea

Imagine the boundary as the center line of a street between two crowds. The SVM builds the *widest possible street* that touches neither crowd; the people leaning against the curbs are the support vectors. Everyone standing far back could go home — removing them would not move the street at all.

14.2 The hard-margin objective

With labels $y_i \in \{-1, +1\}$, a linear SVM scores points with $\mathbf{w}^\top \mathbf{x} + b$ and requires every training point to sit on its correct side with room to spare:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

Decoding the pieces: the constraint $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ says “the score has the right sign, and its size is at least 1” — points exactly at 1 lie on the dashed margin lines. Geometrically, the distance between the two dashed lines works out to $2/\|\mathbf{w}\|_2$, so *minimizing* $\|\mathbf{w}\|$ is exactly *maximizing* the margin; squaring and halving just make the calculus tidy.

14.3 The soft margin: allowing exceptions

Real classes overlap, so the SVM used in practice permits violations and charges for them:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

The second term is the **hinge loss**: zero for points comfortably beyond the margin, growing linearly for points inside it or on the wrong side. The hyperparameter C sets the exchange rate between a wide street and few violations:

- **Large** C : violations are expensive, so the boundary contorts to classify nearly everything correctly — narrow margin, higher risk of overfitting.
- **Small** C : violations are cheap, so the model keeps a wide, calm margin and tolerates some training errors — stronger regularization.

Written as “hinge loss plus an L_2 penalty,” the soft-margin SVM lines up neatly beside regularized logistic regression, which is “cross-entropy loss plus an L_2 penalty”: same skeleton, different loss. The hinge loss stops caring once a point clears the margin, which is why only the support vectors matter; cross-entropy keeps rewarding extra confidence forever, which is why logistic regression uses every point.

14.4 Curved boundaries: mapping up

Some datasets cannot be separated by any straight line — but they can be after the features are transformed.

Worked example

On a number line, let $x \in \{-2, 2\}$ be class A and $x \in \{-1, 0, 1\}$ be class B. No single threshold on x separates A from B (A sits on *both* sides). Now map each point upward to two features, $\phi(x) = (x, x^2)$:

x	-2	-1	0	1	2
x^2	4	1	0	1	4
class	A	B	B	B	A

In the new (x, x^2) plane, class A floats at height 4 and class B sits at height ≤ 1 : the horizontal line $x^2 = 2.5$ separates them perfectly. A linear boundary in the mapped space is a *curved* boundary (here, $|x| = \sqrt{2.5}$) in the original space.

14.5 The kernel trick

Explicitly building rich feature maps gets expensive fast — and, for the most useful maps, impossible (they are infinite-dimensional). The escape hatch is that the SVM’s training and prediction equations can be rearranged to use the data *only through inner products* $\phi(\mathbf{x})^\top \phi(\mathbf{x}')$. A **kernel function** computes that inner product directly from the original inputs,

$$k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}'),$$

so the mapped features never need to be constructed at all — this shortcut is the **kernel trick**. Standard choices:

Kernel	Formula	Boundary it produces
Linear	$\mathbf{x}^\top \mathbf{x}'$	Straight; fast on huge, sparse data
Polynomial	$(\gamma \mathbf{x}^\top \mathbf{x}' + r)^d$	Curved, degree- d interactions
RBF (Gaussian)	$\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ _2^2)$	Smooth, local, very flexible

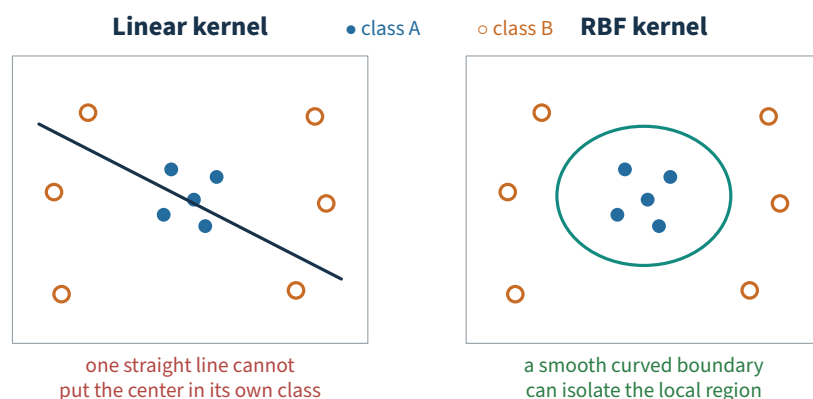
The **RBF kernel** is the default: it reads as “similarity that decays with distance,” and its γ sets how fast. Small γ means every support vector influences a wide area (smooth boundary, may underfit); large γ means influence is a tight bubble around each point (wiggly boundary, may overfit). In practice, C and γ are tuned *together* on a logarithmic grid — they interact strongly.

Plain idea

An RBF-kernel SVM predicts like a panel of landmarks: each support vector shouts its own class with a volume that fades with distance (γ is how fast the shout fades), and a new point goes to whichever side shouts louder where it stands. The kernel trick is the accounting identity that lets this infinitely-rich comparison be computed with one cheap formula per pair of points.

14.5.1 Linear versus RBF boundaries, drawn

The kernel changes the *shape* of boundary the SVM is allowed to use. The same data can therefore be impossible for a linear SVM and easy for an RBF SVM.

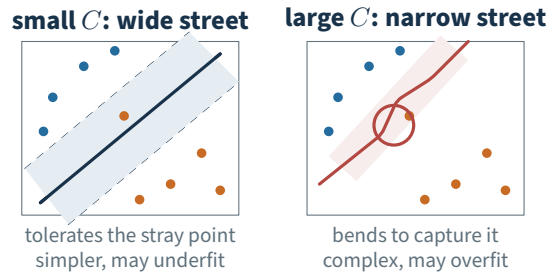


Key idea

A kernel does not “bend” a trained straight line afterward. It changes the similarity representation used while fitting, which lets a linear separator in that representation correspond to a curved boundary in the original feature space.

14.6 What C and γ actually do

An SVM has two dials, and beginners routinely turn them without knowing which way is which.



Dial	Plain meaning	Turn it up when
C	How badly you want to avoid mistakes on the training data	The model underfits; the boundary is too simple
γ	How far a single training point's influence reaches	The boundary needs to be more local and wiggly

Analogy

C is a strictness setting for a teacher marking homework. A low C teacher accepts that a few students will get things wrong and teaches a general rule. A high C teacher insists that every single student's answer be accommodated, and ends up teaching a rule so contorted that nobody can apply it to a new question.

γ is the radius of each training point's voice. Low γ means every point is heard across the whole room, producing a smooth global boundary. High γ means each point is only heard by its immediate neighbors, producing islands around individual examples.

14.7 Multiclass SVMs and support vector regression

The SVM is natively binary. Multiclass problems are handled by reduction: **one-vs-rest** trains K machines (each class against all others), while **one-vs-one** trains a machine for every pair of classes and lets them vote; both work well.

Support vector regression (SVR) recycles the margin idea for numeric labels: build a tube of half-width ε around the prediction and charge nothing for points inside it, using the ε -insensitive loss $\max(0, |y - f(x)| - \varepsilon)$. Only points on or outside the tube become support vectors, giving a sparse, outlier-tolerant regressor with the full kernel toolbox available.

14.8 Practical notes

- **Scale the features.** Apply the distance-geometry rule from Section 3.11.
- **Tune C (and γ)** by cross-validation over logarithmic grids such as $10^{-2}, \dots, 10^3$.
- **Know the sweet spot:** SVMs shine on small-to-medium datasets, high-dimensional problems, and text (linear kernel); training scales poorly to hundreds of thousands of observations, where tree ensembles or linear models are more practical.
- **Probabilities are not native.** SVMs output scores and margins; if calibrated probabilities are needed, apply and validate a calibration step (Chapter 7).

14.9 Choosing a supervised model

Stage 3B ends here, so the supervised model-family menu can now be laid on the table.

Model	Useful strengths	Important limitations
Linear regression	Interpretable, fast, the standard first baseline	Linear relationships only; sensitive to outliers
Ridge / lasso	Controls overfitting; lasso selects features	Still linear; needs feature scaling
Logistic regression	Strong probability baseline, convex training, interpretable odds	Linear boundary; calibration still needs checking
LDA / QDA	Fast, stable on small data	Gaussian class assumptions
k -NN	No training, flexible boundaries	Slow at prediction; scale- and dimension-sensitive
Naive Bayes	Very fast, strong text baseline	Independence and calibration caveats (Section 11.3)
Decision tree	Readable rules, no scaling, mixed data	Unstable; overfits without pruning
Random forest	Robust, accurate, minimal tuning	Larger, slower, less transparent
Gradient boosting	Often a very strong tabular benchmark	Needs careful tuning and early stopping
SVM	Margins and kernels; strong in high dimensions	Scaling required; slow on very large data

A sensible default path: start with a regularized linear model as the baseline; try a random forest for a near-free accuracy check; reach for tuned gradient boosting when accuracy matters most; and consider an SVM when the data is high-dimensional or modest in size. Let validation results — not fashion — make the final call.

Key idea

The SVM contributes two permanent ideas to machine learning. First, among all boundaries that fit, prefer the one with the largest safety margin — a built-in form of regularization. Second, a linear method plus the kernel trick becomes a nonlinear method at almost no conceptual cost. Margins and kernels outlived the SVM's peak popularity and now echo throughout the field.

Support Vector Machines - Quick Guide

Best when	The dataset is small to medium sized, features are informative, and a strong margin-based boundary is useful.
Preprocessing	Standardize numerical features; encode categories; remove clearly irrelevant features when justified by the validated workflow in Section 8.8.
Main controls	C for margin violations; kernel choice; γ for RBF-kernel reach; ε for SVR.
Strengths	Effective in high-dimensional spaces and flexible through kernels.
Watch for	Slow training on very large datasets, sensitive scale, difficult probability interpretation, and expensive kernel tuning.

PRACTICAL MILESTONE

Lab 7 of 10

Support vector machines

Tune a margin-based classifier inside a leakage-safe pipeline

Coverage. Chapter 14.

Source files. Lab07/Task01.ipynb.

Core data. Breast Cancer Wisconsin.

Goal. Standardize features inside the pipeline, tune SVM capacity with cross-validation, and interpret how C and kernel settings change flexibility.

How much is scaffolded?

This is the final supervised model-family lab. Reuse the validation discipline established in Lab 3 rather than inventing a new evaluation process for SVMs.

Part A — Standardized SVM tuning.

```
from pathlib import Path
import pandas as pd
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "breast_cancer_dataset" / "data.csv"
df = pd.read_csv(train_path)
```

```
X = df.drop(columns=["id", "diagnosis", "Unnamed: 32"], errors="ignore")
y = (df["diagnosis"] == "M").astype(int)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", SVC()),
])
search = GridSearchCV(
    pipe,
    param_grid={
        "model__kernel": ["linear", "rbf"],
        "model__C": [0.1, 1, 10, 100],
        "model__gamma": ["scale", 0.01, 0.1],
    },
    cv=StratifiedKFold(5, shuffle=True, random_state=42),
    scoring="roc_auc",
    n_jobs=-1,
)
```

```
search.fit(X_train, y_train)
score = search.decision_function(X_test)
pred = search.predict(X_test)
print("best:", search.best_params_)
print("accuracy:", round(accuracy_score(y_test, pred), 3))
```

```
print("ROC AUC:", round(roc_auc_score(y_test, score), 3))
```

Lab deliverable

Submit the cross-validated SVM settings, validation evidence for the selected configuration, and a short explanation of why scaling and hyperparameter selection must remain inside the validation procedure.

Check your understanding

1. What is the geometric meaning of a larger separating margin in an SVM?
2. How do C and, for an RBF kernel, γ change the flexibility of the fitted decision boundary?
3. Why should scaling and SVM hyperparameter selection both happen inside the validation procedure?

Chapter recap

- A linear SVM chooses a separating boundary with the widest margin.
- Support vectors are the training points that determine the boundary.
- The parameter C balances a wide margin against training violations.
- Kernels create nonlinear boundaries through similarity calculations in a richer feature space.
- Feature scaling follows the distance-geometry rule in Section 3.11.

Stage 4

Unsupervised Learning

Stage 4A

Unsupervised Workflow

Reset the objective: no label is supplied during fitting, so validation must focus on structure, stability, usefulness, and domain evidence.

How Unsupervised Learning Works

Chapter at a glance

Before you start: The shared foundations in Stages 1–2 and the supervised workflow in Stage 3A for comparison.

By the end, you can: explain what changes when no label is supplied, distinguish the main unsupervised tasks, evaluate structure without pretending there is always a ground-truth answer, and follow a safe unsupervised workflow.

Stage 4A begins by resetting the workflow before any unsupervised model family is chosen. Until now, the main modeling chapters had a label y . That label gave training a direction and made evaluation comparatively direct. Unsupervised learning removes that answer key. The input is usually just a feature matrix X , and the goal is to discover or construct useful structure.

Supervised
 $X + y \rightarrow \text{predict } y$

remove the supplied label

Unsupervised
 $X \rightarrow \text{discover structure}$

Core path

Remember one question: **what structure am I trying to preserve or discover?** “Unsupervised learning” is not one objective. Clustering asks for groups, dimensionality reduction asks for a useful lower-dimensional representation, anomaly detection asks which cases are unusual, and density estimation asks where probability mass lies. Association / pattern mining asks which items repeatedly occur together; it appears on the field map for orientation but is **beyond the core of this edition**.

15.1 The unsupervised-learning map

Question	First framing	Main output
Do observations form groups?	Clustering	Cluster assignments, centers, hierarchy, or soft memberships
Can the data be represented with fewer dimensions?	Dimensionality reduction / representation	Components or lower-dimensional coordinates
Which observations are unusual?	Anomaly / novelty detection	Anomaly score or ranked suspicious cases
Where is the data concentrated?	Density estimation	Estimated probability density or mixture components
Which items occur together?	Association / pattern mining (<i>beyond core</i>)	Frequent itemsets and rules

Key idea

These tasks can share algorithms and geometry without sharing an evaluation rule. A good clustering is not defined by explained variance; a good PCA representation is not defined by silhouette score; a useful anomaly detector may be judged by review precision rather than a cluster metric.

15.2 What does fitting mean without labels?

An unsupervised method still has an objective; it simply does not compare predictions with a supplied label during fitting.

Method family	What fitting tries to do	What is learned
<i>K</i> -means	Reduce squared distance from points to assigned centers	Cluster centers and assignments
PCA	Preserve as much variance as possible in a lower-dimensional linear representation	Principal directions and projected coordinates
Density / mixture models	Assign high likelihood to the observed feature distribution under a chosen family	Distribution parameters and soft memberships
Isolation / local methods	Make unusual observations easy to isolate or locally sparse	Anomaly scores / rankings
Association mining (beyond core)	Find item combinations that occur often enough to be interesting	Frequent itemsets and conditional rules

Beyond the core

Association / pattern mining is included here so the unsupervised map is complete, but it is not developed as a Stage 4B model family in this beginner edition. Stage 4B deliberately concentrates on three reusable objectives: grouping observations, building representations, and ranking unusual cases.

15.3 How do we evaluate when there is no answer key?

There is no universal “unsupervised accuracy.” Use evidence that matches the task.

15.3.1 Internal evidence

Uses the features and fitted structure itself: cluster compactness/separation, reconstruction error, explained variance, density, or stability under resampling and reasonable perturbations.

15.3.2 External evidence

Sometimes known labels exist for analysis even though the algorithm did not use them for fitting. Metrics such as adjusted Rand index can compare a clustering with those labels. Treat this as an external diagnostic, not proof that the discovered clusters are the only valid structure.

15.3.3 Practical or downstream evidence

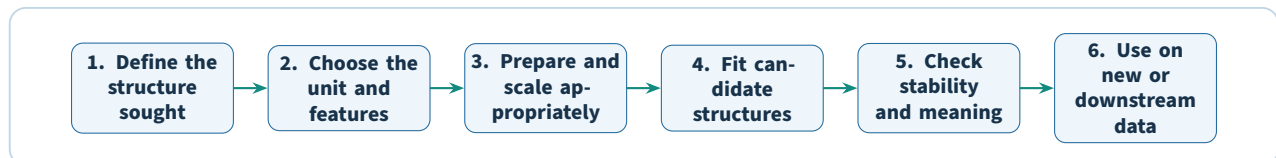
Often the strongest question is whether the representation or discovered structure is useful: are clusters interpretable and stable, does a compressed representation preserve a downstream signal, and do highly

ranked anomalies correspond to cases a reviewer cares about?

Common mistake

Do not evaluate an unsupervised method by inventing a convenient label after the fact and then treating that label as ground truth. Clusters may cut across known categories for legitimate reasons, and a visually pleasing two-dimensional plot can distort neighborhoods. State what property the method was designed to preserve.

15.4 The unsupervised workflow



A train/test split is not automatically required for every exploratory clustering of one fixed dataset. It *is* required when you make a generalization claim, tune a reusable transform using downstream labels, build a novelty detector for future observations, or otherwise claim that the learned structure will transfer. The information-boundary rule from Chapter 3 still applies whenever a transformation is fitted and then evaluated on held-out data.

15.5 Scaling and representation matter even more here

Many unsupervised algorithms see only geometry. If annual income is measured in hundreds of thousands and age in tens, a raw Euclidean distance can become mostly an income distance. Standardization, feature weighting, encoding, and the choice of distance therefore define part of the problem rather than merely polishing the input.

Key idea

In supervised learning, a label can sometimes reveal that a poor representation is unhelpful. In unsupervised learning there may be no such corrective signal. The representation and distance choice therefore deserve explicit justification before interpreting the structure that appears.

15.6 The sequence ahead

- | | |
|-------------------|--|
| Chapter 16 | asks whether observations form meaningful groups and compares center-, hierarchy-, density-, and probability-based approaches. |
| Chapter 17 | asks what information should be preserved when features are compressed, with PCA as the core method and nonlinear visualization as an extension. |
| Chapter 18 | asks which observations are unusual and how anomaly scores become operational alerts. |

Check your understanding

1. Why is there no universal unsupervised-learning accuracy score?
2. Give one internal, one external, and one downstream form of evidence that could support an unsupervised result.
3. When does an unsupervised project need a held-out split even though no label is used during

fitting?

Chapter recap

- Unsupervised learning fits structure in X without using a supplied label during fitting.
- Clustering, dimensionality reduction, anomaly detection, and density estimation answer different questions and therefore need different objectives and evaluation evidence.
- Internal, external, and practical/downstream evaluation are complementary; there is no universal unsupervised accuracy score.
- Scaling, representation, and distance choices are part of the model because many unsupervised methods are geometric.

Stage 4B

Unsupervised Model Families

Apply the unsupervised workflow to clustering, representation learning, and anomaly detection without pretending that internal structure is automatically ground truth.

Clustering

Chapter at a glance

Before you start: Chapter 15, feature scaling, vectors, distances, and the Gaussian distribution from Chapter 5.

By the end, you can: distinguish hard from soft clustering, run K -means by hand, choose and justify cluster structure, and compare K -means, fuzzy C -means, Gaussian mixtures, hierarchical clustering, and DBSCAN under the Stage 4A evaluation workflow.

Stage 4A already established what changes when no label is supplied and how unsupervised evidence should be judged. Stage 4B now holds that workflow fixed while changing the model family. We begin with **clustering**: methods that propose a grouping, hierarchy, density partition, or soft membership structure for observations.

The central question is no longer “what is unsupervised learning?” but **what notion of a group does this clustering method assume?** K -means prefers compact center-based groups; hierarchical methods expose nested structure; DBSCAN defines groups through density connectivity and can leave points as noise; mixture models assign probabilistic memberships. The quantitative score is therefore evidence about a chosen structure, not an answer key.

Without labels there is no single correct output to compare against, so results must be judged by a combination of quantitative scores, stability checks, visualization, and domain knowledge. The Stage 4A warning applies throughout this chapter: internal metrics can compare candidate structures, but they do not prove that a cluster is meaningful for the application.

Clustering asks: do the observations form useful, stable groups — customer segments, document topics, cell types? For distance-based clustering, reuse the scaling decision from Section 3.11; original units may be retained when they intentionally encode domain priorities.

Core path

Core path: understand K -means, scaling, cluster evaluation, and DBSCAN. Treat Gaussian mixtures, fuzzy C -means, and hierarchical clustering as additional cluster shapes to learn after the two core ideas are comfortable.

Assignment style	Meaning	Examples
Hard clustering	Every row receives one cluster label. The memberships are mutually exclusive in the final partition.	K -means, K -medoids, DBSCAN, agglomerative clustering
Soft clustering	Every row can have partial membership in several groups, usually summarized by a vector of membership degrees or component responsibilities.	Fuzzy C -means, Gaussian mixture models

Hard and soft are about the *form of the assignment*, not about whether an algorithm is better. If customers can genuinely sit between segments, a membership vector may preserve useful ambiguity that a single

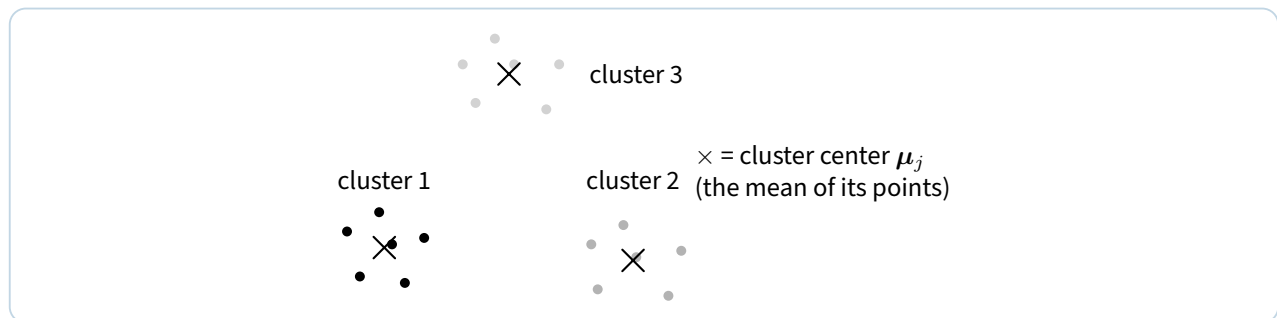
label throws away.

16.1 K -means

K -means divides observations into K groups by minimizing squared distances from group centers:

$$\arg \min_{S_1, \dots, S_K} \sum_{j=1}^K \sum_{x_i \in S_j} \|x_i - \mu_j\|_2^2.$$

In plain language, the formula says: choose K centers, assign every row to a center, and move the centers until the rows are as close as possible to the center of their own group.

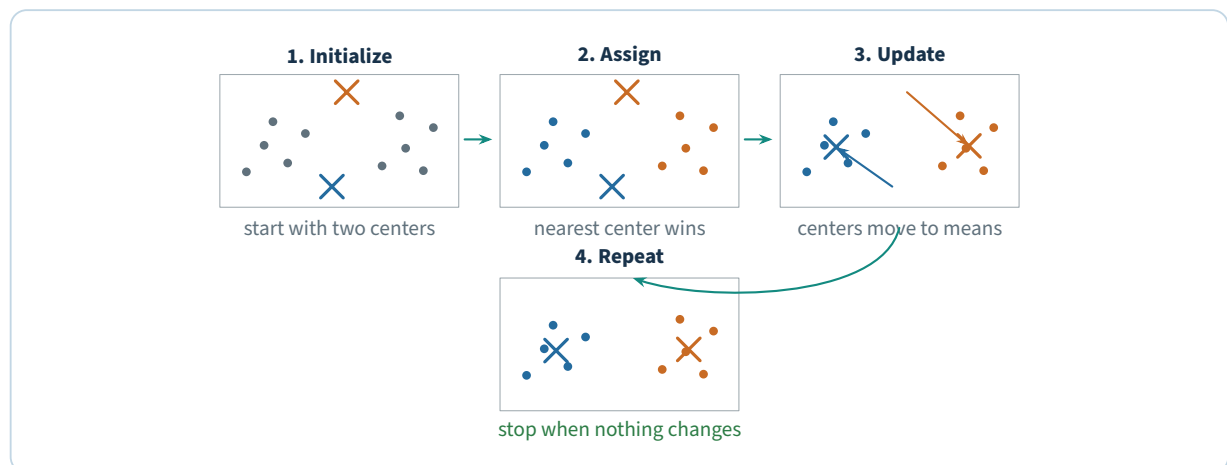


The algorithm (Lloyd's algorithm) alternates between two steps until nothing changes:

1. **Assign:** send each observation to its nearest center.
2. **Update:** move each center to the mean of the observations assigned to it.

16.1.1 K -means as a four-frame loop

The two instructions above become easier to remember as a repeated picture: choose centers, assign points, move centers, and repeat until the assignments stop changing.



Worked example

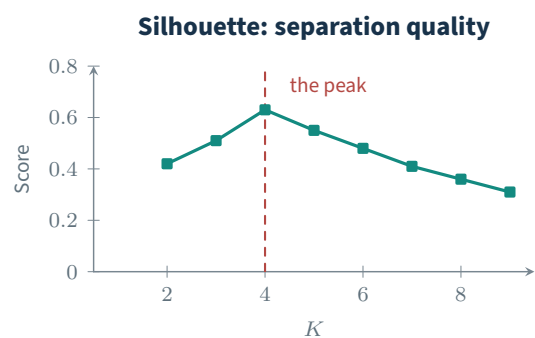
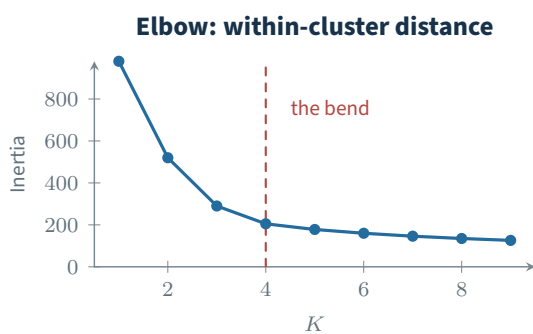
Cluster the one-dimensional points $\{1, 2, 3, 10, 11, 12\}$ with $K = 2$, starting from centers $c_1 = 1$ and $c_2 = 12$. **Assign:** points 1, 2, 3 are nearer c_1 ; points 10, 11, 12 are nearer c_2 . **Update:** $c_1 = (1+2+3)/3 = 2$ and $c_2 = (10+11+12)/3 = 11$. **Assign again:** nothing moves (e.g. point 3 is at distance 1 from c_1 and 8 from c_2), so the algorithm stops. The objective is $(1+0+1) + (1+0+1) = 4$, and the two obvious groups have been found in one round.

Each assignment/update step cannot increase the objective, so with a consistent rule for distance ties, Lloyd's K -means algorithm reaches a stable solution after finitely many distinct assignments. That solution need not be globally best and depends on the starting centers. Standard practice is the **k -means++** initialization, which picks starting centers that are spread out, combined with several random restarts, keeping the run with the lowest objective. For very large datasets, **mini-batch K -means** updates centers from small random batches and trades a little accuracy for a lot of speed.

K -means implicitly assumes clusters are compact, roughly spherical, and similar in size; it struggles with elongated or irregular shapes, very different cluster sizes, and outliers (a single far-away point drags its center toward it).

16.2 Choosing K : the elbow and the silhouette

Clustering has no correct answer to check against, so choosing the number of clusters relies on two diagnostics rather than one.



Plain idea

Inertia always falls as K rises — give every point its own cluster and it reaches zero — so you cannot simply minimize it. Look for a bend after which extra clusters buy little. The silhouette has a maximum, but it also embeds a preference for compact, separated groups. Agreement between diagnostics supports a candidate K ; it does not establish that the groups are stable or useful.

16.3 Lloyd's algorithm and initialization

Lloyd's algorithm alternates between two operations: assign each observation to its nearest center, then replace each center with the mean of the observations currently assigned to that cluster. Each iteration cannot increase the within-cluster sum of squares, so the procedure eventually stops changing.

The important limitation is that the objective is not globally convex in all cluster centers at once. Different initial centers can therefore lead to different local optima. A poor start may produce a clustering with substantially larger inertia than a better start even when the same value of K is used.

The practical principle is simple: use multiple initializations and compare their final objective values and assignments. If the same K repeatedly produces very different solutions, that instability is evidence that the proposed cluster structure is weak or ambiguous rather than a detail to hide.

Key idea

Never interpret one random initialization as definitive. Stability across sensible initializations or resamples is part of the evidence for a useful clustering.

16.3.1 K -medoids

K -medoids is a robust alternative to K -means: each cluster is represented by an actual observation (its **medoid**) rather than a mean, and any distance function may be used. Medoids resist outliers (a far point cannot drag a medoid the way it drags a mean) and make sense for data where averaging is meaningless — at a higher computational price.

16.4 Gaussian mixture models

A Gaussian mixture model (GMM) is a probabilistic relative of K -means. It assumes that the observations were generated from K Gaussian components with mixing weights π_k :

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \Sigma_k), \quad \sum_{k=1}^K \pi_k = 1.$$

Instead of hard assignments, each observation receives a **responsibility**: the posterior probability that component k generated it,

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \Sigma_j)}.$$

The **expectation-maximization (EM)** algorithm alternates between an E-step, which computes the responsibilities with the current parameters, and an M-step, which re-estimates π_k , $\boldsymbol{\mu}_k$, and Σ_k as responsibility-weighted averages. Each cycle increases the data log-likelihood until convergence.

Plain idea

EM solves a chicken-and-egg problem. If we knew which cluster each point belonged to, estimating the cluster shapes would be easy; if we knew the cluster shapes, assigning points would be easy. Since we know neither, EM alternates: guess the memberships, refit the shapes, re-guess the memberships, and repeat until the answer stops changing. K -means does the same dance with hard yes/no assignments.

What the extra machinery buys, compared with K -means: *soft* memberships (“62% component 1, 38% component 2”), *elliptical* components of different shapes and sizes through the covariance matrices Σ_k , and a probability density that can also support anomaly scoring (Chapter 18). These probabilities are conditional on the mixture assumptions; they are not proof that a real-world entity literally belongs to a group with that probability.

16.5 Fuzzy C -means: soft distance-based clustering

Fuzzy C -means (FCM) keeps the centroid idea of K -means but replaces a one-hot assignment with membership degrees $u_{ik} \in [0, 1]$ that sum to 1 across clusters for each observation. A common objective is

$$\sum_i \sum_{k=1}^K u_{ik}^m \|\mathbf{x}_i - \mathbf{c}_k\|_2^2,$$

where $m > 1$ is the **fuzziness parameter**. Larger m produces softer memberships; $m = 2$ is a common teaching default.

The algorithm alternates between two updates:

1. recompute each centroid as a membership-weighted mean;
2. recompute each observation’s memberships from its relative distances to all centroids.

The iterations stop when memberships or centers change by less than a chosen tolerance.

Plain idea

K -means asks, “Which center is closest?” FCM asks, “How strongly does each center describe this point?” A boundary case might be 0.52 in cluster A and 0.45 in cluster B instead of being forced to pretend it is unambiguously A.

Common mistake

FCM membership values are not automatically calibrated probabilities. They are distance-based degrees determined by the chosen scale, number of clusters, and fuzziness parameter. Gaussian-mixture responsibilities have a probabilistic model underneath them; FCM memberships do not make the same generative claim.

16.6 Hierarchical clustering

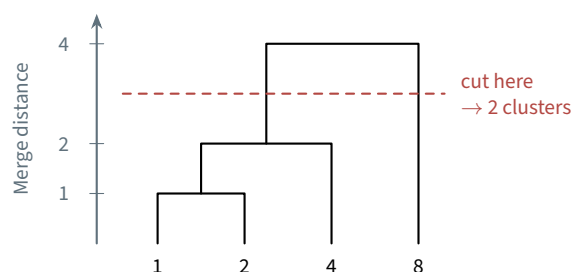
Hierarchical clustering builds a whole family of clusterings at once, organized as a tree called a **dendrogram**. The common **agglomerative** version starts with every observation in its own cluster and repeatedly merges the two closest clusters until one remains.

Direction	Start	Repeated operation
Agglomerative (bottom-up)	One cluster per observation	Merge the closest clusters until one hierarchy remains
Divisive (top-down)	One cluster containing all observations	Split a current cluster into smaller groups until a stopping rule is reached

Agglomerative clustering is far more common in general-purpose software because the merge sequence is straightforward to compute and visualize. Divisive methods work in the opposite direction and need an explicit rule for which cluster to split and how to split it; a practical divisive strategy can use two very dissimilar observations as seeds for a two-way partition, then recurse.

Worked example

Cluster the points $\{1, 2, 4, 8\}$ on a number line with single linkage. The closest pair is $\{1\}$ and $\{2\}$ (distance 1) — merge them. The closest pair is now $\{1, 2\}$ and $\{4\}$ (distance $|4 - 2| = 2$) — merge. Finally $\{1, 2, 4\}$ meets $\{8\}$ at distance $|8 - 4| = 4$. The dendrogram records each merge at the height of its distance:



Cutting the dendrogram at any height produces a clustering, so the number of clusters is chosen *after* seeing the whole merge history — a genuine advantage when K is unclear. The meaning of “closest clusters” is set by the **linkage** rule:

- **single linkage** uses the closest pair of observations — follows chains, finds elongated clusters, sensitive to noise bridges;

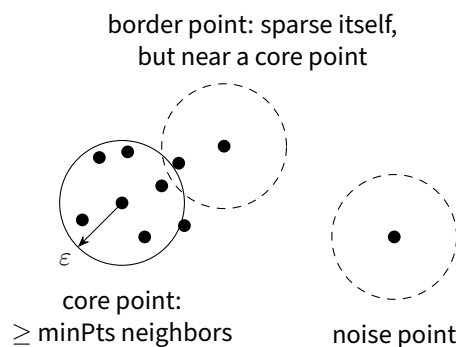
- **complete linkage** uses the farthest pair — compact, similar-diameter clusters;
- **average linkage** uses the average pairwise distance — a middle ground;
- **Ward linkage** merges whichever pair least increases within-cluster variance — behaves most like K -means and is a common default.

The price of the full merge history is computation: pairwise distances make the method expensive for large datasets, and it offers no built-in way to assign brand-new observations to existing clusters.

16.7 DBSCAN: clustering by density

DBSCAN defines clusters as regions where points are packed densely, using two hyperparameters: a neighborhood radius ϵ and a minimum count **minPts**. Every point gets one of three roles:

- Core point** Has at least minPts points within distance ϵ — it sits in a dense region.
- Border point** Is within ϵ of a core point but is not dense enough itself — the edge of a cluster.
- Noise point** Neither — it belongs to no cluster at all.



A cluster is then grown outward: start from any core point, absorb everything within ϵ , and keep expanding through core points until the density runs out; border points join the cluster that reached them, and noise stays unlabeled. The consequences are exactly DBSCAN's selling points — it discovers clusters of arbitrary shape, needs no K in advance, and labels outliers explicitly — and its weaknesses: a single $(\epsilon, \text{minPts})$ pair cannot fit clusters of very different densities, and in high dimensions distance itself becomes uninformative. A common way to choose ϵ is to plot each point's distance to its k -th nearest neighbor, sorted, and take the elbow.

16.8 Which clustering method?

Method	Needs K ?	Cluster shapes	Outliers	Notable trait
K -means	yes	compact, spherical	distort means	fast, scalable, simple
GMM	yes	elliptical	distort fits	soft memberships, density
Hierarchical	no (cut later)	depends on linkage	can distort merges	full dendrogram of groupings
DBSCAN	no	arbitrary	labeled as noise	density-based, finds outliers

K -means and GMM also assign new observations naturally (nearest center; highest responsibility); hierarchical clustering and DBSCAN have no built-in rule for unseen points, so a small classifier trained on the cluster labels often fills that gap in practice.

16.9 Evaluating a clustering

The **silhouette coefficient** for observation i compares cohesion with separation:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where $a(i)$ is its average distance to its own cluster and $b(i)$ is the smallest average distance to another cluster. Values near 1 indicate a point nestled deep in its own cluster; values near 0 sit between clusters; negative values suggest a probable misassignment. Averaging $s(i)$ over the data scores a whole clustering and can compare different K .

When reference labels happen to exist (for validation only), agreement measures such as the **adjusted Rand index** compare the discovered clusters with the labeled groups while correcting for chance.

Key idea

No internal score can prove that clusters are *meaningful*. A trustworthy clustering earns its keep three ways: reasonable scores, stability under resampling and reasonable parameter changes, and an interpretation a domain expert recognizes. Treat every clustering as a hypothesis about the data, not a fact extracted from it.

Clustering - Quick Guide

Method	Good for	Main caution
K-means	Compact, roughly spherical clusters; large numeric datasets	Requires K , scaling, and sensitivity to initialization/outliers
Gaussian mixture	Overlapping elliptical clusters and soft membership	Distribution assumptions and local optima
Hierarchical	Dendrograms and nested group structure	Distance/linkage choices and cost on large data
DBSCAN	Irregular shapes and explicit noise points	Density parameters and varying-density clusters

Check your understanding

1. What kind of cluster geometry does K -means favor, and why can scaling change the resulting partition?
2. Why can a high silhouette score fail to prove that a segmentation is useful in the application?
3. What does DBSCAN do with observations that do not belong to a sufficiently dense region?

Chapter recap

- Clustering searches for groups without supplied labels.
- K -means alternates between assigning points and updating centroids.
- Fuzzy C -means gives distance-based soft memberships; Gaussian mixtures give model-based component responsibilities.

- Hierarchical clustering can build groups bottom-up by merging or top-down by splitting, and a dendrogram records the hierarchy.
- DBSCAN finds dense regions and labels isolated points as noise.
- A clustering must be evaluated for stability, separation, and usefulness in the real application.

Dimensionality Reduction and Representation

Chapter at a glance

Before you start: Chapter 15, dot products and projections from Chapter 5, and scaling from Chapter 3. Eigenvectors and eigenvalues are re-introduced from scratch here.

By the end, you can: Run PCA step by step, decide how many components to keep, interpret what is gained and lost, and use t-SNE and UMAP for visualization without over-reading their pictures.

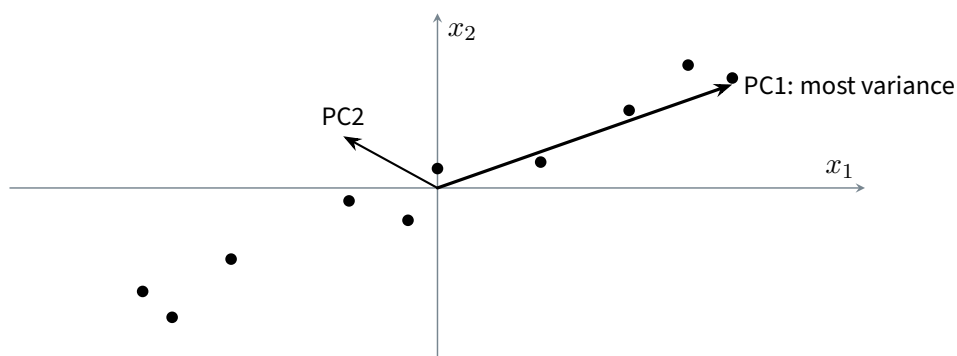
Real datasets often carry many redundant or weakly informative features: sensors that echo one another, survey questions that ask similar things, or thousands of pixels describing a handful of shapes. Dimensionality reduction is the classical form of representation learning developed in this book: it compresses data into fewer variables while preserving a stated property, such as variance or local neighborhoods. Possible payoffs include visualization, denoising, faster downstream models, and relief from the distance problems discussed in Chapter 11.

Core path

Core path: PCA, explained variance, component choice, and the fact that compression loses information. Treat t-SNE and UMAP as second-pass visualization methods; they answer a different question from PCA.

17.1 The idea: find better axes

A cloud of correlated points does not use its dimensions efficiently — most of the action happens along a few directions. Principal component analysis (PCA) rotates the coordinate system to point along those directions.

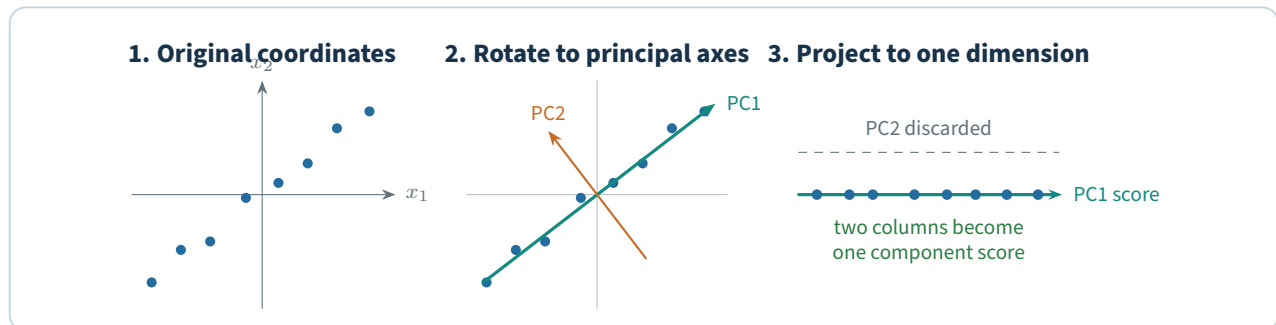


Plain idea

Think of photographing a school of fish. From most angles the school looks like a blob, but from one particular angle its long axis lines up with the frame and one photo captures almost everything. PCA finds that best camera angle: the first principal component is the direction along which the data is most spread out, the second is the most-spread direction perpendicular to the first, and so on.

17.1.1 PCA as rotate, then project

The first component is not merely an arrow drawn on the old graph. It becomes a new coordinate axis, and every observation receives a new number: its position along that axis.



17.2 PCA step by step

1. **Center** the data by subtracting each feature's training mean. Decide separately whether to standardize: do so when equal standardized variation should carry equal weight, but retain original units when their variance is scientifically meaningful.
2. Compute the sample covariance matrix

$$S = \frac{1}{n-1} X^T X$$

for the centered matrix X .

3. Find its eigenvectors and eigenvalues: for centered data matrix X ,

$$Sv = \lambda v.$$

Each eigenvector v is a component direction; its eigenvalue λ measures the variance captured along it.

4. **Sort** components by eigenvalue and keep the top k .
5. **Project:** the new features (**scores**) are the dot products of each observation with the kept components.

The proportion of variance explained by component j is

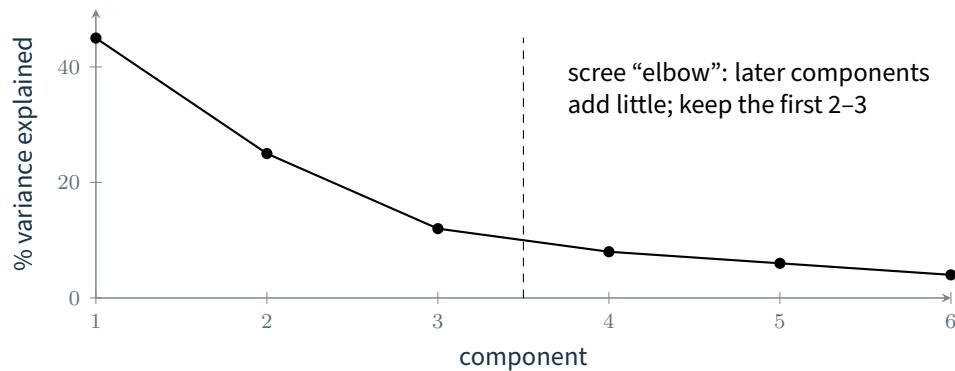
$$\frac{\lambda_j}{\sum_k \lambda_k},$$

which reports how much of the dataset's total variance lies along each new axis. The predictive-value caveat is stated once in Section 17.3.1. Because the components are eigenvectors of a symmetric matrix, they are mutually perpendicular, and their scores are uncorrelated in the fitted sample. The singular value decomposition introduced in Chapter 5 computes the same directions directly from the centered data matrix, which is how software commonly implements PCA.

Worked example

Take five centered points that lie exactly on a line: $(-2, -2)$, $(-1, -1)$, $(0, 0)$, $(1, 1)$, $(2, 2)$. Each feature has variance $10/4 = 2.5$ and their covariance is also 2.5, so the covariance matrix is $\begin{bmatrix} 2.5 & 2.5 \\ 2.5 & 2.5 \end{bmatrix}$ with eigenvalues $\lambda_1 = 5$ and $\lambda_2 = 0$. The first component is $v_1 = (1, 1)/\sqrt{2}$, and it explains $5/(5+0) = 100\%$ of the variance. Projecting gives one score per point — $-2.83, -1.41, 0, 1.41, 2.83$ — so two columns of data have been compressed to one with *no loss at all*. Real data is never this tidy, but the mechanism is identical: keep the big- λ directions, drop the small ones, lose only what the small ones carried.

17.3 Explained variance: the compression budget



Three complementary rules: use a cumulative-variance threshold when the application can justify one; look for an elbow in the **scree plot**; or, when PCA feeds a supervised model, treat k as a hyperparameter and let training-fold validation decide. Under squared reconstruction loss, retaining the top k components gives the best rank- k linear reconstruction, and the discarded eigenvalues quantify the average variance lost.

17.3.1 What PCA cannot promise

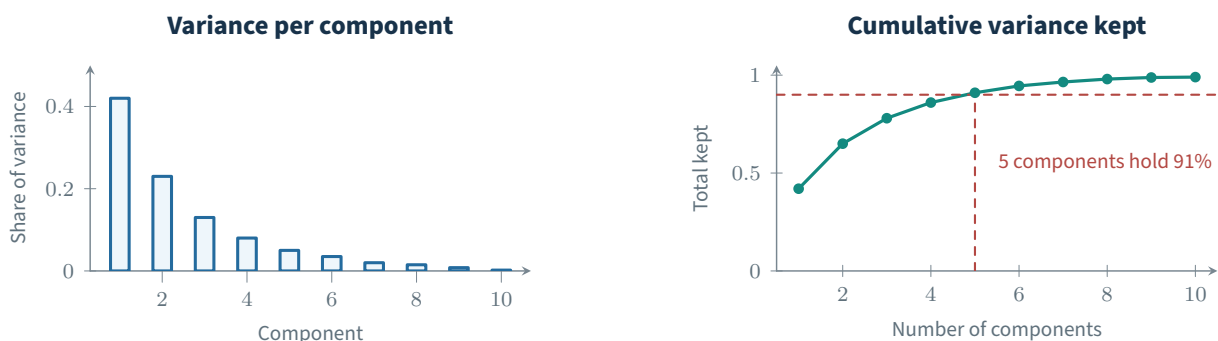
- It is **linear**: a spiral or a curved sheet of data has structure PCA cannot straighten out.
- It is **scale-sensitive**: changing units changes the covariance matrix. Standardize when that influence is arbitrary, not as an automatic rule.
- **Variance is not predictive importance**: the directions that vary most are not necessarily the ones that predict a label — a low-variance component can carry crucial signal. (Chapter 10's LDA is a supervised method that seeks class-separating directions instead.)
- **Components resist interpretation**: each is a weighted blend of all original features, so “component 3 increased” rarely means anything to a domain expert without inspecting its weights.

Common mistake

Using PCA to *plot* clusters is not the same as clustering on the PCA coordinates. If K -means is fitted on four scaled features and the result is then projected to two principal components for a picture, the assignments came from four-dimensional geometry. Fitting K -means on only the two PCA columns is a different model that intentionally discards some variance.

17.4 Choosing the component count in practice

PCA gives you as many components as you like, ordered by how much variance each captures.



Analogy

PCA is like photographing a chair. Photograph it from directly above and you see a confusing square. Photograph it from the side and it is instantly recognizable — same object, better angle. PCA searches for the angle from which the data varies the most, then the next best angle at right angles to that, and so on. The scree plot tells you how many angles you actually need before the rest is just dust.

17.5 t-SNE and UMAP: maps for the eye

PCA preserves global variance; sometimes the goal is instead a two-dimensional *map* in which similar observations sit together — to eyeball cluster structure in document word-counts, customer measurements, or product features. Two nonlinear methods dominate this niche.

t-SNE converts distances into neighbor probabilities and arranges points in 2-D so that neighborhoods are preserved: points close in the original space stay close on the map. Its main hyperparameter, **perplexity**, roughly sets the neighborhood size being honored (typical values 5–50).

UMAP pursues the same goal with different mathematics; it is much faster, tends to preserve somewhat more of the global arrangement, and — unlike classic t-SNE — can place *new* points onto an existing map.

Both need cautious reading:

- cluster sizes, empty gaps, and distances between separated groups can be artifacts; even apparent neighborhoods should be checked against original-space distances;
- different random seeds and hyperparameters can produce visibly different maps, so check that the story survives a few reruns;
- a map can suggest questions but cannot demonstrate that discrete clusters exist; confirm any claimed grouping with stability, held-out information, and domain evidence;
- for ordinary predictive preprocessing, PCA is the simpler default because it has an explicit out-of-sample projection. Nonlinear embeddings need a justified validation design.

	PCA	t-SNE / UMAP
Preserves	global variance, linear structure	local neighborhoods
Deterministic?	yes	no (seed-dependent)
Handles new points	yes (a projection)	UMAP yes; t-SNE awkwardly
Best use	preprocessing, compression, denoising	exploratory visualization

Note the family distinction: PCA, t-SNE, and UMAP perform **feature extraction** — they build new variables. **Feature selection** (Section 8.8) instead keeps a subset of the original variables, often retaining more direct interpretability.

Key idea

Dimensionality reduction trades a little information for a lot of manageability. PCA makes the trade with an exact receipt — the discarded eigenvalues are the price paid — while t-SNE and UMAP make beautiful maps whose distances must not be over-read. Reduce to work; visualize to explore; and never mistake the picture for the data.

Dimensionality Reduction - Quick Guide

PCA is best for	Compressing correlated numerical features, denoising, visualization, and faster downstream modeling.
Preprocessing	Center features; standardize when unit scales are arbitrary and equal standardized influence is intended.
Main control	Number of components, chosen by explained variance and downstream validation.
Visualization tools	t-SNE and UMAP can create useful two-dimensional maps of local neighborhoods.
Watch for	Loss of interpretability, information loss, leakage from fitting on all data, and over-interpreting visual clusters.

PRACTICAL MILESTONE

Lab 8 of 10

Unsupervised structure, clustering, and PCA

Search for structure without pretending an answer key exists

Coverage. Chapters 15–17.

Source files. Lab08/Task01.ipynb through Task03.ipynb.

Core data. Mall Customers and Optical Recognition of Handwritten Digits.

Goal. Define an unsupervised question, compare K-means with DBSCAN, and evaluate PCA as a representation/compression step using downstream validation.

How much is scaffolded?

This lab deliberately joins workflow, clustering, and representation learning. The methods are different, but all require you to define what useful structure means before interpreting a plot or cluster label.

Part A — K-means diagnostics.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "mall_customers_dataset" / "Mall_Customers.csv"
df = pd.read_csv(train_path)
```

```
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = df[features]
X_scaled = StandardScaler().fit_transform(X)
```

```
ks, inertias, silhouettes = [], [], []
```

```
for k in range(2, 9):
    model = KMeans(n_clusters=k, n_init=20, random_state=42)
    labels = model.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    ks.append(k)
    inertias.append(model.inertia_)
    silhouettes.append(sil)
    print("k=", k, "inertia=", round(model.inertia_, 2),
          "silhouette=", round(sil, 3))
```

```
plt.figure(figsize=(6, 4))
plt.plot(ks, inertias, marker="o")
plt.xlabel("Number of clusters k")
plt.ylabel("Inertia")
plt.title("K-means elbow diagnostic")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(6, 4))
```

```
plt.plot(ks, silhouettes, marker="o")
plt.xlabel("Number of clusters k")
plt.ylabel("Silhouette score")
plt.title("K-means silhouette diagnostic")
plt.tight_layout()
plt.show()
```

Part B — K-means versus DBSCAN.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN, KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "mall_customers_dataset" / "Mall_Customers.csv"
df = pd.read_csv(train_path)
```

```
features = ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
X = StandardScaler().fit_transform(df[features])

kmeans = KMeans(n_clusters=5, n_init=30, random_state=42)
k_labels = kmeans.fit_predict(X)
print("KMeans silhouette:", round(silhouette_score(X, k_labels), 3))
```

```
# DBSCAN can mark observations as noise (-1) instead of forcing every row into a cluster.
dbscan = DBSCAN(eps=0.75, min_samples=6)
d_labels = dbscan.fit_predict(X)
mask = d_labels != -1
n_clusters = len(set(d_labels[mask]))
print("DBSCAN clusters:", n_clusters)
print("DBSCAN noise points:", int((~mask).sum()))
if n_clusters >= 2:
    print("DBSCAN silhouette (non-noise only):",
          round(silhouette_score(X[mask], d_labels[mask]), 3))
```

```
out = df.copy()
out["kmeans_cluster"] = k_labels
out["dbscan_cluster"] = d_labels
print(out.groupby("kmeans_cluster")[features].mean().round(1))
```

```
plt.figure(figsize=(6, 4))
plt.scatter(
    df["Annual Income (k$)"],
    df["Spending Score (1-100)"],
    c=k_labels,
    alpha=0.75,
)
plt.xlabel("Annual income (k$)")
plt.ylabel("Spending score")
plt.title("K-means labels in two original features")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(6, 4))
plt.scatter(
    df["Annual Income (k$)"],
    df["Spending Score (1-100)"],
    c=d_labels,
    alpha=0.75,
)
plt.xlabel("Annual income (k$)")
```

```
plt.ylabel("Spending score")
plt.title("DBSCAN labels; noise has label -1")
plt.tight_layout()
plt.show()
```

Part C — PCA compression on handwritten digits.

```
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
```

```
PROJECT_ROOT = Path.cwd().parent
data_dir = PROJECT_ROOT / "all_datasets" / "optical_recognition_dataset"
columns = [f"pixel_{i}" for i in range(64)] + ["label"]
train = pd.read_csv(data_dir / "optdigits.tra", header=None, names=columns)
test = pd.read_csv(data_dir / "optdigits.tes", header=None, names=columns)
```

```
# The dataset ships with an official training file and an official test file.
# Keep the official test file untouched while exploring the compression choice.
X_dev = train.drop(columns="label") / 16.0
y_dev = train["label"]
X_test = test.drop(columns="label") / 16.0
y_test = test["label"]
```

```
X_fit, X_valid, y_fit, y_valid = train_test_split(
    X_dev, y_dev, test_size=0.25, random_state=42, stratify=y_dev
)
```

```
# Development baseline: all 64 features, evaluated on validation data.
baseline = LogisticRegression(max_iter=2000)
baseline.fit(X_fit, y_fit)
baseline_valid = baseline.predict(X_valid)
print(
    "64-feature validation accuracy:",
    round(accuracy_score(y_valid, baseline_valid), 4),
)
```

```
# Compare compression budgets only on development validation data.
for retained_variance in [0.90, 0.95, 0.99]:
    candidate = Pipeline([
        ("pca", PCA(n_components=retained_variance, svd_solver="full")),
        ("model", LogisticRegression(max_iter=2000)),
    ])
    candidate.fit(X_fit, y_fit)
    valid_pred = candidate.predict(X_valid)
    pca = candidate.named_steps["pca"]
    print(
        f"{retained_variance:.0%} variance:",
        "components=", pca.n_components_,
        "validation accuracy=", round(accuracy_score(y_valid, valid_pred), 4),
    )
```

```
# Course policy: 95% retained variance is locked before the official test is opened.
locked_variance = 0.95
final_baseline = LogisticRegression(max_iter=2000)
final_baseline.fit(X_dev, y_dev)

final_model = Pipeline([
    ("pca", PCA(n_components=locked_variance, svd_solver="full")),
    ("model", LogisticRegression(max_iter=2000)),
])
```

```
final_model.fit(X_dev, y_dev)

baseline_test_pred = final_baseline.predict(X_test)
pca_test_pred = final_model.predict(X_test)
pca = final_model.named_steps["pca"]

print("\nFINAL LOCKED TEST")
print("full 64-feature accuracy:", round(accuracy_score(y_test, baseline_test_pred), 4))
print("PCA components kept:", pca.n_components_)
print("PCA variance retained:", round(pca.explained_variance_ratio_.sum(), 4))
print("PCA-to-logistic accuracy:", round(accuracy_score(y_test, pca_test_pred), 4))
```

```
plt.figure(figsize=(6, 4))
plt.plot(np.cumsum(pca.explained_variance_ratio_), marker=".")
plt.axhline(locked_variance, linestyle="--")
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance")
plt.title("Locked PCA compression budget")
plt.tight_layout()
plt.show()
```

```
# This visualization is descriptive after the procedure is locked.
X_test_pca = pca.transform(X_test)
plt.figure(figsize=(6, 4))
scatter = plt.scatter(X_test_pca[:, 0], X_test_pca[:, 1], c=y_test, s=12, alpha=0.7)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Official test digits in the first two locked components")
plt.colorbar(scatter, label="digit")
plt.tight_layout()
plt.show()
```

Lab deliverable

Submit one defensible unsupervised question, K-means and DBSCAN diagnostics with interpretation, and the PCA compression table showing components retained and downstream validation performance. Open the official digit test set only after the compression rule is locked.

Check your understanding

1. What does PCA optimize when it chooses its first principal component?
2. Why can standardization change which PCA directions appear most important?
3. Why should distances and apparent cluster sizes in a t-SNE or UMAP plot not be interpreted as exact properties of the original high-dimensional data?

Chapter recap

- Dimensionality reduction replaces many features with fewer informative coordinates.
- PCA finds orthogonal directions that capture maximum variance.
- Explained-variance ratios help choose how many components to retain.
- PCA is linear and may not preserve class separation or nonlinear structure.
- t-SNE and UMAP are mainly visualization tools; apparent cluster sizes and distances require caution.

Anomaly Detection

Chapter at a glance

Before you start: Chapter 15, distributions and quantiles from Chapter 5, plus clustering and density ideas from Chapter 16.

By the end, you can: flag unusual observations with statistical rules, local density, and isolation forests; distinguish outlier from novelty detection; choose operational thresholds; and evaluate ranked anomalies responsibly.

This chapter completes the core unsupervised sequence with **anomaly detection**: ranking observations that do not fit the reference pattern — fraudulent transactions, failing sensors, intrusions, or data-entry errors.

Core path

An anomaly detector is an alarm system, not a judge. It ranks cases by how unusual they appear so a person or later process can investigate them.

18.1 Anomaly detection

Two closely related setups share the name:

Setup	Training data	Goal
Outlier detection	The training data itself is contaminated with anomalies.	Find the anomalous observations already present in the data.
Novelty detection	The training data contains clean “normal” behavior.	Flag future observations that deviate from the learned normal pattern.

Labels are scarce almost by definition — anomalies are rare, expensive to confirm, and unlike one another — so most methods learn what *normal* looks like and score each observation by how far it strays.

18.1.1 Statistical rules: z-scores and IQR fences

For a single roughly bell-shaped feature, the **z-score** measures how many standard deviations an observation sits from the mean:

$$z = \frac{x - \bar{x}}{s},$$

with $|z| > 3$ a customary alarm threshold — under a normal distribution, only about 0.3% of values land that far out.

Worked example

A temperature sensor averages $\bar{x} = 50$ with standard deviation $s = 5$. A reading of 68 has $z = (68 - 50)/5 = 3.6$: flagged. A reading of 57 has $z = 1.4$: unremarkable.

The z-score has an ironic weakness: the anomalies it hunts inflate the very mean and standard deviation it

depends on, hiding themselves. Robust variants substitute the median for the mean and a scaled median absolute deviation for s . The same robustness motivates the **IQR fences** from Chapter 5: with quartiles Q_1 and Q_3 and $\text{IQR} = Q_3 - Q_1$, flag anything outside

$$[Q_1 - 1.5\text{IQR}, Q_3 + 1.5\text{IQR}],$$

the rule behind the whiskers of a box plot. If delivery times have $Q_1 = 20$ and $Q_3 = 30$ minutes, the fences sit at 5 and 45 minutes, and a 60-minute delivery is flagged — no distributional assumptions required.

Per-feature rules are the right first tool, but they miss *combinations*: a heart rate of 150 is normal, “asleep” is normal, and the two together are an emergency. Multivariate methods exist for exactly this.

18.1.2 Distance and local density

A direct multivariate score is the distance to an observation’s k -th nearest neighbor: normal points sit in crowds, anomalies sit alone. Its refinement, the **local outlier factor (LOF)**, fixes a subtle failure of global distance. In a dataset containing one tight cluster and one naturally diffuse cluster, every member of the diffuse cluster looks “far from neighbors” by a global yardstick. LOF instead compares each point’s local density with the densities around its own neighbors: a score near 1 means “about as crowded as my neighborhood expects,” while scores well above 1 mean “markedly emptier around me than around my neighbors” — an outlier *relative to its local context*.

18.1.3 Model-based scores

Any density estimate can support anomaly scoring by thresholding likelihood: fit a Gaussian or a Gaussian mixture (Chapter 16) to reference data and flag observations the model considers improbable. The **one-class SVM** adapts Chapter 14’s machinery to the same end, learning a frontier around the bulk of the data and flagging observations outside it.

18.1.4 Isolation forests

The isolation forest inverts the usual logic: instead of modeling normal points, it measures how easy each point is to *isolate*. Build many random trees, each splitting on a random feature at a random threshold between the observed minimum and maximum, and record the depth at which each observation ends up alone.

Plain idea

Play twenty questions against the dataset. A point deep inside the crowd needs many random questions before it stands alone — every split keeps plenty of company. A point far outside needs only a couple: “is $x_1 > 90$?” and it is already by itself. Average that question-count over many random trees; a short normalized path length drives a more anomalous score.

Isolation forests need no distance function and can be an efficient general-purpose baseline. Their **contamination** setting calibrates the score threshold to an assumed or operational anomaly fraction; it does not discover the true prevalence.

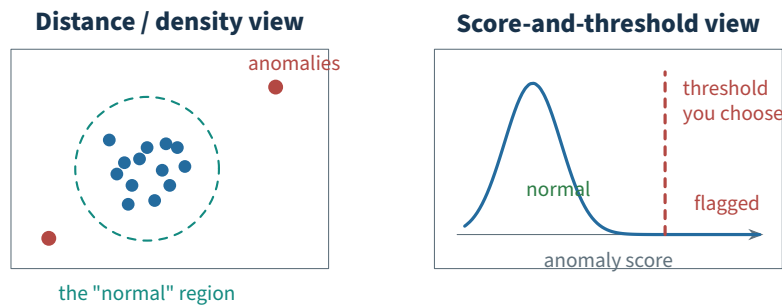
18.1.5 Evaluating and using anomaly detectors

- With few or no labels, evaluation leans on ranked review: send the top-scored cases to a human and measure precision among the top k ; when some labels exist, precision-recall analysis (Chapter 7) is usually more useful than accuracy under extreme imbalance.
- For distance- and kernel-based detectors, apply the scaling rule from Section 3.11.
- Choose thresholds from consequences — how many alarms per day can be investigated? — not from statistical custom.

- An anomaly is not automatically an error. A rare observation may be invalid, or it may be the most important case in the dataset: the fraud, the failing part, the discovery. Investigate before deleting.

18.2 The shape of an anomaly problem

Anomaly detection differs from classification in one important way: you usually cannot see many examples of the thing you are looking for.



Key idea

Every anomaly detector produces a *score*, not a verdict. Turning that score into a decision requires a threshold, and the threshold is a business choice: how many false alarms can your team investigate per day? A detector that flags 10 000 transactions is useless if only 50 can be reviewed.

Analogy

Classification is being shown a thousand photographs of cats and a thousand of dogs and learning to tell them apart. Anomaly detection is being shown a thousand ordinary days at a shop and asked to notice when something is off — without ever having been shown a burglary. You learn what normal looks like and treat anything unlike it as suspicious.

18.3 From anomaly score to investigation

The technical model is only half of an anomaly system. The other half is the review process that consumes its ranked output.

Question	Practical consequence
How many cases can be reviewed?	Choose a top- k or score threshold that respects review capacity.
What happens after a flag?	Define whether the case is blocked, queued, inspected, or merely logged.
Which false alert is costly?	Track precision among reviewed cases and the burden on users or analysts.
Can normal behavior change?	Monitor score distributions and retrain or recalibrate when the reference population drifts.
Can labels arrive later?	Use confirmed outcomes to evaluate ranking quality and eventually consider a supervised model.

Anomaly Detection - Quick Guide

Anomaly detection asks	Which observations are unusually inconsistent with a reference pattern?
Useful methods	Robust statistical rules, distance/local-density methods, isolation forest, and one-class models.
Output	An anomaly score or ranking; a later policy converts that score into an alert.
Evaluation	Precision among reviewed cases, PR AUC when labels exist, stability, and downstream review value.
Main risks	Rare does not always mean harmful; distribution shift can make yesterday's normality model obsolete.

Check your understanding

1. Why is an anomaly score not automatically a probability of fraud, failure, or error?
2. How does a fixed review capacity turn anomaly detection into a ranking problem?
3. What is the difference between detecting outliers in the training sample and novelty detection for future observations?

Chapter recap

- Anomaly detection assigns an unusualness score rather than discovering a guaranteed error or fraud case.
- Robust rules use medians, IQRs, distances, local density, or isolation behavior.
- Evaluation requires labeled cases when available and careful review of false alerts.

Stage 5

Advanced Concepts

Semi-Supervised and Active Learning

Chapter at a glance

Before you start: Supervised learning, validation, classification probabilities, and similarity measures.

By the end, you can: explain why unlabeled data may help, follow pseudo-labeling and graph-based methods, describe active learning, and recognize when these methods can reinforce mistakes.

Stages 3 and 4 showed the two clean endpoints: labels for the supervised branch and no supplied labels for the unsupervised branch. Real projects often sit between them. In ordinary supervised learning, every training row has a label. In many real projects, raw examples are plentiful but labels are expensive. A doctor may need to label scans, an engineer may need to inspect machine failures, or a legal expert may need to categorize documents.

Semi-supervised learning uses a small labeled dataset together with a larger unlabeled dataset. **Active learning** asks a human to label the examples that would be most useful.

Core path

Think of a box of photographs where only a few have names written on the back. Semi-supervised learning uses the named photos for meaning and the unnamed photos for structure. Active learning chooses which unnamed photo would be most useful to ask a person about next.

19.1 The data setting

Let

$$\mathcal{D}_L = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n_L}$$

be the labeled data and

$$\mathcal{D}_U = \{\mathbf{x}_j\}_{j=1}^{n_U}$$

be the unlabeled data, usually with $n_U \gg n_L$.

Unlabeled data can show where observations are concentrated, which examples are similar, and where natural groups may exist. It cannot by itself reveal the meaning of a class label.

Plain idea

Imagine a map with only a few towns named. The unnamed dots still show where settlements are clustered. The labeled towns provide meaning; the unlabeled dots provide structure.

19.2 When unlabeled data can help

Semi-supervised methods rely on assumptions such as:

- nearby observations are likely to share a label;
- observations in the same dense region are likely to share a label;

- the decision boundary should pass through low-density areas rather than cutting through a cluster.

These assumptions may be wrong. Two nearby customers can have different outcomes, or a cluster can contain several classes. Unlabeled data helps only when its structure relates to the label.

19.3 Self-training and pseudo-labeling

Self-training is the simplest semi-supervised approach.

Step 1: Train a supervised model on the labeled data.

Step 2: Predict labels and probabilities for the unlabeled data.

Step 3: Keep only predictions with confidence above a strict threshold.

Step 4: Add those pseudo-labeled examples to the training set, usually with lower weight.

Step 5: Retrain the model and repeat for a limited number of rounds.

Worked example

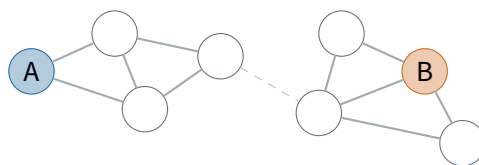
A classifier has 1,000 labeled messages and 20,000 unlabeled messages. It assigns 1,500 unlabeled messages a predicted class probability above 0.98. Those high-confidence predictions are added as temporary labels. The model is retrained, but final evaluation still uses a genuinely labeled validation and test set.

Common mistake

A confidently wrong model can teach itself the wrong answer. Pseudo-labeling should use conservative thresholds, class-balance checks, limited rounds, and validation against human labels.

19.4 Graph-based label propagation

A graph-based method treats each observation as a node. Similar observations are joined by stronger edges. Labels then spread from labeled nodes to nearby unlabeled nodes.



This works well when similarity is meaningful and clusters line up with classes. It can fail when the graph connects unlike observations or when feature scaling is poor.

19.5 Co-training

Co-training uses two different views of the same observation. For example, a webpage may have text content and link information. Two models are trained separately; each adds high-confidence labels for the other.

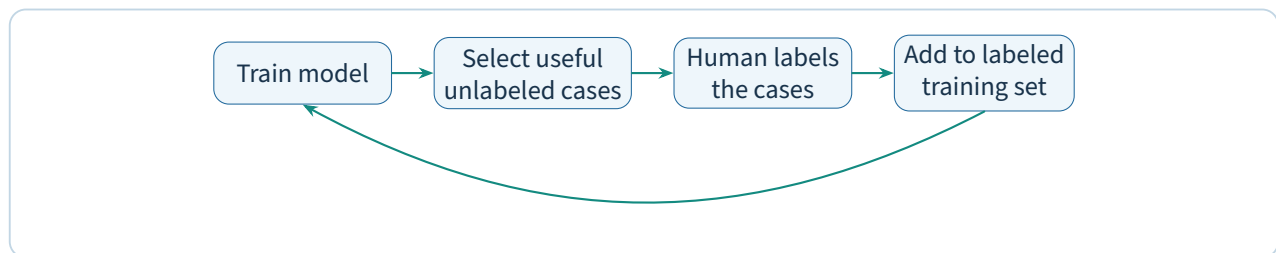
The method is most plausible when both views contain useful information and make partly independent errors. It is not appropriate when the two views are nearly identical.

19.6 Active learning

Active learning spends a labeling budget carefully. Instead of labeling random examples, the model selects examples expected to teach it the most.

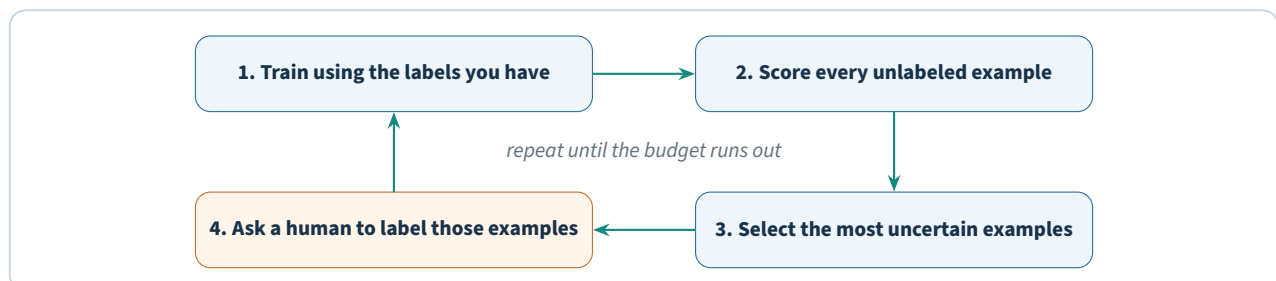
Common selection strategies include:

Selection strategy	Which examples it chooses
Uncertainty sampling	Examples whose predicted probabilities are closest to a tie.
Query by disagreement	Examples on which several models disagree.
Diversity sampling	Uncertain examples that also cover different regions of the feature space.
Expected impact	Examples whose labels are estimated to improve the model most or reduce decision cost.



19.7 The active learning loop, drawn

Active learning inverts the usual arrangement. Instead of being handed labels, the model asks for the ones it most needs.



Analogy

A student revising for an exam has limited time. The inefficient approach is to reread the whole textbook. The efficient approach is to attempt practice questions, notice which topics produce hesitation, and study only those. Active learning is a model doing exactly that: spending its scarce labeling budget on the examples that confuse it most.

19.8 A safe practical workflow

Step 1: Create fixed, human-labeled validation and test sets before using unlabeled data.

Step 2: Train a supervised baseline using only the labeled training set.

Step 3: Inspect whether similarity or cluster structure appears connected to the label.

Step 4: Add one semi-supervised or active-learning method at a time.

Step 5: Compare against the labeled-only baseline using the same evaluation protocol.

Step 6: Review performance by class and subgroup, not only overall average performance.

Step 7: Keep a record of which labels came from humans and which came from the model.

Semi-Supervised and Active Learning - Quick Guide

Best when	Human labels are expensive, unlabeled examples are plentiful, and feature-space structure relates to the label.
Start with	A supervised baseline and a fixed human-labeled validation/test set.
Main controls	Confidence threshold, number of pseudo-labeling rounds, graph similarity, labeling budget, query strategy.
Main risks	Confirmation bias, class imbalance, poor similarity measures, sampling only unusual cases, changing labeling standards.

Key idea

Unlabeled data provides structure, not truth. Human labels remain the source of meaning and the basis of fair evaluation.

PRACTICAL MILESTONE

Lab 9 of 10

Anomaly detection and limited-label learning

Learn when labels are absent, rare, or expensive

Coverage. Chapters 18–19.

Source files. Lab09/Task01.ipynb, Task02.ipynb.

Core data. Credit-card transactions and Breast Cancer Wisconsin.

Goal. Evaluate an unsupervised anomaly ranking with labels held back from fitting, then simulate self-training and active learning under a fixed labeling budget.

How much is scaffolded?

The common theme is label scarcity. In the anomaly activity, labels are unavailable to the fit but available for evaluation; in active learning, labels can be purchased selectively and must not be replaced by repeated test-set inspection.

Part A — Isolation Forest anomaly ranking.

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.model_selection import train_test_split
```

```
PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "credit_card_fraud_dataset" / "credit_card_fraud_10k.csv"
df = pd.read_csv(train_path)
```

```
# Use only behavior/risk features. The transaction ID is an identifier, not a signal.
feature_cols = [
    "amount",
    "transaction_hour",
    "foreign_transaction",
    "location_mismatch",
    "device_trust_score",
    "velocity_last_24h",
    "cardholder_age",
]
X = df[feature_cols]
y = df["is_fraud"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# The fraud labels are deliberately NOT passed to fit.
model = IsolationForest(
    n_estimators=300, contamination="auto", random_state=42, n_jobs=-1
)
model.fit(X_train)
anomaly_score = -model.decision_function(X_test)

prevalence = y_test.mean()
ap = average_precision_score(y_test, anomaly_score)
print("fraud prevalence:", round(prevalence, 4))
print("ROC AUC:", round(roc_auc_score(y_test, anomaly_score), 4))
```

```
print("average precision:", round(ap, 4))
print("AP / prevalence:", round(ap / prevalence, 2))
```

```
ranked = X_test.copy()
ranked["fraud_label"] = y_test.to_numpy()
ranked["anomaly_score"] = anomaly_score
ranked = ranked.sort_values("anomaly_score", ascending=False)
review_budget = min(100, len(ranked))
top_k_precision = ranked.head(review_budget)["fraud_label"].mean()
print(f"precision among top {review_budget}:", round(top_k_precision, 4))
print(ranked[["fraud_label", "anomaly_score", "amount"]].head(20))
```

```
plt.figure(figsize=(6, 4))
plt.hist(anomaly_score[y_test.to_numpy() == 0], bins=30, alpha=0.65, label="non-fraud")
plt.hist(anomaly_score[y_test.to_numpy() == 1], bins=30, alpha=0.65, label="fraud")
plt.xlabel("Anomaly score")
plt.ylabel("Transactions")
plt.title("Isolation Forest score distribution")
plt.legend()
plt.tight_layout()
plt.show()
```

Part B — Self-training and active learning.

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.semi_supervised import SelfTrainingClassifier
```

```
rng = np.random.default_rng(42)
rng_random = np.random.default_rng(123)

PROJECT_ROOT = Path.cwd().parent
train_path = PROJECT_ROOT / "all_datasets" / "breast_cancer_dataset" / "data.csv"
df = pd.read_csv(train_path)

X = df.drop(columns=["id", "diagnosis", "Unnamed: 32"], errors="ignore")
y = (df["diagnosis"] == "M").astype(int).to_numpy()
```

```
# The outer test set stays untouched until the label-acquisition policy is finished.
X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
X_pool, X_val, y_pool, y_val = train_test_split(
    X_dev, y_dev, test_size=0.25, random_state=7, stratify=y_dev
)
X_pool = X_pool.reset_index(drop=True)
y_pool = np.asarray(y_pool)
```

```
# ----- Semi-supervised self-training -----
# Pretend only 30% of the pool labels are initially available.
y_partial = y_pool.copy()
hide = rng.random(len(y_partial)) < 0.70
y_partial[hide] = -1

semi = Pipeline([
    ("scale", StandardScaler()),
    ("model", SelfTrainingClassifier(
        LogisticRegression(max_iter=2000), threshold=0.90
    )),
```

```

])
semi.fit(X_pool, y_partial)
print("self-training validation accuracy:",
      round(accuracy_score(y_val, semi.predict(X_val)), 3))

```

```

# ----- Active-learning simulation -----
# Start from the same small class-balanced labeled set for both policies.
class0 = np.where(y_pool == 0)[0]
class1 = np.where(y_pool == 1)[0]
initial = np.concatenate([
    rng.choice(class0, size=20, replace=False),
    rng.choice(class1, size=20, replace=False),
])
active_labeled = set(initial.tolist())
random_labeled = set(initial.tolist())

label_counts = []
active_val_scores = []
random_val_scores = []
query_batch = 15
rounds = 8

def make_model():
    return Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(max_iter=2000)),
    ])

```

```

for round_no in range(rounds):
    active_idx = np.array(sorted(active_labeled))
    random_idx = np.array(sorted(random_labeled))

    active = make_model()
    random_model = make_model()
    active.fit(X_pool.iloc[active_idx], y_pool[active_idx])
    random_model.fit(X_pool.iloc[random_idx], y_pool[random_idx])

    active_score = accuracy_score(y_val, active.predict(X_val))
    random_score = accuracy_score(y_val, random_model.predict(X_val))
    label_counts.append(len(active_labeled))
    active_val_scores.append(active_score)
    random_val_scores.append(random_score)
    print(
        f"round {round_no + 1}: labels={len(active_labeled)}, "
        f"active_val={active_score:.3f}, random_val={random_score:.3f}"
    )

    if round_no == rounds - 1:
        break

    # Uncertainty sampling: ask for labels closest to probability 0.5.
    active_unlabeled = np.array([
        i for i in range(len(X_pool)) if i not in active_labeled
    ])
    prob = active.predict_proba(X_pool.iloc[active_unlabeled])[:, 1]
    uncertainty = np.abs(prob - 0.5)
    active_query = active_unlabeled[np.argsort(uncertainty)[:query_batch]]
    active_labeled.update(active_query.tolist())

    # Same label budget, but query randomly as a baseline policy.
    random_unlabeled = np.array([
        i for i in range(len(X_pool)) if i not in random_labeled
    ])
    random_query = rng_random.choice(
        random_unlabeled, size=query_batch, replace=False
    )
    random_labeled.update(random_query.tolist())

```

```

plt.figure(figsize=(6, 4))
plt.plot(label_counts, active_val_scores, marker="o", label="uncertainty sampling")

```

```
plt.plot(label_counts, random_val_scores, marker="o", label="random querying")
plt.xlabel("Number of labeled pool examples")
plt.ylabel("Validation accuracy")
plt.title("Label efficiency during development")
plt.legend()
plt.tight_layout()
plt.show()
```

```
# The acquisition policy is now locked. Evaluate each final model on test once.
active_final_idx = np.array(sorted(active_labeled))
random_final_idx = np.array(sorted(random_labeled))
active_final = make_model()
random_final = make_model()
active_final.fit(X_pool.iloc[active_final_idx], y_pool[active_final_idx])
random_final.fit(X_pool.iloc[random_final_idx], y_pool[random_final_idx])
print("final active-learning test accuracy:",
      round(accuracy_score(y_test, active_final.predict(X_test)), 3))
print("final random-query test accuracy:",
      round(accuracy_score(y_test, random_final.predict(X_test)), 3))
```

Lab deliverable

Submit fraud prevalence, anomaly-ranking metrics and a top- k review result, then a label-efficiency curve comparing uncertainty sampling with an equal-budget random-query baseline. Finish with one untouched-test evaluation after the acquisition policy is locked.

Check your understanding

1. Why can pseudo-labeling reinforce mistakes even when the model is highly confident?
2. What does an active-learning query strategy try to spend the labeling budget on?
3. Why must final validation and test labels come from a genuine labeling source rather than from the model itself?

Chapter recap

- Semi-supervised learning combines a small labeled set with a larger unlabeled set.
- Pseudo-labeling adds only high-confidence model predictions and must be checked for error reinforcement.
- Graph methods spread labels through similarity connections.
- Active learning asks humans to label examples that are uncertain, disputed, diverse, or high impact.
- Final validation and testing must use genuine human labels.

Reinforcement Learning Basics

Chapter at a glance

Before you start: Probability, expected value, optimization, and the idea of sequential decisions.

By the end, you can: describe the agent-environment loop, calculate a discounted return, interpret state and action values, follow tabular Q-learning, and explain the exploration-exploitation trade-off.

The final learning signal changes more radically. Supervised learning receives labeled examples, unsupervised learning receives no external answer key, and reinforcement learning receives rewards after actions. Reinforcement learning (RL) studies decisions that affect what happens next. Instead of receiving the correct answer for every action, an agent receives rewards and must discover which behavior produces the best long-term outcome.

Core path

Translate the vocabulary before reading the equations: the **agent** is the decision-maker, the **state** is the current situation, an **action** is a choice, and the **reward** is feedback. The difficult part is that a choice can change later situations, so the best immediate reward may not produce the best long-term result.

Beyond the core

This is an optional survey rather than part of the course's classical supervised/unsupervised core. Teach it after the classical core when students need vocabulary for sequential decision problems; omit it when time is limited.

20.1 The agent-environment loop

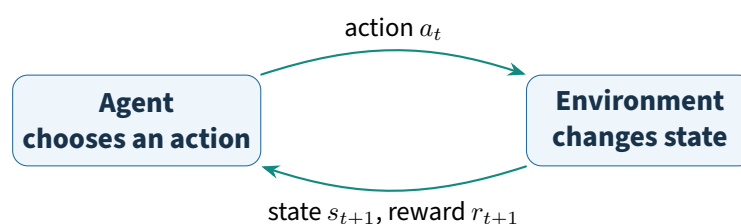
At time t :

Step 1: The agent observes a state s_t .

Step 2: It chooses an action a_t .

Step 3: The environment returns a reward r_{t+1} and a new state s_{t+1} .

Step 4: The agent updates what it believes about the action.



Worked example

A warehouse robot observes its location and remaining battery. It chooses to move, wait, or recharge. It may receive a small cost for time, a larger cost for collisions, and a positive reward for delivering an item. A good policy must consider future consequences, not only the next reward.

20.2 A simpler case: multi-armed bandits

A multi-armed bandit has several actions but no changing state. Each action gives an uncertain reward. The goal is to discover and repeatedly choose the most rewarding action.

This creates the central RL problem:

Exploration Try uncertain actions to learn more about them.

Exploitation Choose the action currently believed to be best.

An ε -greedy strategy explores with probability ε and otherwise exploits.

20.3 Markov decision processes

A classical RL problem is often written as a Markov decision process (MDP) with:

- a set of states $\mathcal{S}$;
- a set of actions $\mathcal{A}$;
- transition probabilities $P(s' \mid s, a)$;
- a reward rule;
- a discount factor $\gamma \in [0, 1)$.

The Markov assumption says the current state contains the information needed to predict the next state distribution. The full distant past does not need to be stored separately.

20.4 Return and discounting

The return from time t is the discounted sum of future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

The discount factor γ controls how much future rewards matter.

- Small γ : favor immediate rewards.
- Large γ : value long-term rewards more strongly.

Worked example

Suppose future rewards are 5, 4, and 3, with $\gamma = 0.9$. Then

$$G_t = 5 + 0.9(4) + 0.9^2(3) = 11.03.$$

The later rewards count, but each is discounted.

20.5 Policies and value functions

A **policy** $\pi(a | s)$ describes how the agent chooses actions in each state.

The state-value function is

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s],$$

the expected return from state s while following policy π .

The action-value function is

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a],$$

the expected return from taking action a in state s and then following π .

Plain idea

$V(s)$ asks, “How good is this situation?” $Q(s, a)$ asks, “How good is this particular move from this situation?”

20.6 The Bellman idea

A long-term value can be broken into two parts:

$$\text{value now} = \text{reward next} + \text{discounted value later.}$$

For an optimal action-value function,

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right].$$

This recursive relationship is the basis of Q-learning.

20.7 Tabular Q-learning

When the state and action spaces are small, store one Q-value for every state-action pair in a table.

After observing transition $(s_t, a_t, r_{t+1}, s_{t+1})$, update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right].$$

The bracketed quantity is the **temporal-difference error**: the difference between the new backup value and the current estimate.

Step 1: Initialize the Q-table, often with zeros.

Step 2: Observe the current state.

Step 3: Choose an action using an exploration strategy such as ϵ -greedy.

Step 4: Perform the action and observe reward and next state.

Step 5: Update the chosen Q-table entry.

Step 6: Repeat over many episodes, gradually reducing exploration when appropriate.

Worked example

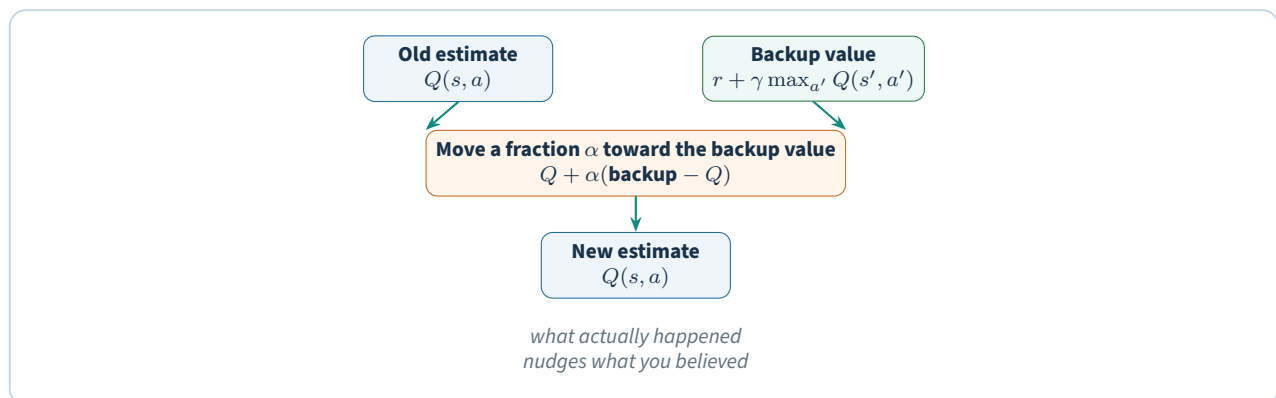
Suppose $Q(s, a) = 2$, reward $r = 1$, $\gamma = 0.9$, the best next-state Q-value is 4, and $\alpha = 0.5$. The backup value is $1 + 0.9(4) = 4.6$. The temporal-difference error is $4.6 - 2 = 2.6$. The update gives

$$Q(s, a) \leftarrow 2 + 0.5(2.6) = 3.3.$$

The estimate moves halfway toward the new backup value.

20.8 One Q-learning update, drawn

The whole of tabular Q-learning is one line of arithmetic applied millions of times. It is worth seeing that line as a picture.

**Analogy**

You believed the journey to school takes 20 minutes. Today it took 26. You do not now believe it takes 26 minutes — one bad day is not the truth — but you do not keep saying 20 either. You settle on something like 21. The learning rate α is how much you let a single experience change your mind, and γ is how much you care about consequences that arrive later rather than immediately.

20.9 Exploration strategies

Exploration strategy	How it encourages exploration
ϵ -greedy	Usually chooses the best-known action, but chooses a random action with probability ϵ .
Optimistic initialization	Starts Q-values high so untried actions look attractive.
Upper confidence methods	Prefer actions with either strong estimated reward or high uncertainty.

Too little exploration can trap the agent in a poor policy. Too much exploration wastes reward and may be unsafe.

20.10 Evaluation and responsible use

RL evaluation is harder than ordinary prediction because the policy changes the data it experiences. Use repeated runs with different random seeds and report both average return and variation.

Important concerns include:

- reward design that accidentally encourages unwanted behavior;
- unsafe exploration in physical, medical, financial, or public systems;
- changing environments that make past experience unreliable;
- delayed rewards that make credit assignment difficult;
- simulation assumptions that do not match the real world.

For high-stakes applications, exploration should occur in a safe simulator or under strict constraints before real-world use.

Classical Reinforcement Learning - Quick Guide

Best when	Actions affect future states and rewards, and interaction or a realistic simulator is available.
Start with	A bandit or small tabular MDP before moving to larger problems.
Main controls	Learning rate α , discount factor γ , exploration rate ϵ , state representation, reward design.
Main risks	Unsafe exploration, reward hacking, unstable estimates, poor state definition, unrealistic simulation.

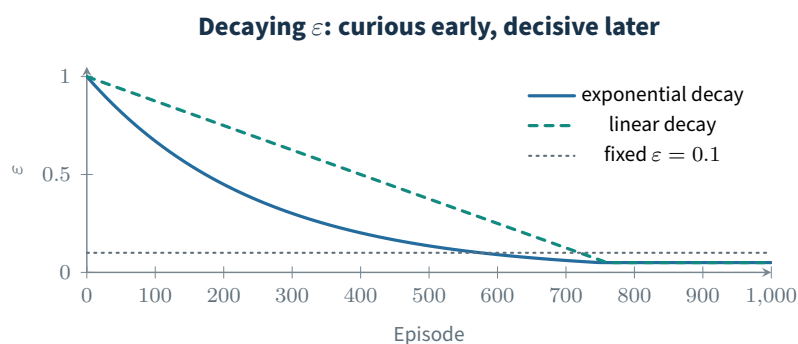
Key idea

Reinforcement learning does not learn from a correct answer for every action. It learns from consequences, balancing immediate reward, future reward, and the need to explore.

20.11 Bandit exploration behavior

Key idea

Look at the pull counts. The agent spent almost all its attempts on the best machine but never stopped sampling the others entirely. A fixed 10% of decisions use a random exploratory action; some of those random actions still choose the current best arm. Continued exploration reduces the risk of settling early on the wrong answer, at the cost of occasionally taking an action believed to be worse.



20.12 Tabular Q-learning dynamics

Symbol	Typical value	What raising it does
alpha	0.01–0.2	Learn faster from each experience, but more erratically
gamma	0.9–0.99	Care more about distant future rewards
epsilon	1.0 down to 0.05	Explore more; too high and the agent never commits
episodes	thousands	More practice; tabular methods need a lot

Common mistake

In a continuing task, an undiscounted infinite sum may be undefined, so the discounted formulation normally uses $\gamma < 1$. Episodic tasks with guaranteed finite termination can permit $\gamma = 1$. A fixed nonzero ϵ deliberately keeps exploring; that may waste reward in a stationary deployment, but can remain useful in a changing environment. Evaluate the exploratory behavior separately from the greedy policy.

Reference note

[gymnasium](#) (the maintained successor to OpenAI Gym) provides ready-made environments with a uniform interface, so you can practice without writing the environment mechanics yourself. For problems beyond small tables — for example, image observations or continuous state spaces — [stable-baselines3](#) provides implementations of standard deep-RL algorithms on top of PyTorch.

Key idea

Tabular Q-learning needs one table cell per state-action pair. For very large or continuous state spaces, the table cannot exist. Replacing it with a function approximator, sometimes a neural network that estimates $Q(s, a)$, leads toward deep reinforcement learning (the Epilogue).

PRACTICAL MILESTONE

Lab 10 of 10

Reinforcement learning and final integration

Learn a policy by interaction, then connect the result to the full course workflow

Coverage. Chapter 20 with a handoff to the capstone.

Source files. Lab10/Task01.ipynb.

Core environment. A self-contained 4×4 GridWorld.

Goal. Train a tabular Q-learning agent, evaluate its greedy policy separately from exploratory training, and articulate how reinforcement learning changes the data/label/evaluation assumptions used earlier in the course.

How much is scaffolded?

This is the tenth and final scheduled lab. The capstone that follows is an integration project, not Lab 11.

Part A — Tabular Q-learning in GridWorld.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
rng = np.random.default_rng(42)

# A tiny 4x4 GridWorld. Start at the top-left, reach the goal at bottom-right,
# and avoid the trap. No external environment package is required.
ROWS, COLS = 4, 4
START = (0, 0)
GOAL = (3, 3)
TRAP = (1, 3)
ACTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up, down, left, right
ACTION_NAMES = ["up", "down", "left", "right"]

Q = np.zeros((ROWS, COLS, len(ACTIONS)), dtype=float)
alpha = 0.15
gamma = 0.95
epsilon = 1.0
min_epsilon = 0.05
```

```
def step(state, action):
    r, c = state
    dr, dc = ACTIONS[action]
    nr = min(max(r + dr, 0), ROWS - 1)
    nc = min(max(c + dc, 0), COLS - 1)
    next_state = (nr, nc)

    if next_state == GOAL:
        return next_state, 1.0, True
    if next_state == TRAP:
        return next_state, -1.0, True
    return next_state, -0.04, False
```

```
def choose_action(state, explore=True):
    if explore and rng.random() < epsilon:
        return int(rng.integers(len(ACTIONS)))
    r, c = state
    return int(np.argmax(Q[r, c]))
```

```

training_returns = []

for episode in range(5000):
    state = START
    done = False
    episode_return = 0.0

    for _ in range(100):
        action = choose_action(state, explore=True)
        next_state, reward, done = step(state, action)
        episode_return += reward

        r, c = state
        nr, nc = next_state
        backup_value = reward if done else reward + gamma * np.max(Q[nr, nc])
        Q[r, c, action] += alpha * (backup_value - Q[r, c, action])
        state = next_state

    if done:
        break

    training_returns.append(episode_return)
    epsilon = max(min_epsilon, epsilon * 0.9985)

```

```

window = 100
moving = np.convolve(training_returns, np.ones(window) / window, mode="valid")
plt.figure(figsize=(6, 4))
plt.plot(np.arange(window - 1, len(training_returns)), moving)
plt.xlabel("Episode")
plt.ylabel("Mean return over last 100 episodes")
plt.title("Q-learning training curve")
plt.tight_layout()
plt.show()

```

```

# Evaluate the learned greedy policy.
successes = 0
returns = []
for _ in range(200):
    state = START
    total_reward = 0.0
    for _ in range(100):
        action = choose_action(state, explore=False)
        state, reward, done = step(state, action)
        total_reward += reward
        if done:
            successes += int(state == GOAL)
            break
    returns.append(total_reward)

print("success rate:", successes / 200)
print("mean return:", round(float(np.mean(returns)), 3))
print("learned action at start:", ACTION_NAMES[choose_action(START, explore=False)])

```

Try it yourself

Change one of reward, trap location, exploration schedule, or discount factor. Predict the behavioral consequence before retraining and compare the learned greedy policy afterward.

Lab deliverable

Submit the reward specification, one pre-training prediction, the learning curve, repeated greedy-policy success rate, and one controlled environment or hyperparameter change. End with a short capstone handoff naming the learning signal, validation boundary, baseline, and metric you would

choose for a new problem.

Check your understanding

1. What are the state, action, reward, and transition in a reinforcement-learning formulation?
2. What two pieces of information appear in the tabular Q-learning backup value?
3. Why should the exploratory training policy be evaluated separately from the final greedy policy?

Chapter recap

- An agent observes a state, takes an action, receives a reward, and moves to a new state.
- Return is the discounted sum of future rewards.
- $V(s)$ values a state; $Q(s, a)$ values an action in a state.
- Tabular Q-learning moves an old estimate toward reward plus discounted best future value.
- Exploration is necessary for learning but must be controlled for safety.

Capstone Project: From Raw Data to Defensible Evidence

The capstone is not a Kaggle-score contest. It is evidence that you can carry an unfamiliar problem from raw files to a defensible conclusion while respecting data quality, information boundaries, validation, and uncertainty.

Course choice

Choose a small public dataset that is not the main dataset from your favorite earlier lab. You may use one of the course transfer datasets or propose another dataset with a manageable download size. Keep the original download in its own folder under `all_datasets/` and keep all analysis in one or more notebooks under `all_experiments/`.

Required capstone sequence

1. **Problem contract.** State the observation unit, label or unsupervised objective, prediction/use moment, population, success metric, and cost of mistakes.
2. **Data provenance and quality assessment.** Document files, columns, units, missingness, duplicates, impossible values, suspicious identifiers, and all cleaning decisions.
3. **Validation design.** Choose random, stratified, grouped, or chronological splitting based on how future data will arrive. Reserve a final test set when the task is supervised.
4. **Baseline.** Establish a no-skill or simple domain baseline before trying complex models.
5. **Leakage-safe preprocessing.** Put learned transformations inside pipelines or inside the training folds of the validation procedure.
6. **Model comparison.** Compare at least three appropriate model families. Include one simple interpretable model.
7. **Model development.** Tune hyperparameters on development data only. Use feature selection or resampling only when justified, and validate the complete procedure.
8. **Visual diagnostics.** Include plots that expose errors, residual structure, thresholds, learning curves, feature behavior, clusters, or score distributions as appropriate.
9. **Error and subgroup analysis.** Inspect where the method fails rather than reporting only an average metric.
10. **Final evaluation.** Evaluate once on untouched supervised test data, or present multiple forms of unsupervised evidence when no answer key exists.
11. **Interpretation limits.** Separate predictive association from causal claims and state what the experiment cannot establish.
12. **Reproducibility.** Record data paths, package requirements, random seeds, notebook order, and the final configuration.

Required learning-signal extension

Add *one* extension that changes the question rather than merely changing the model:

- an unsupervised representation or clustering analysis that complements a supervised capstone;
- an anomaly-ranking analysis with a realistic review threshold;
- a limited-label simulation comparing full supervision with scarce labels;
- or, when the project is naturally sequential, a small reinforcement-learning formulation separate from the static-data analysis.

Do not force reinforcement learning onto a static tabular problem merely to tick a box. The capstone should demonstrate that you know *when* a learning paradigm fits, not that every paradigm can be applied to every dataset.

Key idea

A strong capstone is easy to review and reproduce. A reader should be able to tell what you knew at each stage, which data influenced each decision, why the final method was selected, and where the result may fail.

Submission package

Submit a compact, traceable project package rather than a collection of disconnected screenshots:

- **Notebook(s):** one main `.ipynb` notebook, or a clearly numbered sequence, containing the analysis, plots, results, and interpretation.
- **Short project note:** a concise README or report stating the problem contract, data source, prediction/use moment, primary metric, selected procedure, final conclusion, and main limitations.
- **Reproducibility record:** `requirements.txt`, random seeds, expected dataset paths, and the order in which notebooks should be run.
- **Data provenance:** the public source link, the exact downloaded files used, and any deterministic cleaning rules. Follow the dataset license and course submission rules when deciding whether the raw data itself should be redistributed.
- **Evidence table:** one final table separating development/validation evidence from the single final supervised test result, or separating the different forms of unsupervised evidence when no test labels exist.

Key idea

The capstone should make the information boundary visible. Development results may influence the design; the final supervised test result must not influence development decisions. If the test score makes you want to change the model, record that idea as future work rather than reopening model selection.

Epilogue: Where to Go Next

Chapter at a glance

Before you start: Everything in this book. Nothing new is required.

By the end, you can: see where classical machine learning sits in the wider field, understand what deep learning adds and why, choose a next direction that matches your interests, and follow a concrete twelve-month plan.

You now know something real. You can take a messy table, prepare it without leaking, choose an appropriate model, evaluate it honestly, and explain what it does and where it fails. That is the foundation, and a surprising number of working practitioners do exactly this and nothing more.

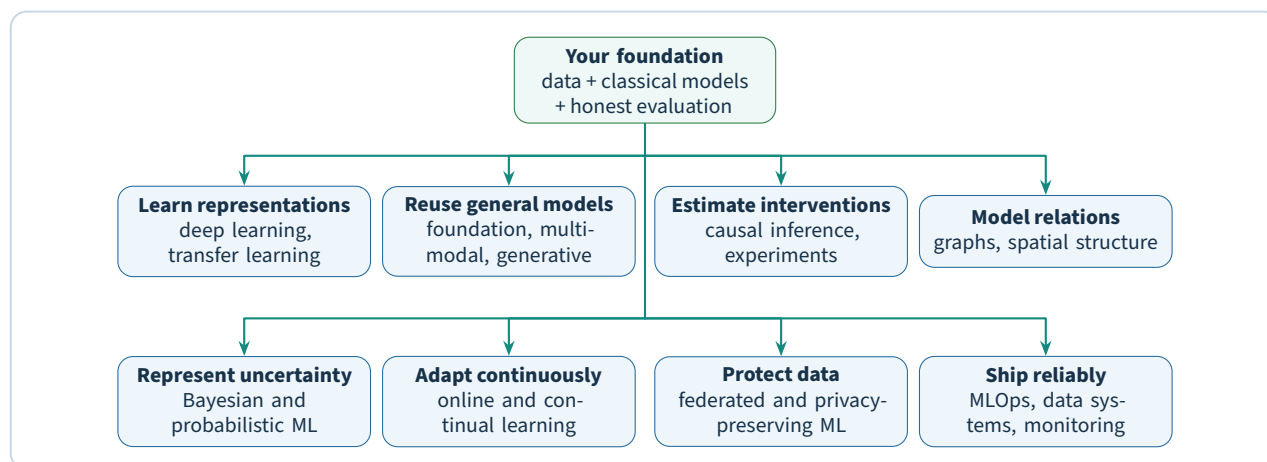
This epilogue is a map of what lies beyond it.

Key idea

Many ideas ahead extend concepts you have already met. Neural networks optimize a larger parameterized function; transformers add attention-based architectures; deep reinforcement learning replaces small tables with function approximators. You are extending the foundation, not starting again.

E.1 The map of the field

The opening map organized machine learning by feedback, task, data structure, model family, and lifecycle. At the end of the book, use the same map in the opposite direction: identify the capability your current toolkit is missing, then learn the branch that supplies it.



If your next problem says...	Learn next
The useful features are hidden inside pixels, audio, or long text.	Deep learning and representation learning.
I want to adapt a model that already knows a lot.	Transfer learning and foundation models.
I need to generate text, images, audio, or other samples.	Generative modeling: autoregressive, diffusion, latent-variable, and related models.
My examples are connected by edges or locations.	Graph ML, network methods, and spatial statistics.
I need probabilities and uncertainty, not only point predictions.	Probabilistic and Bayesian modeling, calibration, conformal and interval methods.
I need to know what an action will change.	Causal inference, experimental design, uplift / treatment-effect modeling.
Data arrive continuously and the world changes.	Online, streaming, continual, and adaptive learning.
The data cannot be centralized freely.	Privacy-preserving, federated, and secure learning.
I want the system to automate model search.	AutoML and optimization — but keep the same leakage-safe evaluation discipline.

Key idea

Foundation models, RAG, tool use, and agents belong at different layers. A foundation model is a learned model; RAG adds retrieval; tool use adds external actions; an agent adds a control loop around model calls and tools. Keeping those layers separate makes systems easier to evaluate and debug.

E.2 Optional optimization depth

Beyond the core

This section is a second-pass reference. It is not required before the classical model chapters. Return here when you study neural networks or when optimization behavior itself becomes the problem you need to diagnose.

E.2.1 Gradient descent mechanics

Training often means finding parameters θ that minimize a loss function $J(\theta)$. Gradient descent updates one parameter according to

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}.$$

Read this as: “new setting equals old setting minus step size times slope.” The derivative supplies the slope and therefore the direction; the learning rate decides how boldly to move.

The learning rate α controls the step size. A value that is too small learns slowly; a value that is too large can make training unstable.

Worked example

Minimize $J(w) = (w - 4)^2$, whose answer is obviously $w = 4$. The derivative is $J'(w) = 2(w - 4)$, so the update rule is

$$w \leftarrow w - \alpha \cdot 2(w - 4).$$

Start at $w = 0$ every time and change only α .

$\alpha = 0.1$ (**sensible**). The rule simplifies to $w \leftarrow 0.8w + 0.8$:

$$0 \rightarrow 0.80 \rightarrow 1.44 \rightarrow 1.95 \rightarrow 2.36 \rightarrow \dots \rightarrow 4.$$

Check the first step by hand: the gradient at $w = 0$ is $2(0 - 4) = -8$, and $w = 0 - 0.1(-8) = 0.8$. Each step closes 20% of the remaining distance, so after n steps $w = 4 - 4(0.8)^n$. Steady, and it never overshoots.

$\alpha = 0.6$ (**too large, but survivable**). Now $w \leftarrow -0.2w + 4.8$:

$$0 \rightarrow 4.80 \rightarrow 3.84 \rightarrow 4.03 \rightarrow 3.99 \rightarrow \dots \rightarrow 4.$$

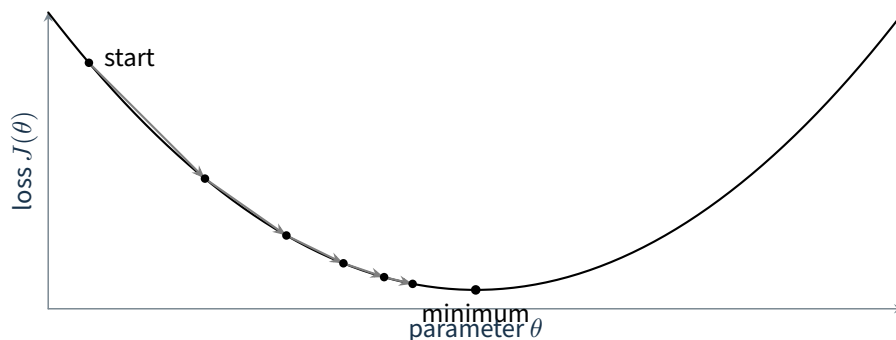
It jumps straight past the answer on step one and then bounces from side to side, closing in. This still works, but you would see the loss zig-zag rather than fall smoothly.

$\alpha = 1.1$ (**too large**). Now $w \leftarrow -1.2w + 8.8$:

$$0 \rightarrow 8.80 \rightarrow -1.76 \rightarrow 10.91 \rightarrow -4.29 \rightarrow \dots$$

Each overshoot is larger than the last. The multiplier on w is -1.2 , and because $|-1.2| > 1$ the distance from the answer grows by 20% every step. This is divergence, and no amount of extra epochs will rescue it.

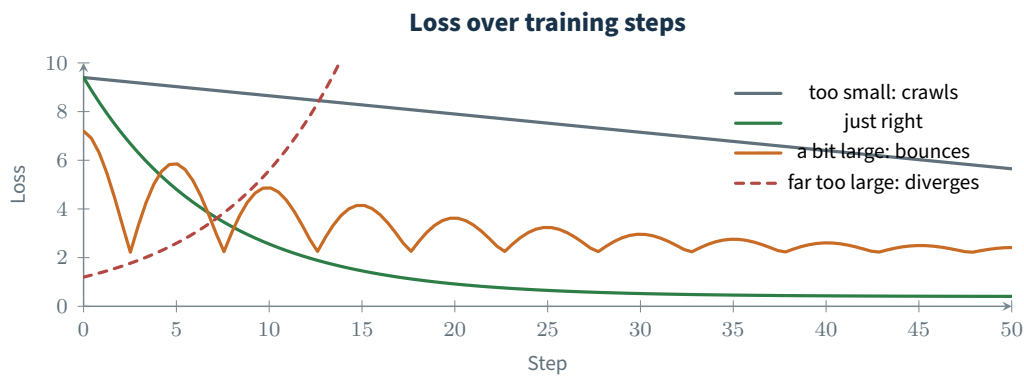
The lesson generalizes: for this loss, any $\alpha < 1$ converges, α between 0.5 and 1 oscillates on the way, and $\alpha > 1$ diverges. Real losses do not hand you that number, which is why the learning rate is tuned.

**Plain idea**

Picture standing on a foggy hillside and wanting to reach the valley floor. You cannot see far, but you can feel which way the ground slopes under your feet. Gradient descent takes a small step downhill, checks the slope again, and repeats. The learning rate is your stride length: too short and the walk takes forever, too long and you overshoot the valley and stumble up the far side.

E.2.2 Learning rate: three outcomes

The learning rate is the single most important number in gradient-based training, and its three failure modes are easy to recognize once you have seen them.



Analogy

Learning rate is stride length when walking downhill in fog. Tiny steps get you there eventually, if you have all day. Sensible steps get you there quickly. Steps that are slightly too big have you bouncing between the two sides of the valley, never quite reaching the bottom. Steps that are far too big launch you over the opposite ridge and further from the valley than when you started.

Key idea

A rapidly rising or non-finite loss often indicates an excessive learning rate, but it can also reveal unscaled features, bad gradients, numerical overflow, or corrupt data. Lowering the rate is a useful diagnostic; inspect inputs and implementation as well.

E.2.3 Gradient descent, written out

Three learning rates, three completely different stories, one line of arithmetic. Now the same idea on real data with many parameters:

Implementing it once removes the mystery permanently.

E.2.4 Batch, stochastic, and mini-batch

Variant	Rows per update	Character
Batch	All of them	Smooth, slow, needs all data in memory
Stochastic	One	Very noisy, very fast per step, can escape shallow traps
Mini-batch	32–512	Common for large-scale and neural optimization; size is a tunable trade-off

E.2.5 Momentum and adaptive optimization

Momentum accumulates a velocity,

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla J(\boldsymbol{\theta}_t), \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \mathbf{v}_t.$$

It smooths oscillations and accelerates progress when gradients point consistently in one direction.

Adaptive methods such as Adam maintain moving averages of both the gradient and its elementwise square. With gradient $\mathbf{g}_t = \nabla J(\boldsymbol{\theta}_t)$,

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2,$$

followed by bias correction and the parameter update

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Typical defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. Dividing by $\sqrt{\hat{v}_t}$ gives each parameter its own effective step size: parameters with consistently large gradients take smaller steps, and rarely updated parameters take larger ones.

Plain idea

Adam is momentum plus a personal speed limit for every parameter. The first average (m) remembers which direction the gradient has been pushing, like a rolling ball. The second average (v) remembers how violent the gradient has been, and slows down exactly the parameters that have been jumpy while letting quiet ones move faster.

Worked example

Do the very first Adam step by hand. Use the defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\alpha = 0.001$, $\epsilon = 10^{-8}$, starting from $m_0 = v_0 = 0$, with a gradient of $g_1 = 0.5$.

Step 1: $m_1 = 0.9(0) + 0.1(0.5) = 0.05$ and $v_1 = 0.999(0) + 0.001(0.5)^2 = 0.00025$.

Step 2: Bias-correct. At $t = 1$ the divisors are $1 - \beta_1^1 = 0.1$ and $1 - \beta_2^1 = 0.001$, giving $\hat{m}_1 = 0.05/0.1 = 0.5$ and $\hat{v}_1 = 0.00025/0.001 = 0.25$.

Step 3: Step: $\alpha \hat{m}_1 / (\sqrt{\hat{v}_1} + \epsilon) = 0.001 \times 0.5 / 0.5 = 0.001$.

Now repeat the whole thing with a gradient a hundred times larger, $g_1 = 50$. Then $m_1 = 5$, $v_1 = 2.5$, $\hat{m}_1 = 50$, $\hat{v}_1 = 2500$, and the step is $0.001 \times 50 / 50 = 0.001$ — *identical*.

That is the point of the $\sqrt{\hat{v}}$ divisor, and it explains why Adam is so forgiving in practice. Plain gradient descent with $\alpha = 0.001$ would have taken a step of 0.0005 in the first case and 0.05 in the second, so the same learning rate would be far too timid for one problem and far too aggressive for the other. Adam normalizes the gradient by its own recent size, so the first step is α whatever the units of your loss happen to be.

The flip side is worth knowing: because the step size is roughly α regardless of the gradient, Adam does not slow down near a minimum the way plain gradient descent does. That is why a learning-rate schedule still matters.

Adam is convenient and fast, but optimizer defaults are not universally optimal. The optimizer, learning rate, weight decay, and batch size should be treated as interacting choices.

E.2.6 Learning rates and schedules

The learning rate is often the most important optimization hyperparameter. Warning signs include:

- loss decreases extremely slowly: the rate may be too small;
- loss oscillates or diverges: the rate may be too large;
- early progress stops: a lower rate may be needed later in training.

Schedules may reduce the rate after a fixed number of epochs, after validation performance plateaus, according to cosine decay, or through warm-up followed by decay. Learning-rate curves should be recorded with the experiment.

Second pass

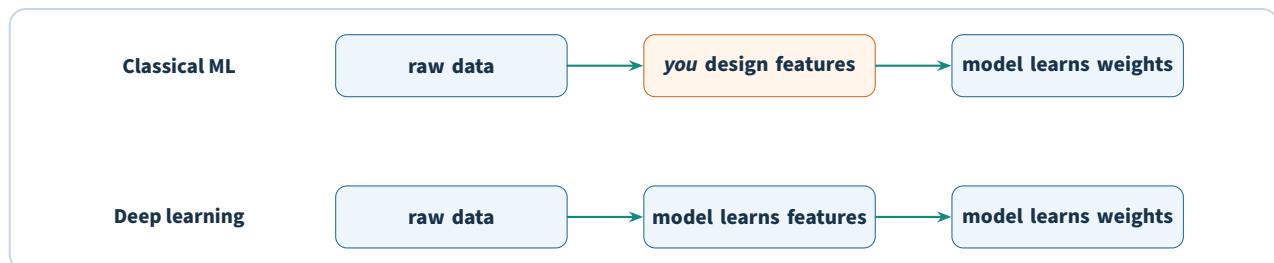
Optional optimizer check: explain why a learning rate can be too small or too large, why momentum can smooth noisy updates, and why Adam's adaptive normalization does not remove the need to monitor or schedule the learning rate.

E.3 Deep learning: what it adds and when you need it

A basic feed-forward neural network composes affine transformations with nonlinear activations. Modern systems add specialized architectures, normalization, residual connections, attention, and large-scale optimization. Their key practical advantage on many unstructured inputs is representation learning: useful features can be learned jointly with the predictor.

Beyond the core

Deep learning, transfer learning, and language models are orientation topics here. They are not required for the classical machine-learning course assessment.

**Analogy**

Classical image systems often relied on designed features such as edges, color histograms, and corners. A convolutional network can learn increasingly task-useful representations from pixels, although what an internal layer represents varies by architecture, data, and training objective.

E.3.1 When deep learning is worth it, and when it is not

Reach for deep learning when	Stay with classical ML when
The input is images, audio, video, or long text	The data is a table of rows and columns
You have tens of thousands of examples or more	You have a few thousand rows
Useful features are hard to describe in words	The features are already meaningful
A pre-trained model exists that you can fine-tune	You must explain every decision to a regulator
Accuracy justifies the compute and complexity	A gradient-boosted tree already works

Key idea

On small and medium tabular datasets, linear models and tree ensembles remain strong baselines and may be faster, easier to validate, and easier to explain than a neural network. Benchmark under the same split, metric, and compute budget; novelty is not evidence of suitability.

E.3.2 Your first neural network

Plain idea

Compare that optimization loop with the gradient-descent idea introduced in Section 4.4. It repeats the same four moves: predict, measure the error, compute derivatives, and update parameters. Automatic differentiation and the network architecture add important machinery, but the optimization pattern is familiar.

E.3.3 The architectures worth knowing by name

Architecture	Built for	Core idea
MLP	Tables, simple inputs	Stacked linear layers with non-linearities
CNN	Images	The same small filter slides across the whole picture
RNN / LSTM	Sequences (older)	Carry a memory forward through time
Transformer	Text, and now everything	Attention: every position looks at every other
Autoencoder	Compression, anomalies	Squeeze the input down, then rebuild it
GAN / Diffusion	Generating images	Learn to produce data that looks real
GNN	Networks and graphs	Pass messages between connected nodes

E.3.4 Transfer learning: the shortcut that makes it practical

Key idea

Transfer learning is the reason a student with a laptop can build a good text or image classifier. Someone else spent millions of pounds teaching a model what language or vision looks like in general; you spend an afternoon teaching it your specific task. Almost nobody trains a large model from scratch, and you should not plan to either.

E.4 Large language models, briefly and honestly

Large language models are usually transformer-based models pretrained with token-prediction objectives, then adapted with one or more post-training methods such as supervised instruction data, preference optimization, or reinforcement-style objectives. They are the most visible part of the field, and one of the easiest places to confuse a learned model with the larger system built around it.

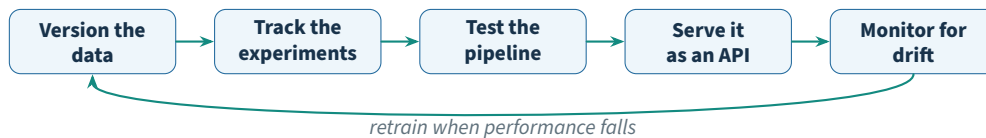
Skill	What it involves
Prompting	Writing instructions that reliably produce what you want
RAG	Retrieving your own documents and giving them to the model as context
Fine-tuning	Adapting a model to a specific style or task with your data
Evaluation	Deciding whether the output is actually good — the hard part
Agents	Building a control loop that lets a model plan or choose tool calls, while the surrounding system checks state, permissions, and outcomes

Common mistake

A language model produces fluent text, not guaranteed truth. It can generate unsupported citations, statistics, and function names. Serious applications therefore need task-specific evaluation, retrieval or verification where appropriate, and human oversight for consequential use — the evaluation discipline from Chapter 7 still applies.

E.5 Getting models into the world

Training a model is perhaps a fifth of a real project. The rest is engineering.



Operational handoff field	What to record
Decision and population	Who is eligible, when scoring occurs, what action follows, and any capacity limit.
Label and timing	Exact outcome definition, prediction horizon, feature cutoff, and label delay.
Validation evidence	Split design, baseline, primary deployment metric, secondary checks, and final untouched result.
Policy	Threshold or ranking rule, deterministic tie handling, exclusions, and expected batch size.
Monitoring	Schema/data-quality checks, drift signals, delayed outcome metrics, subgroup counts, and intervention outcomes where relevant.
Governance	Model/data version, owner, privacy/security controls, known limitations, rollback condition, and causal-status statement.

Tool	Job	Why it matters
<code>git</code>	Version control	Records intentional changes and supports review.
<code>docker</code>	Containers	Packages an application environment consistently.
<code>mlflow</code>	Experiment tracking	Remembers which settings produced which score.
<code>dvc</code>	Data versioning	Git for datasets too large for git.
<code>fastapi</code>	Serving	Turns a model into something other software can call.
<code>pytest</code>	Testing	Catches the pipeline breaking before users do.
<code>evidently</code>	Drift monitoring	One option for detecting changes that warrant investigation.

Key idea

Input or score drift is an alert, not proof that predictive performance fell; stable inputs do not guarantee stable outcomes either. Monitor data quality, eligibility, predictions, actions, and delayed outcomes. Investigate changes, then retrain, recalibrate, revise, or retire the system according to a prespecified response plan.

E.6 Mathematics, if and when you want it

You do not need more mathematics to be useful. You need it to read research papers and to invent methods rather than apply them. In order of return on effort:

#	Topic	What it unlocks
1	Linear algebra	Everything. Matrices, eigenvectors, projections, SVD.
2	Probability	Distributions, expectation, Bayes, likelihood.
3	Calculus	Derivatives, chain rule — and therefore backpropagation.
4	Optimization	Convexity, constraints, why some problems are hard.
5	Statistics	Estimation, hypothesis testing, confidence, experiment design.
6	Information theory	Entropy, cross-entropy, KL divergence.

Plain idea

Learn these when a specific gap blocks you, not in advance. Trying to complete a mathematics curriculum before building anything is the single most common way people stall out. Build first; when a paper defeats you, go and learn the piece you were missing. Motivated learning sticks.

E.7 Responsibility is part of the skill

A model that is accurate can still be harmful. This is not a footnote to the technical work; it is part of it.

Concern	The question to ask
Bias	Does it perform or allocate actions differently across relevant groups? Prespecify lawful, meaningful evaluations and include counts and uncertainty.
Feedback loops	Does acting on the prediction change the data it will be retrained on?
Proxies	Is a feature standing in for something you are not allowed to use?
Consent	Did the people in the training data agree to this use?
Explanation	Can you tell someone why the model decided what it did about them?
Recourse	If it is wrong about someone, how do they get that fixed?

A single diagnostic is only a starting point. Group definitions, data collection, legal basis, privacy, intersectional sample sizes, appropriate metrics, and uncertainty require domain and stakeholder judgment.

Common mistake

Removing a sensitive attribute does not by itself remove inequity: other variables can act as proxies, labels can encode past decisions, and representation can differ across groups. Where lawful and appropriate, protected attributes may be needed for fairness evaluation even when excluded from prediction. No single metric “solves” fairness; examine performance, allocation, harms, and recourse in context.

E.7.1 Project ideas that teach the most

Project	Skills	Why it is a good first choice
Predict something about your own life	Everything	Real data you understand; you can tell when it is wrong
Classify your own photos	CNN, transfer	Small data, fast feedback, visible results
Sort your email or messages	Text, NB, TF-IDF	Cheap labels, immediately useful
Local weather or transport forecast	Time series	Teaches why random splits fail
Cluster a music or film library	Clustering, PCA	No labels needed; interpretation is the challenge
Reproduce a paper's result	Everything, hard	The fastest route from user to practitioner

Key idea

Choose a project where you can recognize a bad answer yourself. On data you understand, a nonsense prediction is obvious and you learn from it. On an unfamiliar benchmark, a nonsense prediction is just a number, and you learn nothing.

Analogy

Learning machine learning is like learning an instrument. Nobody becomes a musician by reading about music theory; they become one by playing badly, listening carefully, and playing again. The theory helps enormously — but only alongside the playing, never instead of it.

Key idea

The field will keep changing, and much of what is fashionable today will look dated in five years. What will not change is the underlying discipline: define the problem, respect the data, hold out an honest test, compare against a baseline, measure what actually matters, and be clear about what your model cannot do. That is what this book was really teaching, and it will still be true when the tools have all been replaced.

Chapter recap

- Classical machine learning provides strong baselines and a durable evaluation discipline.
- Deep learning is especially useful for many image, audio, and text tasks, but its added complexity must earn its place.
- Transfer learning makes deep learning practical without enormous resources.
- Deployment, monitoring, and drift are as important as training.
- Learn mathematics when a specific gap blocks you, not in advance.
- Responsible evaluation considers groups, allocation, privacy, uncertainty, and recourse; deleting a sensitive column does not remove bias.
- Build, evaluate, document, and share in a sequence that suits the project.